



ORION AI

Final Report

Project Name: Invisio

Team Members:

Emre Karadoğan KARADOĞAN	211101068
Mehmet Mehmet Umur Özü ÖZÜ	211401025
Kübra Arslan ARSLAN	201401024
Fatih BAYAZIT	231401038

Course Name: YAP496

Instructor: Doç. Dr. Yücel ÇİMTAY

Submission Date: 17.04.2026

Contents

1	Executive Summary	6
2	Introduction	7
2.1	Background	7
2.2	Problem Definition	7
2.3	Objectives	7
2.4	Scope & Limitations	8
3	Related Work & Literature Review	9
3.1	Traditional Image Editing Tools	9
3.2	AI-Based Image Editing Systems	9
3.3	GAN & Deep Learning Approaches	10
3.4	Hybrid and Human-in-the-Loop Systems	10
3.5	Comparison and Research Gap	11
4	System Overview	12
4.1	System Vision	12
4.2	Key Features	12
4.3	User Workflow	13
4.4	System Characteristics	15
5	System Architecture	15
5.1	High-Level Architecture	15
5.2	Layered Architecture	16
5.3	MVC Design Pattern	17
5.4	Subsystem Decomposition	18
5.5	Data Flow	20
6	Detailed System Design	22
6.1	General Design Approach	22
6.2	Backend Design	22
6.3	API and Interface Design	23
6.4	Frontend Design	25
6.5	Model Layer Design	26
6.6	AI Model Abstraction and Processing Modules	28
6.7	AI Model Factory Design	30
6.8	Controller Layer Design	31
6.9	Session and Data Lifecycle Design	32
6.10	Refinement Workflow Design	35
6.11	Download and Output Generation Design	36

6.12	Error Handling and Failure Management Design	36
6.13	Security-Oriented Design Details	39
6.14	Community Feature Design	40
6.15	Design Consistency with Project Goals	40
7	Engineering Decisions & Trade-offs	42
7.1	Automation vs User Control	42
7.2	Accuracy vs Performance	42
7.3	Privacy vs Personalization	43
7.4	Security vs Usability	43
7.5	Stateless Architecture vs System Efficiency	44
7.6	Model Generalization vs Task-Specific Optimization	44
7.7	Feature Richness vs Interface Simplicity	45
7.8	Modularity vs Implementation Complexity	45
7.9	Ethical Capability vs Misuse Risk	46
7.10	Summary of Engineering Decisions	46
8	Security & Privacy Design	47
8.1	Privacy-by-Design Approach	47
8.2	Data Lifecycle	47
8.3	Secure Communication	48
8.4	Input Validation & Safety	48
8.5	Request Protection and Access Control	49
8.6	Content Safety and NSFW Detection	49
8.7	Data Minimization and Limited Exposure	50
8.8	Optional Persistence and Consent-Based Sharing	50
8.9	Error Handling as a Security Mechanism	50
8.10	Security and Privacy Benefits of the Architecture	51
8.11	Limitations and Areas Requiring Clarification	51
9	Implementation	52
9.1	Technologies Used	52
9.2	Frontend Implementation	55
9.3	Backend Implementation	55
9.4	AI Processing Implementation	56
9.5	Implementation of Design Patterns	58
9.6	Session and Temporary Data Handling in Implementation	59
9.7	Integration of Frontend, Backend, and AI Components	59
9.8	Deployment and Execution Environment	62
9.9	Implementation Limitations and Practical Gaps	63
9.10	Consistency Between Design and Implementation	63

10 Testing & Evaluation	64
10.1 Testing Objectives	64
10.2 Testing Strategy and Methodology	64
10.3 Test Environment	65
10.4 Functional Test Scope	66
10.5 Test Cases and Requirement Coverage	67
10.6 Test Results Summary	68
10.7 Evaluation of Core Functional Areas	69
10.8 Security and Validation Testing	69
10.9 Performance and Scalability Considerations in Testing	70
10.10 User-Centered Evaluation	70
10.11 Testing Limitations	71
10.12 Overall Evaluation	71
11 Impact Analysis	73
11.1 Technological Impact	73
11.2 Economic Impact	73
11.3 Social Impact	74
11.4 Ethical Impact	74
11.5 Privacy and Regulatory Impact	75
11.6 Environmental Impact	75
11.7 Educational and Academic Impact	76
11.8 Positive Impact Summary	76
11.9 Potential Negative Impact and Risk Summary	77
11.10 Overall Impact Assessment	77
12 Ethical Considerations	78
12.1 Ethics in AI-Assisted Image Editing	78
12.2 Responsible Use of AI Capabilities	78
12.3 Privacy and User Consent	78
12.4 Risk of Misleading or Harmful Representation	79
12.5 Beautification and Representation Ethics	79
12.6 Safety Boundaries and NSFW Restrictions	80
12.7 Fairness and Model Limitations	80
12.8 Transparency and User Awareness	80
12.9 Ethical Value of Privacy-by-Design Architecture	81
12.10 Ethical Limitations of the Project	81
12.11 Overall Ethical Assessment	81
13 Contemporary Issues	82
13.1 AI Image Editing in the Contemporary Technology Landscape	82

13.2	Generative AI and the Shift from Assistance to Transformation	82
13.3	Trust, Authenticity, and Edited Visual Content	82
13.4	Deepfakes, Manipulation, and Misuse Risk	83
13.5	Privacy in an Era of Data-Driven AI Systems	83
13.6	Human-in-the-Loop as a Contemporary Design Response	84
13.7	Regulation, Governance, and Responsible AI	84
13.8	Contemporary Relevance of Accessible Creative Tools	84
13.9	Limitations in Addressing Contemporary Issues	85
13.10	Overall Contemporary Assessment	85
14	Tools & Technologies	86
14.1	Overview of the Technology Stack	86
14.2	Frontend Technologies	86
14.3	Backend Technologies	87
14.4	AI and Image Processing Technologies	88
14.5	Data, Storage, and Infrastructure Technologies	90
14.6	Testing and Development Tools	90
14.7	Why These Technologies Were Chosen	91
14.8	Limitations and Clarifications	92
15	General Discussion of the Project Outcome	93
15.1	Implementation Plan and Realization	93
15.2	Task Distribution and Component Ownership	94
15.2.1	Frontend Components	95
15.2.2	Backend Components	97
15.2.3	AI and Processing Components	99
15.2.4	Validation Through Representative Test Cases	100
15.3	Discussion of the Realized Outcome	102
16	What Worked Well	102
16.1	Main Technical Challenges	103
16.2	Gaps Between Design and Implementation	103
16.3	Strength of the Project as an Academic System	104
16.4	Limitations of the Current System	104
16.5	Lessons Learned	105
16.6	Discussion of Project Maturity	105
16.7	Overall Discussion	105
17	Conclusion	106
17.1	Project Conclusion	106
17.2	Achievement of Project Objectives	107

17.3	Final Assessment of the System	107
17.4	Closing Statement	107
18	Future Work	108
18.1	General Future Development Direction	108
18.2	Expansion of AI Capabilities	108
18.3	Stronger Quantitative Evaluation	109
18.4	Production-Grade Scalability Improvements	109
18.5	Security and Privacy Enhancements	110
18.6	Improvement of Human-in-the-Loop Interaction	110
18.7	Better Support for Diverse Users and Inputs	111
18.8	Community and Collaboration Features	111
18.9	Cross-Platform and Product Evolution	111
18.10	Research-Oriented Future Work	112
18.11	Final Future Work Assessment	112
19	References	113
20	Appendix	117
20.1	Appendix A	117
20.2	Appendix B	118
20.3	Appendix C	118
20.4	Appendix D	119
20.5	Appendix E	119
20.6	Appendix F	120
20.7	Appendix G	120
20.8	Appendix H	121
20.9	Appendix I	121
20.10	Appendix J	122
20.11	Appendix K	123
20.12	Appendix L	123

1. Executive Summary

Invisio is a web-based, AI-powered image editing platform designed to bridge the gap between fully automated image processing systems and complex professional editing tools. While traditional software such as Adobe Photoshop provides extensive manual control, it requires significant expertise and time. On the other hand, modern AI-based tools offer fast and automated solutions but often lack mechanisms for correcting errors in generated outputs.

To address this limitation, Invisio introduces a hybrid human-in-the-loop approach, where artificial intelligence performs the primary image processing tasks, and users are allowed to make constrained refinements when necessary. This design ensures both usability and accuracy without exposing users to low-level editing complexity.

The system is built on a modular, stateless, and privacy-oriented architecture. All image processing operations are performed within a temporary session context, and no user data is stored persistently by default. This approach aligns with modern data protection regulations such as GDPR and KVKK, ensuring secure and ethical handling of user-generated content.

Technically, Invisio integrates multiple deep learning models for tasks such as object removal, image enhancement, style transfer, and colorization. The system follows a client-server architecture with a clear separation of concerns, supported by a Model-View-Controller (MVC) design pattern.

The project demonstrates that combining AI automation with limited user control significantly improves the reliability and usability of image editing systems. The final implementation successfully meets its design objectives in terms of performance, scalability, and user experience, while maintaining strong guarantees in privacy and security.

Overall, Invisio contributes a practical and responsible approach to AI-assisted image editing, providing a scalable foundation for future enhancements and real-world applications.

2. Introduction

2.1 Background

Recent advancements in artificial intelligence, particularly in deep learning and computer vision, have significantly transformed the field of image processing. Techniques such as semantic segmentation, generative adversarial networks (GANs), and neural style transfer have enabled automated systems to perform complex image editing tasks with high accuracy.

Despite these advancements, a gap remains between algorithmic capability and practical usability. Professional image editing tools provide extensive control but require technical expertise and time-consuming workflows. In contrast, AI-based tools offer fast and automated solutions but often lack flexibility and fail to handle edge cases effectively.

This gap highlights the need for systems that combine the strengths of both approaches. Thus, Invisio aims to combine automation and user control without inheriting their respective limitations.

2.2 Problem Definition

Existing image editing solutions typically fall into two categories: fully manual tools and fully automated AI-based systems. Manual tools provide precision but are not accessible to non-technical users. Automated systems, while user-friendly, often produce imperfect results that users cannot easily correct.

This limitation is particularly evident in tasks such as object removal, segmentation, and background editing, where AI models may fail to accurately capture complex boundaries such as hair, shadows, or overlapping objects.

The core problem addressed in this project is the absence of a balanced framework that harmonizes automation with user agency. Specifically, there is a need for an image editing system that overcomes the constraints of fully automated AI by allowing users to leverage AI capabilities while maintaining the ability to intuitively correct inaccuracies.

2.3 Objectives

The primary objective of the Invisio project is to develop a web-based image editing system that harmonizes advanced artificial intelligence with intuitive user interaction to enhance both usability and output accuracy. The specific objectives of the system are defined as follows:

- **Accessibility:** To provide an intuitive image editing platform tailored for non-technical users, removing the steep learning curve of professional software.
- **AI Integration:** To implement state-of-the-art AI-based processing techniques, including semantic segmentation, image inpainting, enhancement, and neural style

transfer.

- **Hybrid Workflow:** To introduce a "human-in-the-loop" editing model that empowers users to refine and correct AI-generated results through constrained manual adjustments.
- **Privacy & Security:** To ensure robust data protection through a session-based, non-persistent architecture, aligning with GDPR and KVKK standards.
- **Scalability:** To design a modular system architecture that facilitates seamless integration of future AI models and functional extensions.

These objectives collectively aim to establish a system that remains technically robust while prioritizing a seamless user experience.

2.4 Scope & Limitations

The scope of the Invisio system is limited to image-based processing tasks within a web environment. The system supports a variety of AI-driven editing operations, including object removal, background modification, enhancement, and artistic transformations.

However, several limitations are intentionally defined as part of the system design:

- The system does not support video or real-time processing
- All operations require an active internet connection.
- Performance depends on the availability of backend computational resources, particularly GPUs.
- Advanced manual editing features are not included to preserve simplicity and usability.

These constraints are considered deliberate design decisions that align with the project's objectives and available resources.

3. Related Work & Literature Review

3.1 Traditional Image Editing Tools

Traditional image editing tools such as Adobe Photoshop and GIMP have long been the standard solutions for professional image manipulation. These tools provide a wide range of functionalities, including layer-based editing, masking, color correction, and manual retouching.

The primary advantage of such tools is the level of control they offer. Users can precisely manipulate individual pixels, apply complex transformations, and fine-tune visual details. However, this level of control comes at the cost of usability. Effective use of these tools requires significant experience, technical knowledge, and time investment.

From a system design perspective, traditional tools rely entirely on manual workflows, where the user is responsible for both decision-making and execution. This makes them unsuitable for casual users or scenarios requiring fast and automated processing.

Therefore, while traditional tools provide high accuracy and flexibility, they fail to meet the needs of users seeking efficient and accessible image editing solutions.

3.2 AI-Based Image Editing Systems

Recent advancements in artificial intelligence have led to the development of automated image editing systems that aim to simplify complex editing tasks. Applications such as remove.bg, Canva AI tools, and mobile-based photo editing applications utilize deep learning models to perform tasks like background removal, enhancement, and filtering.

These systems significantly improve usability by automating the editing process. Users can achieve visually appealing results with minimal effort, often through a single interaction. However, the reliance on fully automated pipelines introduces limitations.

One of the major drawbacks of AI-based systems is the lack of user control. When the output generated by the model is incorrect or suboptimal, users typically have no mechanism to correct the result within the system. This is particularly problematic in cases involving complex image regions such as hair, transparent objects, or overlapping elements.

Additionally, many commercial AI-based platforms store user data persistently, raising concerns regarding privacy and data security. This is especially critical when processing personal or sensitive images.

These limitations highlight the need for systems that retain the usability of AI-based solutions while introducing mechanisms for controlled user intervention.

3.3 GAN & Deep Learning Approaches

Deep learning techniques, particularly convolutional neural networks (CNNs) and generative adversarial networks (GANs), form the foundation of modern image editing systems.

CNN-based architectures are widely used for tasks such as image segmentation and feature extraction. A notable example is the U-Net architecture, which has been extensively applied in pixel-level segmentation tasks due to its ability to capture both local and global image features.

GAN-based models have enabled significant advancements in generative image editing. These models consist of a generator and a discriminator network trained in an adversarial manner, allowing the system to generate realistic images. GANs have been successfully applied to tasks such as image inpainting, super-resolution, and facial enhancement.

Neural style transfer techniques further extend the capabilities of deep learning by allowing the visual style of one image to be applied to another. This is typically achieved using pretrained convolutional networks that separate content and style representations.

Despite their effectiveness, these approaches are not without limitations. Deep learning models may struggle with generalization, particularly when encountering out-of-distribution inputs. Moreover, high-quality models often require significant computational resources, which can impact system performance in real-time or web-based environments.

Specific foundational references for the methods discussed here include U-Net for encoder-decoder segmentation (Ronneberger et al., 2015), GANs for adversarial generative modeling (Goodfellow et al., 2014), and neural style transfer based on separating content and style representations in CNN feature space (Gatys et al., 2015). Representative downstream image editing applications include semantic inpainting (Pathak et al., 2016), GAN-based super-resolution (Ledig et al., 2017), and GAN-based facial beautification (Diamant et al., 2019).

3.4 Hybrid and Human-in-the-Loop Systems

To address the limitations of both manual and fully automated systems, recent research has explored hybrid approaches that integrate human input into AI-driven workflows. This concept, often referred to as human-in-the-loop (HITL), allows users to guide or refine the outputs generated by machine learning models.

In the context of image editing, hybrid systems typically allow users to interact with intermediate representations such as segmentation masks or control points. Instead of performing low-level pixel manipulation, users provide high-level corrections that influence the final output.

This approach offers a balance between usability and accuracy. AI models handle the computationally intensive tasks, while users retain the ability to correct errors in a structured and intuitive manner.

However, existing implementations of hybrid systems are often limited in scope or require specialized interfaces that may still be complex for non-technical users. Furthermore, many systems do not integrate privacy considerations into their design, focusing primarily on functionality.

Invisio builds upon this concept by introducing a constrained refinement mechanism that limits user interaction to mask-level corrections. This design ensures that user input remains simple and corrective, rather than technical or creative, thereby preserving system usability while improving output reliability.

3.5 Comparison and Research Gap

The analysis of existing systems reveals a clear trade-off between control and usability. Traditional image editing tools provide high precision but require expertise, while AI-based systems offer simplicity at the cost of flexibility.

Deep learning approaches have significantly improved the quality of automated image editing but remain limited by issues such as lack of user control, computational cost, and sensitivity to complex image scenarios.

Hybrid approaches partially address these limitations by incorporating user input into the editing process. However, current implementations often lack a clear balance between simplicity, control, and privacy.

The key research gap identified in this study is the absence of a system that simultaneously satisfies the following criteria:

- **Provides** high-level automation through AI models.
- **Allows** limited and intuitive user correction.
- **Maintains** a simple and accessible user interface.
- **Ensures** strong privacy guarantees through non-persistent data handling.

Invisio is designed to address this gap by combining a hybrid editing workflow with a privacy-by-design architecture, resulting in a system that is both practical and responsible.

4. System Overview

4.1 System Vision

Invisio is designed as an intelligent, web-based image editing platform that aims to make advanced visual manipulation accessible to a broad range of users without requiring technical expertise. The system combines artificial intelligence with structured user interaction to create a balanced editing experience that prioritizes both usability and accuracy.

The core vision of the system is to redefine how users interact with AI-based image editing tools. Instead of treating artificial intelligence as a fully autonomous decision-maker, Invisio positions AI as an assistive component that performs the majority of processing tasks while still allowing users to intervene when necessary.

This vision is grounded in three fundamental principles: accessibility, control, and responsibility. Accessibility ensures that non-technical users can effectively use the system without prior experience. Control allows users to refine AI-generated outputs in a constrained and intuitive manner. Responsibility reflects the system’s commitment to ethical AI usage and privacy-preserving data handling.

By integrating these principles, Invisio aims to provide a practical and scalable solution for modern image editing needs.

4.2 Key Features

Invisio provides a set of AI-powered image editing features designed to automate complex tasks while maintaining ease of use. These features are exposed to users through a simplified interface that abstracts underlying technical complexity.

The key functionalities of the system include:

- **Automated object removal** using inpainting techniques
- **Background removal and replacement** through segmentation models
- **Image enhancement** operations such as brightness, contrast, and color correction
- **Facial enhancement and beautification** for portrait images
- **Neural style transfer** for artistic transformations
- **Colorization** of grayscale images using deep learning models

In addition to these core AI capabilities, the system includes a refinement mechanism that allows users to correct minor inaccuracies in AI-generated outputs. This refinement is performed at the mask level, ensuring that user interaction remains intuitive and does not require pixel-level manipulation.

The system also supports preview and download functionalities, enabling users to visualize results before finalizing the editing process.

4.3 User Workflow

The user interaction model in Invisio is designed as a task-oriented workflow that simplifies the editing process into a series of intuitive steps. This workflow abstracts the complexity of underlying AI operations and presents users with a clear and guided interaction sequence.

The typical workflow consists of the following stages:

1. The user uploads an image through the web interface.
2. The user selects one or more AI-based editing operations.
3. The system processes the image using the selected AI models.
4. The user reviews the generated output.
5. If necessary, the user refines the result using the provided mask-based tools.
6. The final image is generated and made available for download.

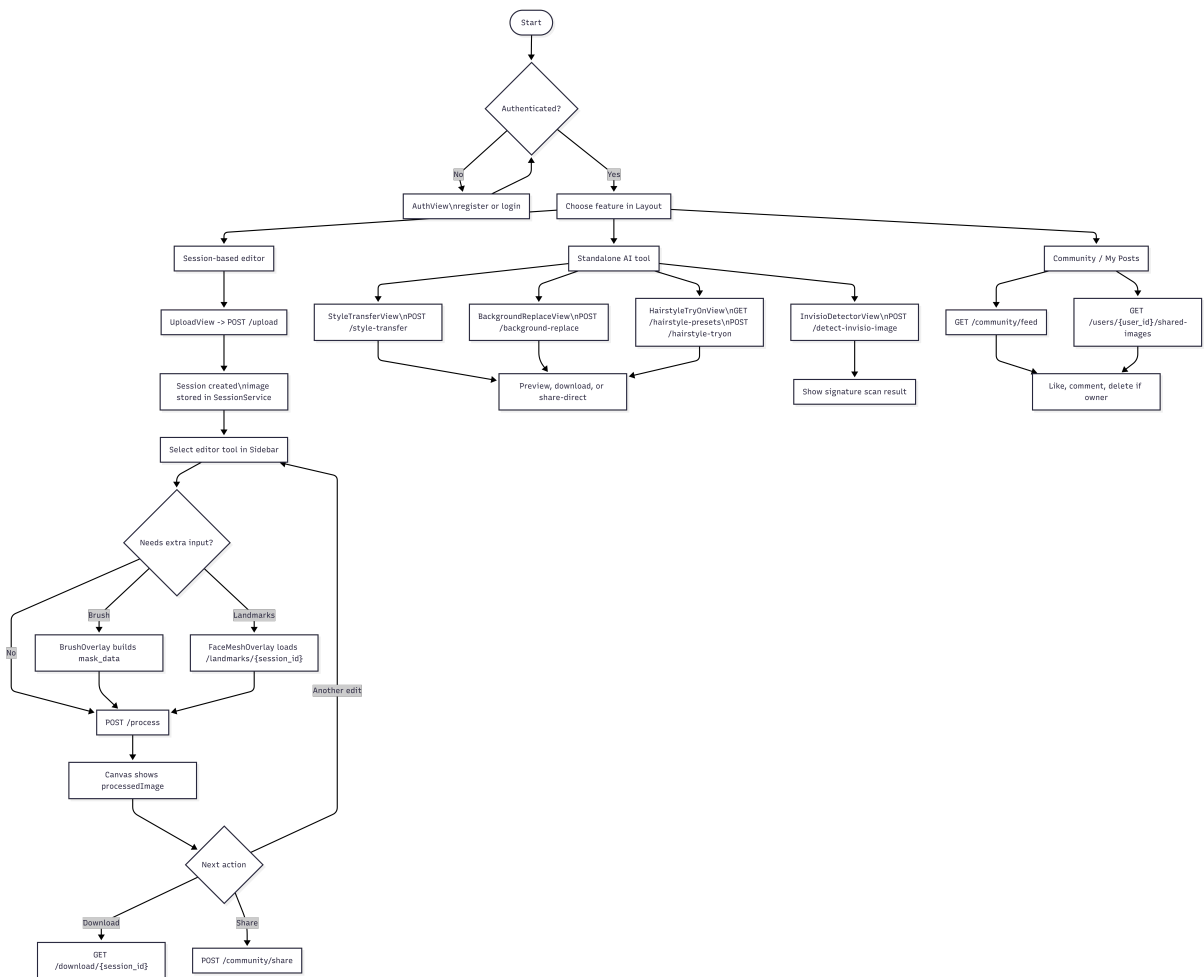


Figure 1: Main User Flow of the Invisio System

Figure 1 illustrates the user-facing workflow of the system, showing how a user progresses from image upload to AI-assisted processing, optional refinement, preview, and final download. The diagram makes explicit that the interaction model is designed as a guided flow in which complex backend and AI operations are hidden behind a sequence of high-level user actions.

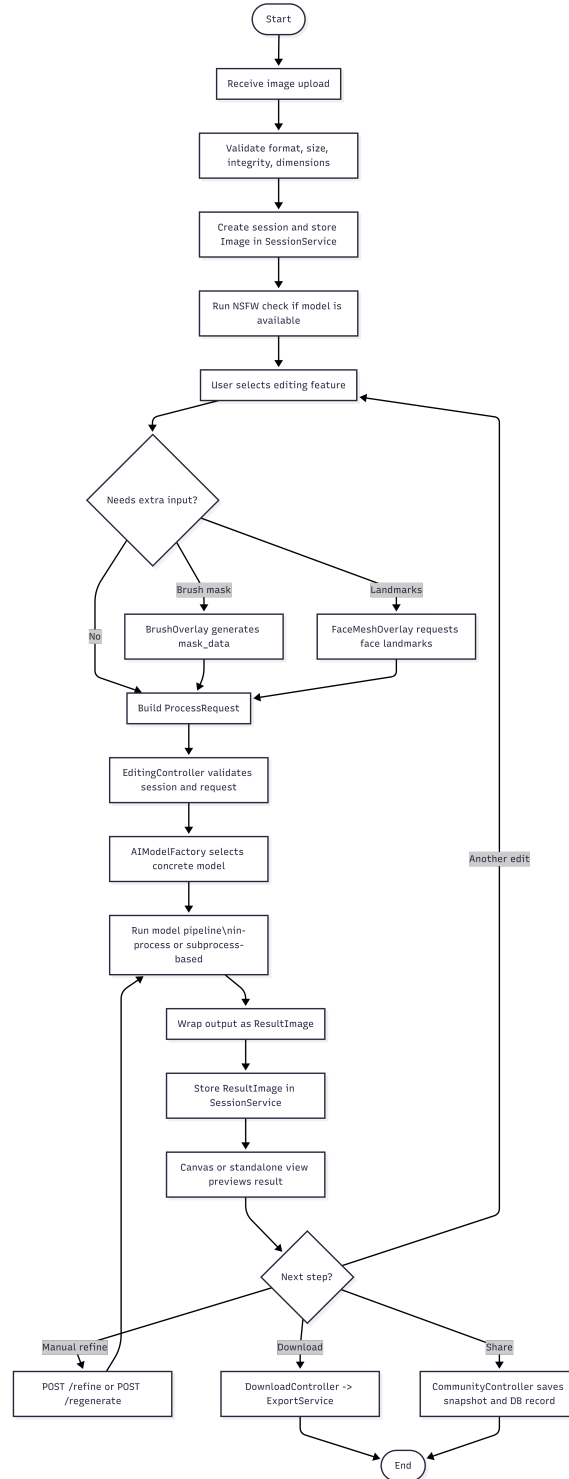


Figure 2: Overall Processing Workflow of the Invisio System

Figure 2 complements the user-flow view by showing the internal activity-oriented progression of the processing workflow. While the previous diagram emphasizes the user’s interaction path, this activity diagram highlights the system-side sequence of actions, decision points, and transitions that occur during upload, validation, AI processing, optional refinement, preview generation, and final output delivery.

This workflow supports both fully automated and hybrid interaction modes. Users who are satisfied with AI-generated outputs can proceed directly to download, while others can utilize refinement tools to improve the results.

The design of this workflow ensures a balance between efficiency and control, allowing users to achieve high-quality results with minimal effort.

4.4 System Characteristics

The Invisio system is characterized by several key architectural and operational properties that distinguish it from traditional and fully automated image editing solutions.

First, the system follows a stateless interaction model, where each request is processed independently without maintaining long-term user data. This design enhances scalability and simplifies system behavior while supporting privacy requirements.

Second, all image processing operations are performed within a session-based context using volatile memory. This ensures that user data is not stored persistently and is automatically deleted after processing is completed.

Third, the system adopts a modular architecture in which AI processing tasks are encapsulated within independent components. This allows for easy integration of new models and supports future system evolution without major architectural changes.

Finally, the system is designed to operate asynchronously, ensuring that long-running AI tasks do not block user interaction. This improves responsiveness and enhances the overall user experience in a web-based environment.

These characteristics collectively contribute to a system that is scalable, maintainable, and aligned with modern software engineering and AI design principles.

5. System Architecture

5.1 High-Level Architecture

The Invisio system is designed based on a distributed client–server architecture, where user interaction and computational processing are clearly separated. This architectural choice allows the system to handle computationally intensive AI tasks on the server side while maintaining a lightweight and responsive user interface on the client side.

In this model, the client is responsible for user interaction, including image upload, feature selection, and result visualization. The server, on the other hand, handles request

processing, AI model execution, and result generation. Communication between the client and server is performed through RESTful APIs over secure HTTP connections.

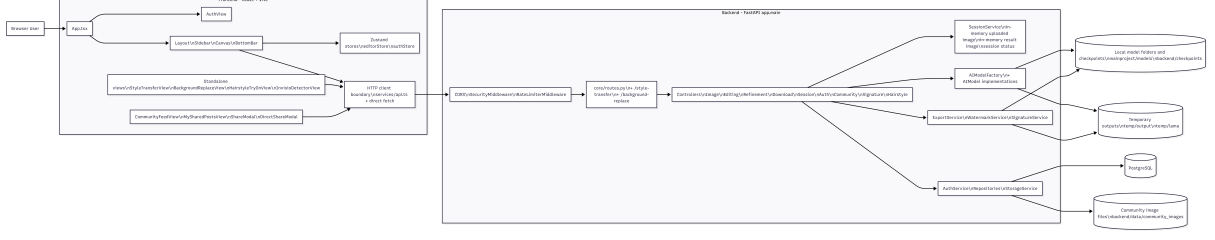


Figure 3: Top-Level System Architecture of the Invisio System

As illustrated in Figure 3, the overall architecture separates presentation responsibilities from backend coordination and AI execution responsibilities. This separation provides several advantages. It ensures consistent performance regardless of the user’s device capabilities, enables centralized management of AI resources such as GPUs, and simplifies system scalability. Additionally, it enhances security by restricting sensitive data processing to controlled backend environments.

Overall, the client–server architecture forms the foundation of the Invisio system, supporting both performance efficiency and architectural clarity.

5.2 Layered Architecture

Invisio follows a layered architectural model to ensure clear separation of concerns and maintainability. The system is divided into multiple layers, each responsible for a specific aspect of system functionality.

The primary layers are defined as follows:

- **Presentation Layer:** This layer is responsible for all user-facing components, including the graphical interface, user inputs, and result visualization. It does not contain any business logic or AI processing functionality.
- **Control Layer:** The control layer acts as the central coordination mechanism of the system. It handles incoming requests, performs validation, and determines the appropriate processing workflow. It ensures consistent system behavior and manages communication between the presentation and processing layers.
- **Processing Layer:** This layer encapsulates all AI-related operations, including preprocessing, model inference, and postprocessing. Each image editing task is executed as a discrete processing unit within this layer.
- **Resource Layer:** The resource layer manages computational resources such as CPU and GPU allocation. It abstracts hardware dependencies and ensures efficient execution of AI workloads.



Figure 4: Module Structure of the Invisio System

Figure 4 illustrates how these layers are organized into a coherent module structure. The diagram shows that user-facing modules remain isolated from coordination logic, while processing and resource-related responsibilities are kept in separate backend-oriented layers. This organization reduces coupling between modules, improves maintainability, and allows individual parts of the architecture to evolve independently without disrupting the overall system structure.

This layered approach improves system modularity, reduces coupling between components, and allows individual layers to evolve independently without affecting the overall system structure.

5.3 MVC Design Pattern

The Invisio system adopts the Model–View–Controller (MVC) design pattern to further enforce separation of concerns within the application.

In this structure, the **Model** component represents the core data structures and business logic of the system. It includes entities such as images, sessions, editing requests, and AI processing modules. The Model is responsible for managing data and executing image processing operations.

The **View** component is responsible for presenting information to the user and handling user interactions. It includes all user interface elements such as upload screens, editing options, preview displays, and download interfaces.

The **Controller** component acts as an intermediary between the Model and View layers. It receives user inputs from the View, processes them, invokes the appropriate Model operations, and returns the results back to the View.

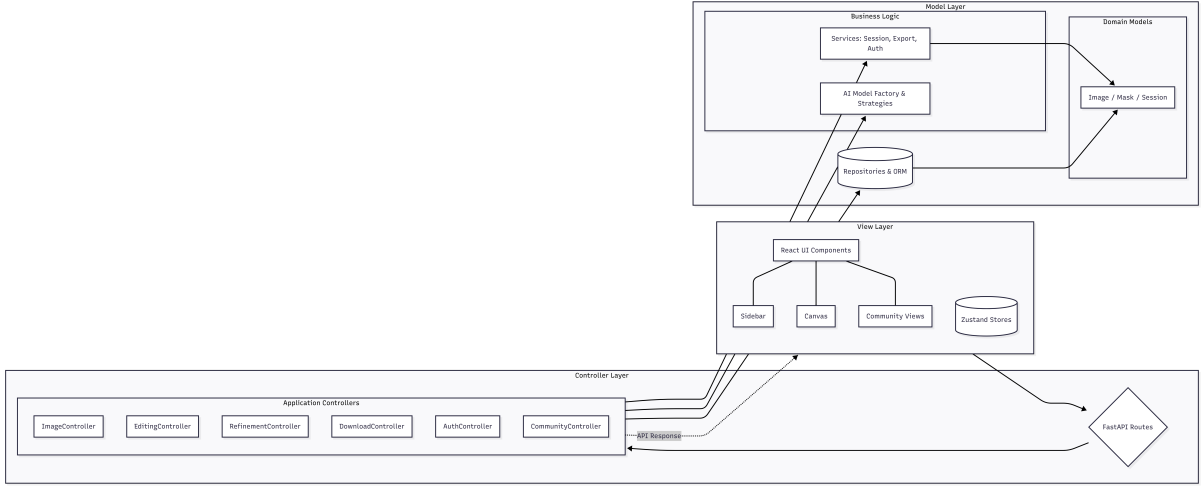


Figure 5: MVC-Based Architectural Organization of the Invisio System

Figure 5 illustrates how the implemented system is organized around the MVC pattern. The diagram shows the separation between frontend interaction components, backend coordination components, and model-related processing structures, thereby making the flow between user-facing views, controller logic, and processing responsibilities more explicit.

This architectural pattern improves maintainability and testability by isolating responsibilities. It also prevents tight coupling between user interface components and core processing logic, which is particularly important in systems that integrate complex AI pipelines.

5.4 Subsystem Decomposition

To manage system complexity, Invisio is decomposed into multiple subsystems based on functional responsibilities. This decomposition enables modular development and improves system clarity.

The main subsystems are defined as follows:

- **User Interaction Subsystem:** Handles all user-facing operations, including image upload, feature selection, preview display, and download functionality.
- **Request Management Subsystem:** Processes incoming requests, performs validation checks, and coordinates the execution of processing tasks.
- **Image Processing Subsystem:** Responsible for preparing input data, performing transformations, and handling intermediate processing steps.
- **Model Execution Subsystem:** Executes AI models responsible for tasks such as segmentation, inpainting, enhancement, and style transfer.

- **Output Generation Subsystem:** Formats processed images and prepares them for visualization and download.
- **Monitoring and Error Handling Subsystem:** Tracks system behavior, detects failures, and provides meaningful feedback to users.

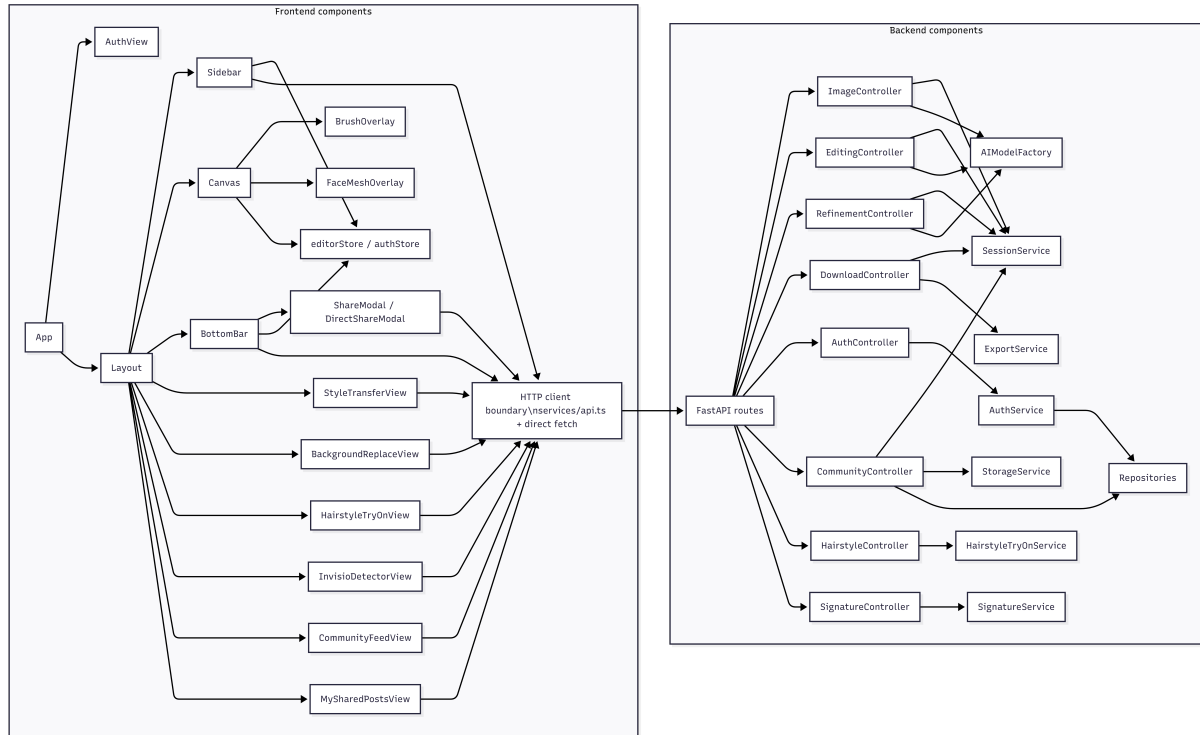


Figure 6: Component Diagram of the Invisio System

Figure 6 presents the component-level organization of the Invisio system. The diagram shows how frontend interface components, backend controllers, service-layer elements, and model-selection structures are separated into distinct but interacting parts. It also makes visible the communication paths between user-facing modules, API-facing coordination components, session-related services, and backend processing elements.

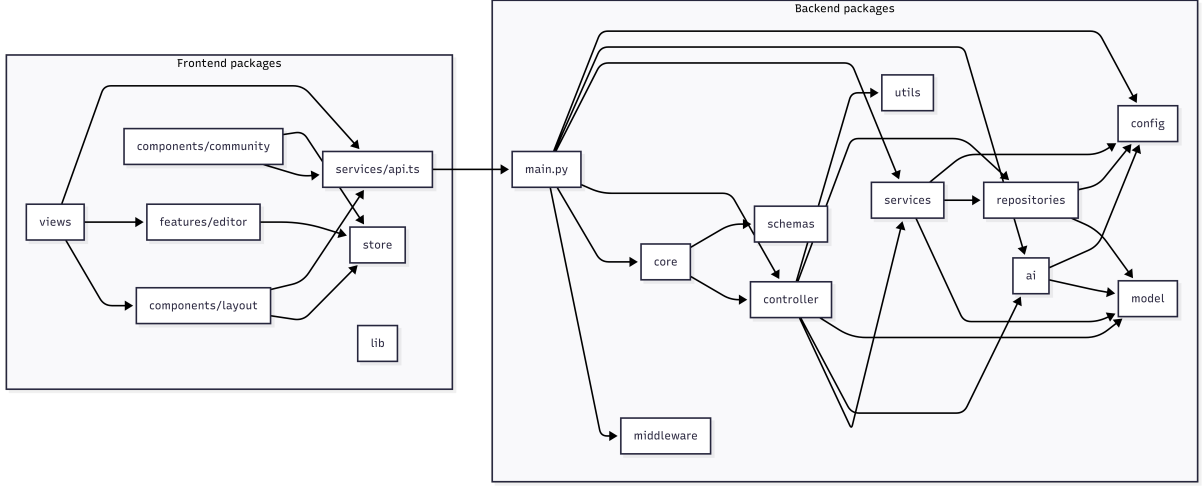


Figure 7: Package Diagram of the Invisio System

Figure 7 complements the component view by showing the organizational decomposition of the system into packages and grouped software units. While the component diagram emphasizes runtime-level interaction between major parts, the package diagram highlights how the codebase and architectural responsibilities are structured into coherent modules. This makes the modular organization of the system more explicit and supports the maintainability-oriented design approach adopted in the project.

This decomposition ensures that each subsystem has a well-defined responsibility, reducing system complexity and supporting scalability and maintainability.

5.5 Data Flow

The data flow in Invisio follows a structured request–response lifecycle that ensures controlled and predictable system behavior.

When a user uploads an image, the request is first received by the backend through the API layer. The system validates the input data, including file format, size, and content safety checks. If the validation is successful, the image is stored temporarily within a session context.

Next, the user’s editing request is processed by the control layer, which determines the appropriate AI model and processing pipeline. The image is then passed to the processing layer, where preprocessing, model inference, and postprocessing steps are executed sequentially.

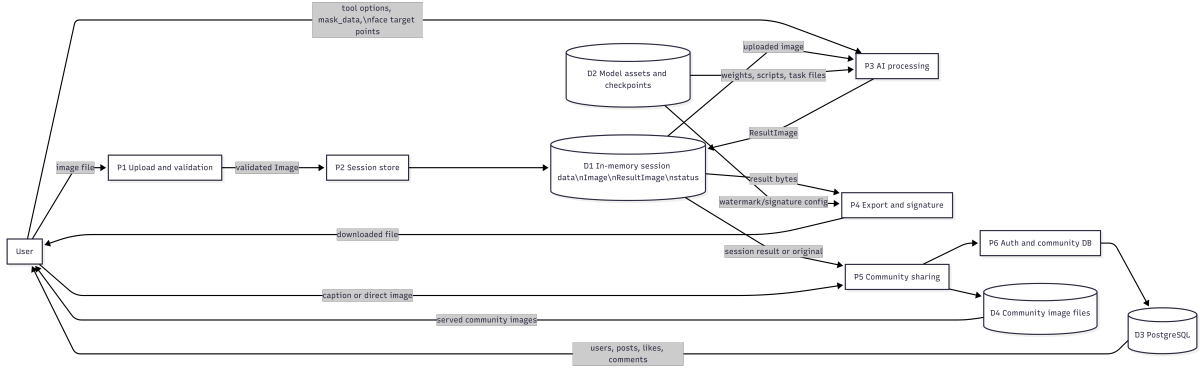


Figure 8: Data Flow in the Invisio System

Figure 8 illustrates the end-to-end flow of data across the system. The diagram shows how uploaded content moves from the user-facing layer to the backend validation and coordination layers, continues through AI processing and intermediate session-based handling, and is finally returned to the client as a preview or downloadable result. It also makes explicit that the workflow is organized around temporary processing and controlled transitions between architectural layers.

Once processing is completed, the resulting image is returned to the client for preview. If the user applies refinement, the updated data is reprocessed through the same pipeline. Finally, the completed result is delivered to the user for download.

After the request lifecycle is completed, all session-related data is deleted from memory, ensuring that no persistent data is retained.

This structured data flow ensures reliability, security, and consistency across all system operations.

6. Detailed System Design

6.1 General Design Approach

The detailed design of Invisio is derived from the high-level architectural principles established in the earlier stages of the project and translates them into concrete software components, interfaces, and execution flows. The system is designed to support AI-assisted image editing in a modular, maintainable, and privacy-preserving manner.

At the design level, Invisio combines a layered architecture with the Model–View–Controller (MVC) pattern. This allows the system to separate user interface responsibilities, request coordination, and AI-based image processing into clearly defined and independently manageable components.

A key design objective is to ensure that complex AI operations remain hidden from the user while still being controllable through high-level interactions. For this reason, the system does not expose low-level model parameters or pixel-based editing controls. Instead, it provides a constrained workflow in which users select editing intent, receive AI-generated results, and optionally refine the output through structured mask-based corrections.

The detailed system design also reflects the project’s privacy-by-design philosophy. Image data is handled only within temporary sessions, stored in volatile memory, and deleted after processing is completed. Persistent storage is intentionally avoided in the default workflow.

From an engineering perspective, the design emphasizes modularity, reusability, extensibility, and controlled failure handling. AI models are abstracted behind common interfaces, controllers are isolated from implementation details of specific models, and frontend components are restricted to presentation and user interaction concerns.

6.2 Backend Design

The backend of Invisio is responsible for coordinating user requests, managing sessions, enforcing security constraints, and orchestrating AI-based image processing operations. It acts as the central execution environment of the system and forms the bridge between the web interface and the AI processing modules.

The backend is designed around RESTful communication principles. Requests originating from the frontend are transmitted to backend endpoints over HTTPS, using JSON for structured data exchange and multipart/form-data for image uploads. The backend validates requests before forwarding them to the appropriate controller logic.

The main responsibilities of the backend include:

- Receiving uploaded image files
- Validating file format, size, and integrity

- Creating and managing temporary sessions
- Routing editing requests to the appropriate AI model
- Returning processed outputs in a standardized response format
- Handling errors and communicating failure states safely
- Cleaning up temporary data after processing is completed

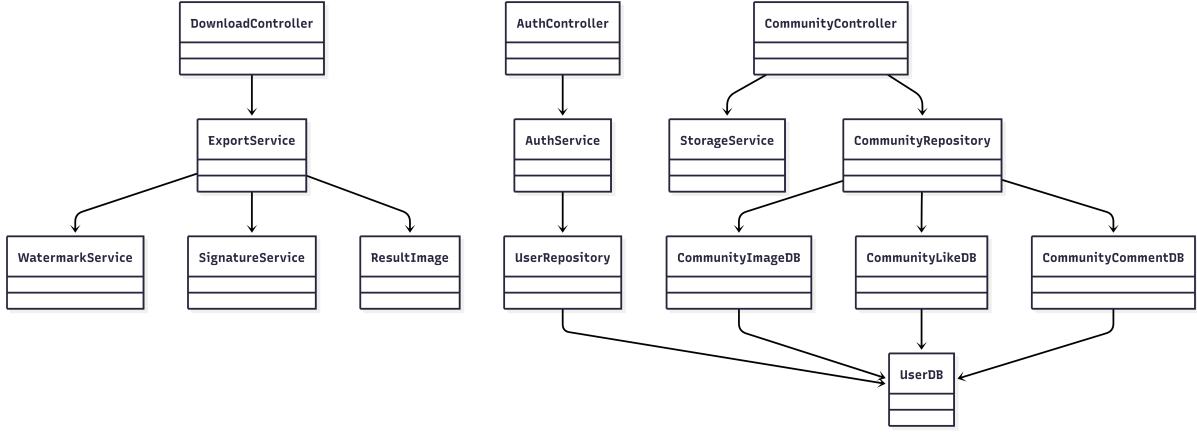


Figure 9: Detailed Class Diagram of the Export, Auth, and Community Subsystem

Figure 9 illustrates the detailed backend-side organization of the export, authentication, and community-related subsystem. The diagram shows how account-related classes, export-oriented structures, and community-sharing components are grouped within the broader backend architecture and how they interact with persistence, access control, and result-delivery responsibilities. This representation makes clear that these features are implemented as structured backend subsystems rather than isolated endpoint-level additions.

The backend design follows the controller-oriented coordination model defined in the MVC architecture. Controllers are responsible for request-specific orchestration, while AI-related execution remains encapsulated in the model layer. This separation avoids tight coupling between request handling and computational logic.

In addition, the backend is designed to be compatible with asynchronous processing. Since AI inference tasks may be computationally intensive, the backend structure supports non-blocking request handling and progress-aware workflow management at the architectural level.

6.3 API and Interface Design

The communication between system components is based on clearly defined interfaces. At the frontend–backend boundary, Invisio uses RESTful APIs that provide predictable and

standardized request–response behavior. At the communication level, the implemented API supports both JSON-based structured requests and multipart/form-data uploads for image-oriented workflows. In addition, binary result delivery is supported through response mechanisms suitable for downloadable processed outputs and export-related file transfer. This interface structure simplifies frontend integration, improves maintainability, and supports consistent error handling.

The core API operations include:

- **Image upload endpoint:** accepts image files and initializes a temporary session
- **Processing endpoint:** receives the selected editing operation and any related parameters
- **Status endpoint:** provides processing state information for a given session
- **Download-related response flow:** returns processed results to the user in an exportable format

All API responses follow a unified structure in order to provide consistency across successful and failed operations. A typical response contains:

- **status**
- **message**
- **data**

In error scenarios, the response also includes a structured error code. This design prevents ambiguity at the frontend level and ensures that failures can be handled in a controlled and user-friendly manner.

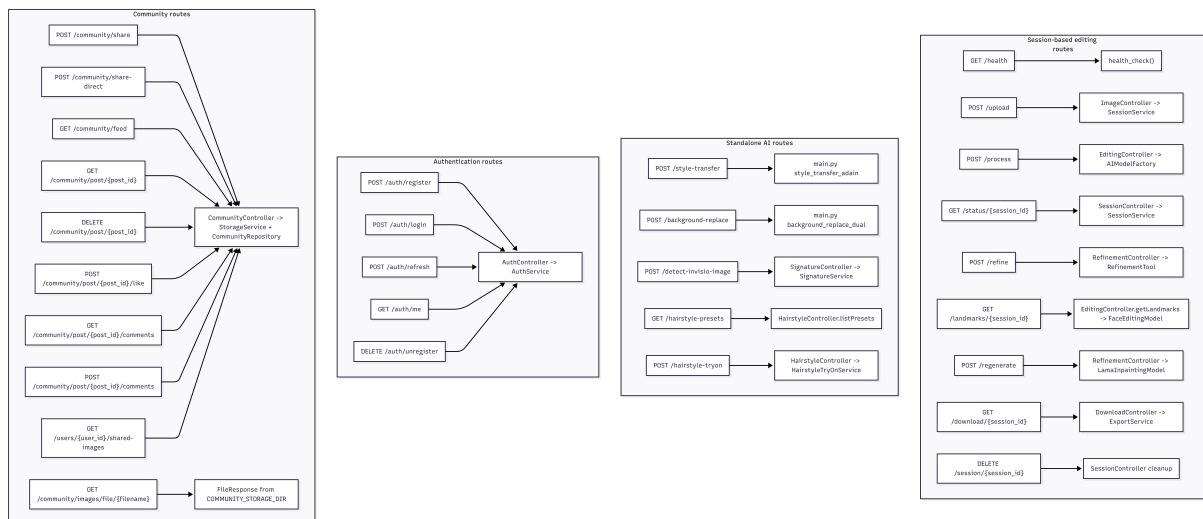


Figure 10: API Route Map of the Invisio System

Figure 10 illustrates the route-level organization of the backend API surface. The diagram shows how the main endpoints are grouped according to their functional responsibilities, including upload and processing operations, session-oriented request flow, refinement-related endpoints, standalone AI routes, authentication paths, community endpoints, and specialized detection or download-related interfaces. This representation makes the interface structure of the system more explicit and helps relate the conceptual API design to the implemented request architecture.

The backend–AI interface is also standardized. Rather than allowing controllers to directly manipulate individual models in custom ways, each AI processing component follows a shared execution contract. This contract includes model loading, preprocessing, inference, and postprocessing stages. Such standardization reduces integration complexity and makes the system more extensible.

The interface design of Invisio therefore serves two purposes simultaneously: it enables reliable communication across software layers and ensures independence between orchestration logic and model-specific implementation details.

6.4 Frontend Design

The frontend of Invisio is responsible for all user-facing interactions and visual presentation. It provides a simple and accessible web interface through which users can upload images, choose editing operations, inspect results, and download outputs. In line with the overall architecture, the frontend does not contain business logic or AI execution responsibilities.

The frontend design is organized around view components that correspond to the main stages of the user workflow. These components are responsible for rendering the interface, collecting user input, displaying system feedback, and invoking the relevant controller actions through the backend interface.

The main frontend views are:

- **MainLayout:** serves as the root layout of the application and manages navigation between screens
- **UploadView:** handles image selection and upload initiation
- **EditingOptionsView:** allows users to choose AI-based editing operations and define high-level parameters
- **RefinementView:** provides mask-based interaction for correcting AI-generated segmentation results
- **PreviewView:** displays temporary processing results before final confirmation
- **DownloadView:** enables users to export the final processed image

- **CommunityView:** supports browsing and sharing public images if community functionality is enabled
- **ErrorView:** presents validation, authorization, or processing errors in a structured form

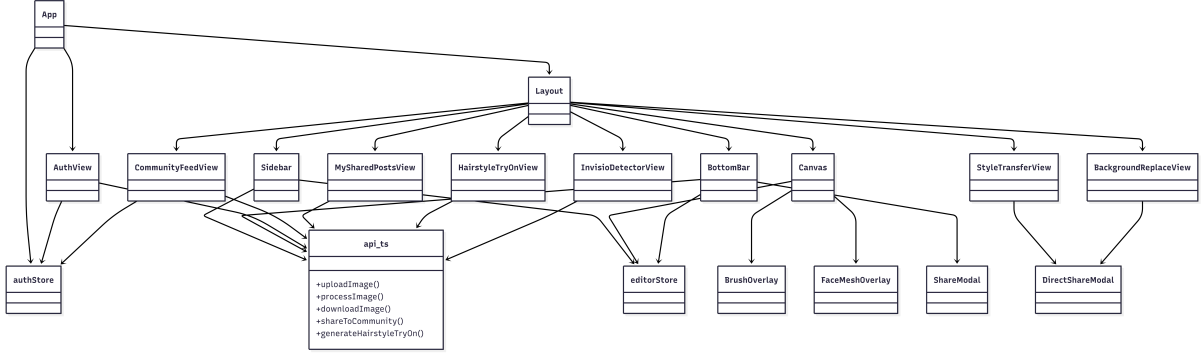


Figure 11: Detailed UI Subsystem Class Diagram of the Invisio System

Figure 11 illustrates the structural organization of the UI subsystem in greater detail. The diagram shows how layout components, view-level classes, interaction-oriented frontend units, and user-facing workflow elements are grouped within the presentation layer. This representation makes explicit that the frontend is not implemented as a single interface block, but as a structured subsystem composed of distinct but coordinated view components.

The interface design deliberately avoids exposing advanced technical controls. Users interact with the system through action-oriented operations such as “remove object,” “enhance image,” or “apply style,” rather than selecting model architectures or numerical inference parameters.

This design decision supports usability for non-technical users while preserving consistency across different editing tasks. It also reduces cognitive load and prevents the interface from evolving into a professional-grade editing environment, which is outside the intended scope of the project.

6.5 Model Layer Design

The model layer constitutes the computational and representational core of the Invisio system. It contains the data structures, domain entities, and AI processing abstractions required to execute image editing workflows.

The primary entities defined in the model layer include:

- User
- Session
- Image

- EditingRequest
- Mask
- ResultImage
- CommunityImage
- RefinementTool
- AIModel and its derived processing models
- AIModelFactory

Each of these classes serves a distinct responsibility within the system.

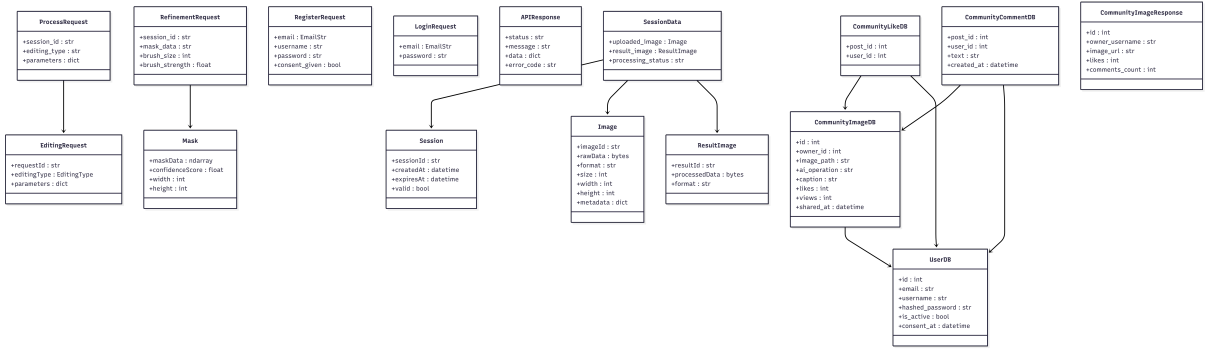


Figure 12: Entity Data Model of the Invisio System

Figure 12 illustrates the structural relationships between the main entities defined in the model layer. The diagram makes explicit how user-related, session-related, image-related, request-related, and result-related entities are organized and how they interact with processing-oriented abstractions such as masks, refinement tools, and AI model components. This representation helps clarify that the model layer is not limited to data storage alone, but also captures the domain structure required for coordinating editing workflows.

The User entity represents a system user and maintains authentication-related state and image-sharing associations where applicable. Although mandatory registration is not required in the default workflow, the class exists to support optional authenticated or community-related interactions.

The Session entity manages temporary processing context, including the session identifier, lifecycle timestamps, and validity status. It plays a critical role in enforcing privacy constraints by ensuring that image data exists only within a controlled session scope.

The Image entity represents uploaded user content. It stores raw image data together with metadata such as format, size, width, height, and additional descriptive properties. Validation and normalization operations are associated with this entity.

The `EditingRequest` entity encapsulates the user’s requested editing action. It stores the selected editing type, request parameters, and timing information. This abstraction allows different operations to be handled through a unified request structure.

The `Mask` entity is used in operations that require region-based manipulation, particularly segmentation and refinement workflows. It stores pixel-level or structured mask information and supports compatibility validation with the related image.

The `ResultImage` entity represents the output of an AI processing operation. It provides the processed data and export functionality. The `CommunityImage` entity extends this concept for publicly shared outputs by adding social interaction metadata such as like count and view count.

The `RefinementTool` supports controlled user intervention by enabling brush-based mask adjustments. Importantly, it operates on masks rather than directly on image pixels, which preserves the project’s human-in-the-loop design philosophy.

Overall, the model layer is designed to provide a clean representation of system state and to isolate core processing logic from interface and coordination responsibilities.

6.6 AI Model Abstraction and Processing Modules

A major design goal of Invisio is to support multiple heterogeneous AI-based editing operations without creating tight coupling between the system architecture and specific model implementations. To achieve this, all AI-based processing components are abstracted through a shared `AIModel` interface.

The abstract `AIModel` class defines the common contract that all processing modules must satisfy. This contract includes:

- loading model weights into memory
- preprocessing input images into a model-compatible format
- executing inference
- postprocessing raw outputs into user-consumable results
- reporting model readiness state

This design directly reflects the use of the Strategy Pattern. In this pattern, a family of related algorithms is encapsulated behind a common interface so that they can be selected and used interchangeably at runtime. In Invisio, each AI-based editing capability is represented as a concrete strategy derived from the `AIModel` abstraction. As a result, the system can switch between different processing behaviors depending on the selected editing request without altering the higher-level orchestration flow.

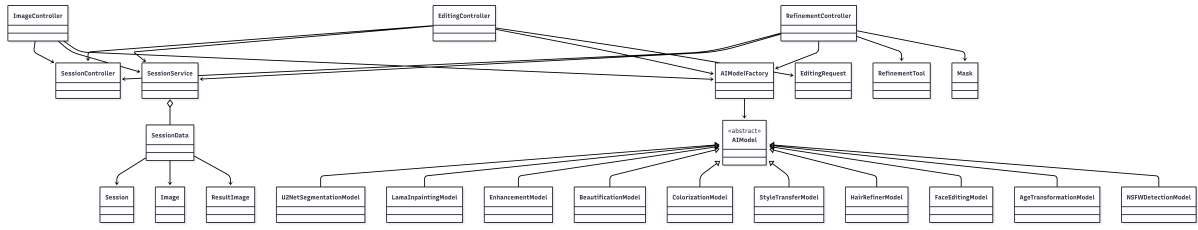


Figure 13: Detailed Processing Subsystem Class Diagram of the Invisio System

Figure 13 illustrates the detailed class-level structure of the processing subsystem. The diagram shows how abstract processing contracts, specialized AI model classes, preprocessing and postprocessing responsibilities, and task-specific execution units are organized within the model-oriented processing layer. This representation makes clear that different AI capabilities are implemented as separate but structurally compatible modules rather than as tightly coupled feature-specific scripts.

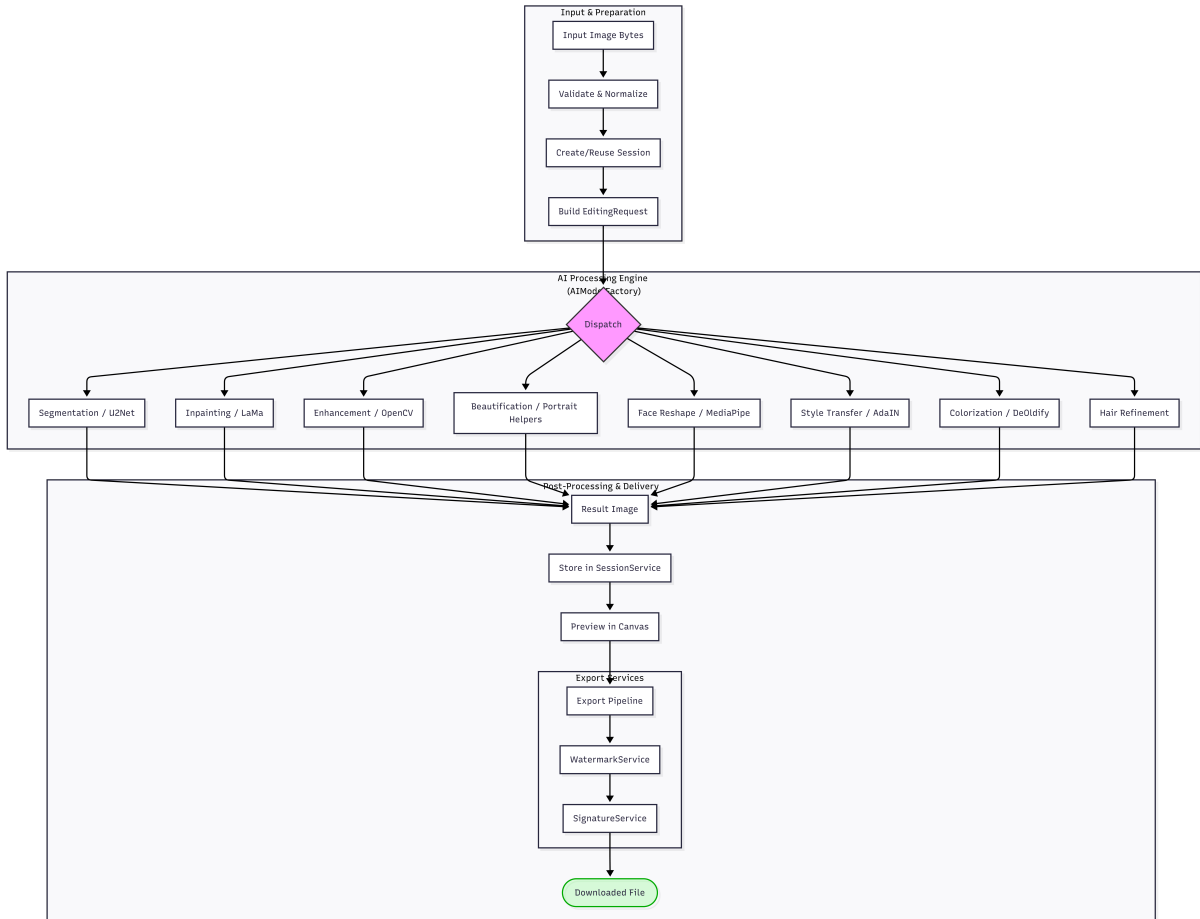


Figure 14: AI Processing Pipeline of the Invisio System

Figure 14 complements the class-oriented view by illustrating the operational pipeline followed during AI-based processing. The diagram shows how input images move through preprocessing, model selection, task-specific inference, intermediate transformation stages,

and postprocessing before being returned as usable results to the higher-level coordination flow. In this way, the pipeline view explains the runtime behavior of the processing subsystem, while the previous class diagram explains its structural organization.

By enforcing a uniform processing structure, the system ensures that different editing models can be integrated, replaced, or upgraded without changing higher-level controller logic. This improves modularity and allows the architecture to remain model-agnostic.

The specialized AI processing modules defined in the system include:

- **SegmentationModel:** Used for identifying foreground regions, object boundaries, or editable areas. This model is particularly important for background removal, mask generation, and refinement workflows.
- **InpaintingModel:** Used for filling removed or masked regions in a visually coherent manner. This model supports object removal and region reconstruction scenarios.
- **EnhancementModel:** Responsible for improving image quality through operations such as denoising, sharpening, and contrast optimization.
- **StyleTransferModel:** Applies artistic transformations to images by transferring stylistic properties while preserving content structure.
- **BeautificationModel:** Focuses on portrait enhancement tasks, including facial refinement and aesthetic corrections.
- **ColorizationModel:** Converts grayscale images into colorized outputs through learned color mapping.
- **NSFWDetectionModel:** Used as a safety-oriented validation component that checks whether uploaded content violates content policy constraints before allowing further processing.

This modular design reflects the project’s task-based approach to AI integration. Instead of relying on a single monolithic model, each editing capability is isolated in a dedicated component and behaves as a separate processing strategy. This improves flexibility, encourages maintainability, and allows the system to evolve as individual model technologies improve.

6.7 AI Model Factory Design

To support extensibility and avoid direct dependency between controllers and individual AI model classes, Invisio uses a factory-based model creation mechanism. This role is fulfilled by the `AIModelFactory` component.

The `AIModelFactory` maps editing types to their corresponding processing modules and returns the correct `AIModel` instance when a request is received. Through this

structure, the controller does not need to know the implementation details of each model class. It only requests a model appropriate for the selected editing operation.

This design has several advantages:

- **It centralizes model registration and selection logic**
- **It supports the Open/Closed Principle** by allowing new models to be added without modifying controller code
- **It simplifies future extension of the system** with additional editing capabilities
- **It improves maintainability** by decoupling orchestration logic from object creation responsibilities

The factory mechanism is especially useful in a system like Invisio, where multiple AI-based operations coexist and may evolve independently over time.

By introducing the `AIModelFactory`, the design ensures that model management remains scalable and architecturally consistent.

6.8 Controller Layer Design

The controller layer coordinates system behavior by receiving user-triggered requests, validating them, invoking the appropriate model operations, and returning the results. Controllers serve as the primary coordination mechanism between the frontend and the model layer.

The key controllers in the design are as follows:

- **ImageController:** Responsible for receiving uploaded image files, validating format and size constraints, checking file integrity, and forwarding valid data into temporary session storage.
- **EditingController:** Acts as the main coordination point for AI-based editing operations. It receives an `EditingRequest`, selects the appropriate model through the `AIModelFactory`, validates request consistency, and triggers the processing workflow.
- **RefinementController:** Manages user-driven corrections applied to AI-generated masks. It receives mask updates from the frontend, invokes the `RefinementTool`, and regenerates results when refinement is completed.
- **DownloadController:** Handles export-related operations and prepares processed images for user download in the selected output format.
- **SessionController:** Manages the entire lifecycle of temporary sessions, including creation, validation, expiration, image storage, retrieval, and final cleanup.

- **SecurityController:** Acts as an interceptor-style controller that performs request validation, token checking where relevant, and rate limiting. It exists to ensure that invalid or abusive requests do not reach the core processing logic.

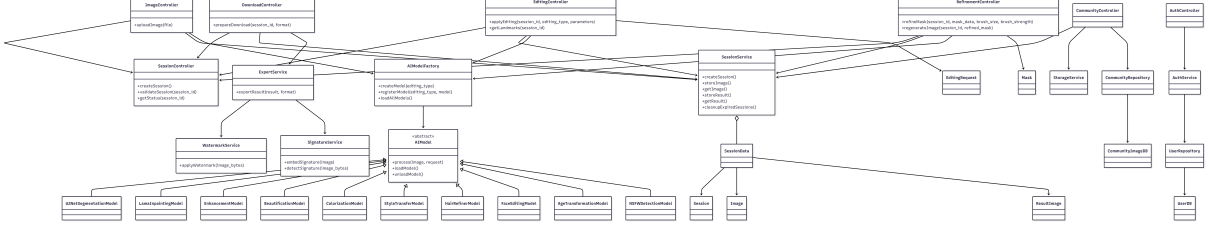


Figure 15: Core Backend Class Structure of the Invisio System

Figure 15 illustrates the structural organization of the main backend classes and their relationships. The diagram makes explicit how controllers, session-related components, request-handling structures, and processing-oriented backend classes are connected within the coordination layer. This representation supports the architectural view that backend logic is organized around distinct but cooperating responsibility areas rather than a monolithic request-processing structure.

Among these components, the **EditingController** plays a particularly important architectural role because it reflects the Facade Pattern. The Facade Pattern provides a simplified and unified interface to a more complex subsystem. In Invisio, AI-based editing involves multiple internal components such as request validation, model selection, session-based image retrieval, preprocessing, inference, postprocessing, and result generation. Instead of exposing this complexity to the frontend or distributing it across unrelated components, the **EditingController** presents a single, coherent entry point for editing operations.

This facade-based design improves architectural clarity in several ways. First, it reduces coupling between the user interface and the internal AI processing workflow. Second, it hides the complexity of heterogeneous model handling behind a stable interface. Third, it makes the system easier to test, maintain, and extend, since changes within the AI processing subsystem do not require corresponding changes in the presentation layer.

This controller decomposition ensures that system responsibilities are not concentrated in a single component. Instead, each controller focuses on a limited responsibility area, while the **EditingController** specifically acts as a simplified facade over the more complex AI editing subsystem.

6.9 Session and Data Lifecycle Design

Session and data lifecycle management are central to the detailed design of Invisio because the system is explicitly built around privacy-by-design and data minimization principles.

Whenever a user uploads an image, the system creates or uses a temporary session context that acts as the boundary for all subsequent operations. Within this session, the system may store:

- the uploaded image
- selected editing operation
- intermediate masks or processing state
- the final generated result

This data is not intended for persistent retention. Instead, it exists only for the duration of the active editing workflow and is stored in volatile memory or equivalent temporary runtime structures. Once the user completes the process or the session expires, the related data is removed.

The SessionController is responsible for managing this lifecycle. It creates new sessions, validates whether a session is active, stores temporary image references, retrieves data when needed by the processing pipeline, and performs cleanup after completion.

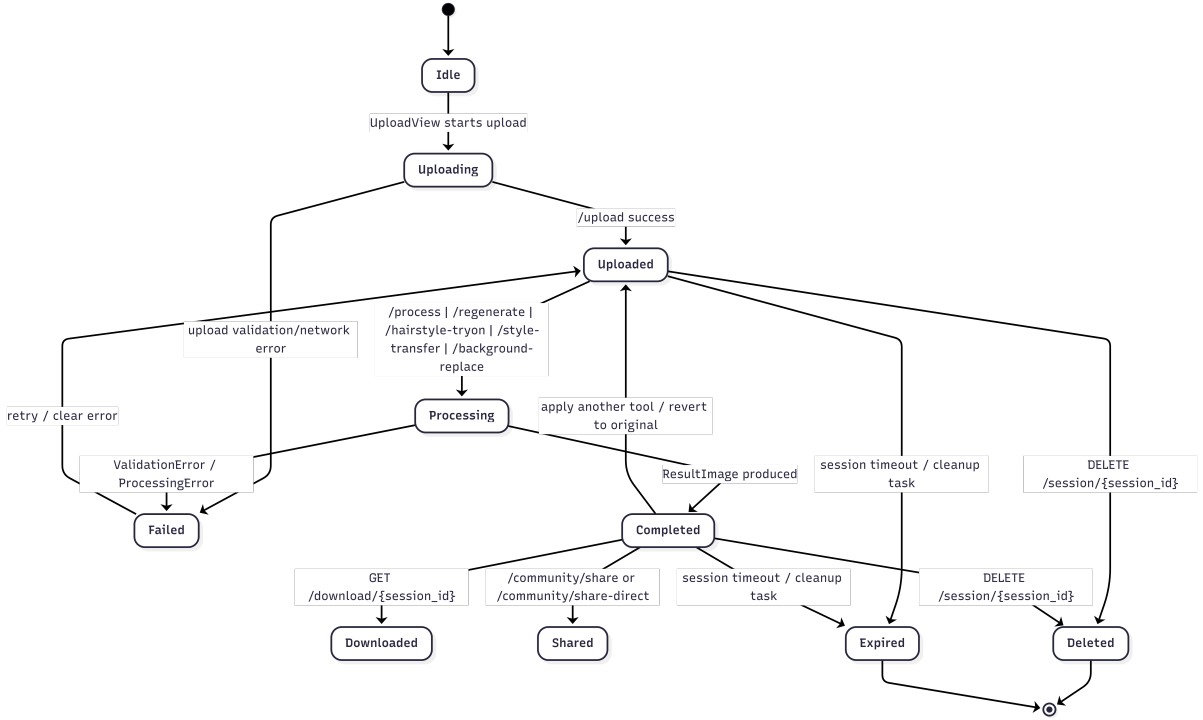


Figure 16: Session and Processing State Transitions in the Invisio System

Figure 16 illustrates the state-based progression of a temporary session throughout the processing lifecycle. The diagram shows how uploaded content moves through session creation, validation, active processing, intermediate result handling, optional refinement-related transitions, and final cleanup or expiration. This makes explicit that the system is organized around controlled state transitions rather than long-term persistence.

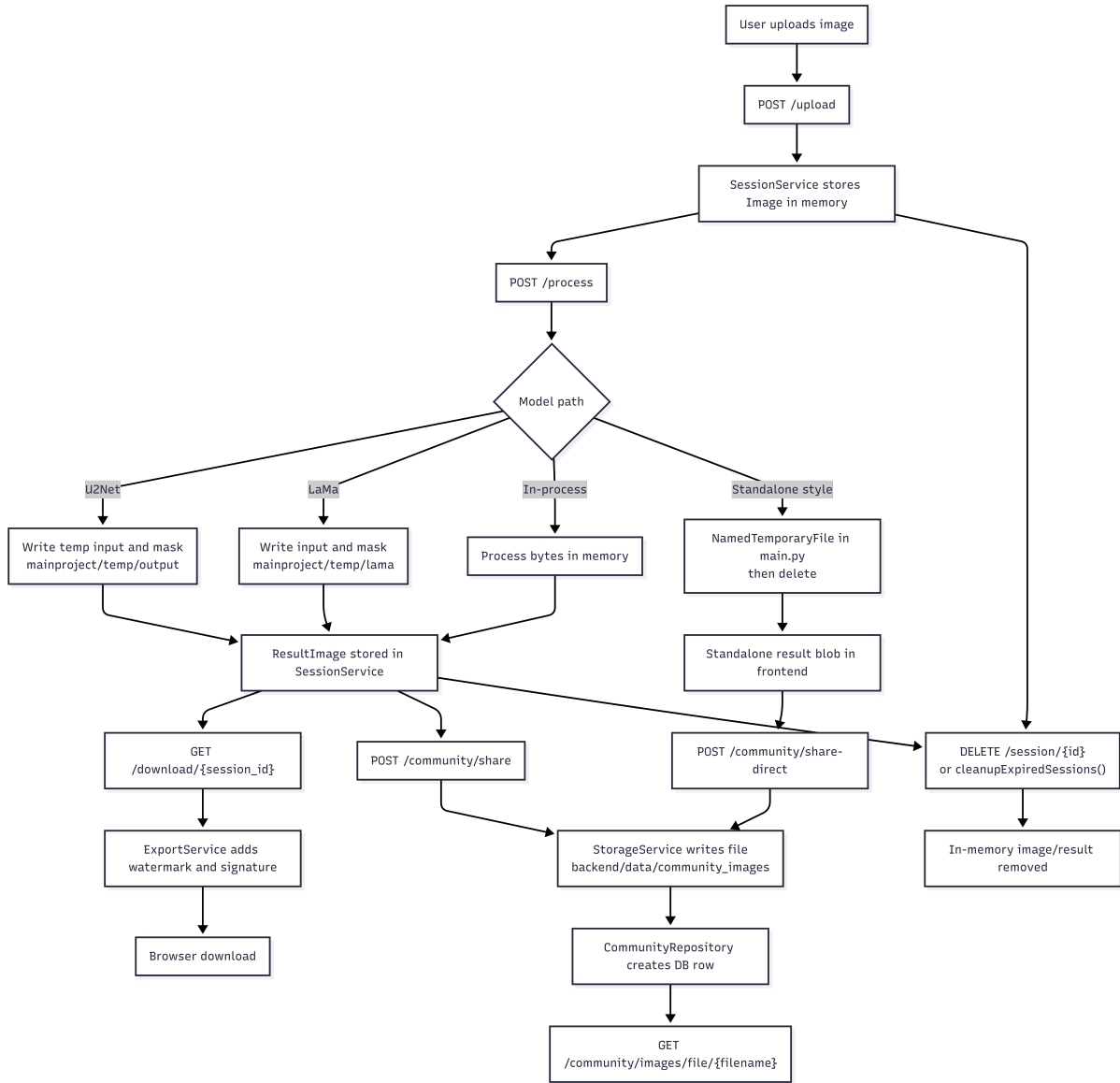


Figure 17: File Lifecycle in the Invisio System

Figure 17 complements the session-state view by showing the lifecycle of image-related artifacts handled during the workflow. While the previous diagram emphasizes session-level state transitions, this diagram highlights how uploaded files, intermediate processing outputs, refined artifacts, generated results, and optionally persisted community outputs move through creation, temporary usage, transformation, delivery, and cleanup stages. In this way, it clarifies that file handling in Invisio is organized around controlled and short-lived lifecycle boundaries rather than unrestricted storage.

This session-based design provides several advantages:

- It reduces the legal and ethical risks associated with long-term storage of user images.
- It lowers the attack surface by minimizing stored sensitive content.

- It aligns the system with GDPR and KVKK-inspired privacy expectations.
- It supports a stateless architectural philosophy at the application level while still enabling temporary workflow continuity.

An important nuance in the project documents is that the default workflow avoids persistent storage, but optional persistence may exist in explicitly user-authorized scenarios such as community sharing. In such cases, the persisted artifact is the processed result rather than the original session data, and personal metadata should be removed before publication.

6.10 Refinement Workflow Design

One of the defining design characteristics of Invisio is its support for a hybrid editing workflow in which automated AI processing can be followed by limited user refinement. This mechanism is designed to improve output quality in cases where AI-generated masks or transformations are not fully accurate.

The refinement workflow begins after an initial AI operation produces an intermediate result, typically a segmentation mask or region-based output. If the user observes inaccuracies, such as imprecise boundaries or missed regions, the system allows corrective interaction through the `RefinementView`.

Rather than exposing full manual editing tools, the system restricts refinement to structured mask editing using brush-based input. The user interacts with the mask representation, not directly with the final image pixels. This distinction is essential because it preserves the role of AI as the primary editing engine while allowing human input only for correction.

The refinement input is transmitted to the backend through the `RefinementController`, which coordinates the refinement process. The `RefinementTool` applies the brush operations to the mask, validates consistency, and returns the updated mask. The system then regenerates the output using the refined mask through the appropriate processing pipeline.

This workflow offers several benefits:

- It increases result accuracy in edge cases
- It avoids the complexity of professional editing interfaces
- It preserves the usability goals of the platform
- It operationalizes the project’s human-in-the-loop design principle

The refinement workflow is therefore not an auxiliary feature, but one of the core mechanisms that differentiates Invisio from fully automated AI editing systems.

6.11 Download and Output Generation Design

After image processing is completed, the system prepares the output for user access through the download and export workflow. The goal of this design is to ensure that processed results are delivered in a user-friendly and consistent format.

The ResultImage abstraction represents the output produced by any AI processing module. It stores the processed image bytes together with format information and generation metadata. This abstraction allows the system to treat outputs from different AI pipelines uniformly.

When a user requests the final result, the DownloadController retrieves the processed output associated with the current session and prepares it in the selected export format. If required, format conversion is performed before delivery.

The output generation process is designed with the following goals:

- consistency across different editing operations
- separation between AI processing and export responsibilities
- support for user-friendly delivery of final results
- compatibility with preview and download workflows

The preview stage also depends on this design. Before download, the user is shown a temporary representation of the result through the PreviewView. This allows confirmation or optional return to refinement, preventing premature finalization of unsatisfactory outputs.

In this way, output generation is integrated into the system as the final stage of a controlled and reversible workflow rather than as a direct one-step response to AI processing.

A detailed sequence-level representation of the final download and export workflow is provided in Appendix H. A detailed sequence-level representation of the watermark application workflow is provided in Appendix I.

6.12 Error Handling and Failure Management Design

Error handling in Invisio is designed to provide predictable system behavior and safe failure responses. Since the system performs file validation, AI inference, session management, and user interaction across multiple layers, failure conditions must be handled in a structured and consistent manner.

The design recognizes three broad categories of errors:

- **Validation errors:** These occur when user input violates expected constraints, such as unsupported file formats, excessive file size, or corrupted files.

- **Processing errors:** These occur when AI-related operations fail, such as model inference problems, invalid intermediate states, or incompatible processing parameters.
- **System errors:** These refer to backend or infrastructure issues, such as internal exceptions or unavailable services.

To ensure safe handling of such situations, the system returns structured error responses rather than exposing internal details. Error messages are written in a human-readable form, while internal stack traces or implementation-specific details are hidden from the client. This protects both usability and security.

The `ErrorView` in the frontend is responsible for presenting these problems to users in a clear and understandable form. The backend controllers classify and propagate errors using a consistent response structure so that the frontend can react appropriately.

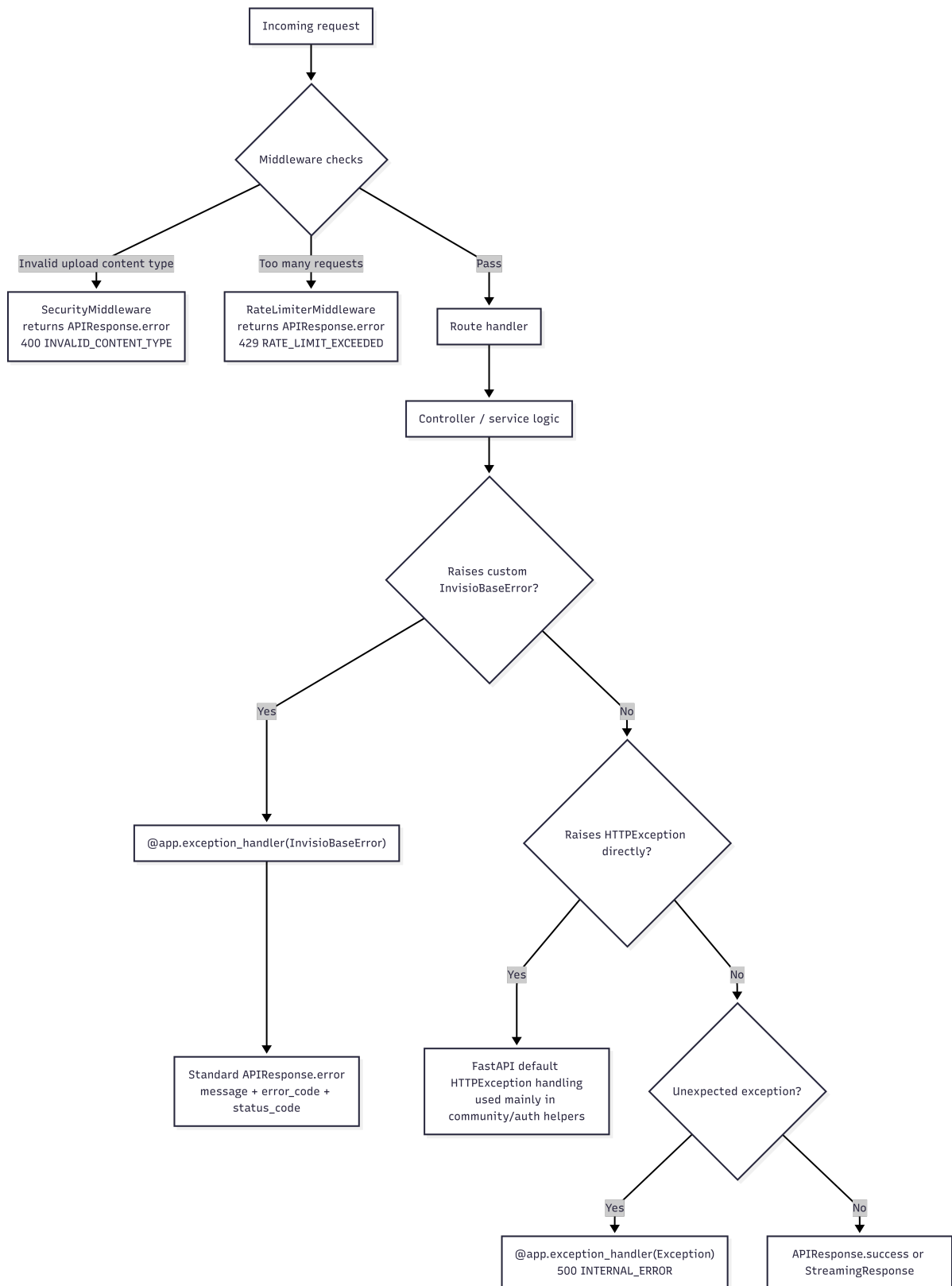


Figure 18: Error Handling and Exception Flow in the Invisio System

Figure 18 illustrates how error conditions propagate through the system and how exception handling is structured across architectural layers. The diagram makes explicit

how validation failures, processing exceptions, and internal system faults are intercepted, classified, converted into controlled backend responses, and ultimately presented to the user through the frontend error-handling path. This representation complements the textual design discussion by showing that failure management in Invisio follows a defined and bounded flow rather than ad hoc exception behavior.

The system is also designed to fail gracefully. When invalid input is detected, processing is halted before AI resources are engaged. When content fails safety checks, the request is rejected before entering the editing pipeline. When runtime processing errors occur, the system returns a controlled failure response and preserves architectural consistency rather than entering an undefined state.

6.13 Security-Oriented Design Details

Security considerations are embedded into the detailed design of Invisio rather than added as external protections after implementation. The system processes user-uploaded images, which makes secure handling of input and controlled access to resources especially important.

The main security-oriented mechanisms in the design include:

- secure communication over HTTPS
- input validation before processing
- file extension and MIME-type verification
- request interception before controller execution
- rate limiting to reduce abuse risk
- NSFW detection before further AI processing
- restricted exposure of internal processing details in error responses

The SecurityController plays a central role in enforcing request-level protections. It acts as an interceptor that checks authorization-related conditions where applicable and prevents invalid or malicious requests from reaching the business logic.

A particularly important design rule is that AI processing modules do not require unrestricted external access during inference. This reduces the attack surface and keeps model execution isolated from external network dependencies.

From a privacy-security perspective, the avoidance of persistent storage in the default workflow also acts as a protective design decision. Since user images are not retained longer than necessary, the risk associated with unauthorized access to stored image data is significantly reduced.

6.14 Community Feature Design

In addition to its core editing workflow, Invisio includes the conceptual design of a community-oriented sharing feature. This feature allows users to publish selected edited images for public browsing and interaction, provided that sharing is explicitly initiated by the user.

The design introduces the `CommunityImage` entity as an extension of `ResultImage`. In addition to processed image data, this entity stores metadata such as owner identifier, sharing timestamp, view count, and like count. This makes it suitable for use in a community gallery context.

The `CommunityView` on the frontend is responsible for presenting shared images and supporting lightweight interactions such as viewing and liking. From a design standpoint, this feature is intentionally separated from the core editing workflow. Users are not required to interact with community functionality in order to use the main editing features.

An important design requirement of this feature is consent-based persistence. Images are not shared automatically. Only the final output chosen by the user may be published, and session-related or personal metadata should be removed before publication. This ensures that the community feature does not violate the project’s overall privacy-by-design principles.

6.15 Design Consistency with Project Goals

The detailed system design of Invisio is consistent with the project’s original objectives and architectural principles. Each major design decision corresponds to a core project goal.

The use of modular AI processing components supports extensibility and maintainability. In particular, the Strategy Pattern applied through the `AIModel` abstraction ensures that different AI-based editing behaviors can be introduced and managed in a uniform way. This strengthens the model-agnostic nature of the architecture and supports future expansion without requiring structural redesign.

Similarly, the Facade Pattern applied through the `EditingController` supports usability and separation of concerns by hiding the internal complexity of the AI processing workflow behind a single, simplified coordination interface. This keeps the frontend interaction model clean and prevents direct dependency on low-level processing details.

The MVC and layered architectural structures support clarity, separation of concerns, and testability. Session-based temporary data handling supports privacy-by-design and data minimization. The refinement workflow supports the human-in-the-loop principle by combining AI automation with limited user correction.

Likewise, the standardized interfaces between controllers and AI modules reduce implementation ambiguity and allow the architecture to remain flexible. Structured error

handling and request validation support system reliability, while security-oriented processing constraints reduce misuse and exposure risks.

Taken together, the detailed design demonstrates that Invisio is not merely a collection of AI features, but a coherent software system built around usability, privacy, modularity, and controlled intelligence.

7. Engineering Decisions & Trade-offs

The design of Invisio required balancing multiple competing objectives related to usability, performance, privacy, security, and maintainability. As an AI-powered, web-based image editing system, Invisio operates in a problem space where technical optimization alone is not sufficient. Instead, the system must simultaneously satisfy user expectations, computational constraints, and ethical responsibilities.

For this reason, the engineering process involved a series of deliberate trade-off decisions. Rather than maximizing a single attribute such as model accuracy or feature richness, the design prioritizes a balanced architecture that remains practical, scalable, and user-friendly.

The following subsections describe the main trade-offs considered during the design and development of the system, together with the rationale behind the final design choices.

7.1 Automation vs User Control

One of the most fundamental design decisions in Invisio concerns the balance between automation and user control. AI-based image editing systems typically aim to automate complex editing tasks in order to simplify the user experience. However, fully automated systems may generate inaccurate results in difficult scenarios such as fine object boundaries, overlapping visual regions, or unusual lighting conditions.

Providing extensive manual control could improve correction accuracy, but it would also increase interface complexity and require users to possess technical editing skills. Such an approach would conflict with the project’s goal of building an accessible system for non-technical users.

To resolve this trade-off, Invisio adopts a hybrid human-in-the-loop workflow. AI performs the primary editing operations, while users are given limited corrective capabilities through structured mask refinement. This ensures that the system retains the efficiency of automation while still allowing users to improve outputs when needed.

The final design therefore does not aim for complete automation or full manual flexibility. Instead, it intentionally positions the system between these two extremes in order to maximize usability without sacrificing result quality.

7.2 Accuracy vs Performance

Another major trade-off in the system design concerns the relationship between output quality and computational performance. In image editing systems based on deep learning, larger and more sophisticated models often produce better visual results. However, these models also require more memory, longer inference time, and stronger hardware support.

Because Invisio is designed as an interactive web-based application, predictable and acceptable response times are essential for a satisfactory user experience. Excessive pro-

cessing latency would reduce usability and make the system impractical in real-world use.

For this reason, the design prioritizes bounded and stable performance over the use of the heaviest possible models. AI operations are selected and structured in a way that supports responsive interaction in a web environment. This does not mean that accuracy is ignored; rather, the system aims for a balanced level of output quality that remains computationally feasible.

As a result, the chosen design reflects an engineering compromise: the system favors practical deployability and usability over maximum theoretical visual performance.

7.3 Privacy vs Personalization

Modern intelligent applications often rely on persistent user data in order to provide personalized experiences, such as edit history, user preferences, adaptive recommendations, or model fine-tuning. While these features can improve convenience, they require long-term storage and analysis of user data.

In the context of image editing, such persistence introduces significant privacy, ethical, and legal concerns. User-uploaded images may contain personal, biometric, or sensitive content. Persistent retention of this data would increase the attack surface of the system and complicate compliance with regulations such as GDPR and KVKK.

Invisio intentionally resolves this trade-off in favor of privacy. The system follows a privacy-by-design architecture in which images are processed only within temporary sessions and are not stored permanently by default. Users are not required to create accounts for the standard workflow, and no long-term behavioral tracking is implemented.

This decision limits certain personalization opportunities, but it greatly strengthens the system’s data minimization strategy and reinforces user trust. The result is a system that prioritizes responsible handling of user data over convenience features that depend on persistence.

7.4 Security vs Usability

Security mechanisms are necessary in any system that receives and processes user-generated content, especially in a web-based AI platform. At the same time, overly restrictive security controls may negatively affect the user experience by increasing friction and reducing accessibility.

For example, strict authentication requirements, aggressive request restrictions, complex verification steps, or highly limited upload policies could improve protection but make the system less convenient for legitimate users.

Invisio addresses this trade-off by applying security-by-default while avoiding unnecessary barriers. Secure communication is enforced through HTTPS, uploaded files are validated before processing, content safety checks are applied, and request-level controls

such as rate limiting are included in the architectural design. However, the default workflow does not require mandatory account creation, and interaction remains simple for the end user.

This balance ensures that the system remains accessible and easy to use while still applying essential protections against invalid input, misuse, and unsafe content.

7.5 Stateless Architecture vs System Efficiency

Invisio is designed according to a stateless interaction philosophy in which each request is handled independently and long-term cross-session state is avoided. This design supports scalability, architectural simplicity, privacy, and easier deployment in distributed environments.

However, statelessness also introduces inefficiencies. Without persistent caching or retained session history across user interactions, the system may need to repeat certain computations or reinitialize parts of the workflow for similar requests. This can reduce efficiency compared with systems that retain more operational state.

The project addresses this trade-off by using temporary in-session handling while still avoiding long-term persistence. During active processing, session context is used to maintain the continuity necessary for the editing workflow. At the same time, once the workflow ends, the related data is removed to preserve the intended privacy guarantees.

This decision reflects a deliberate architectural preference: stronger privacy and clearer system boundaries are prioritized over the efficiency gains that might result from persistent cross-session optimization.

7.6 Model Generalization vs Task-Specific Optimization

Another important design question concerns whether the system should rely on a unified general-purpose AI model or on separate task-specific models for different editing capabilities.

A general-purpose model could reduce architectural complexity and potentially simplify deployment. However, different image editing tasks such as segmentation, inpainting, colorization, and style transfer have distinct computational requirements and often benefit from specialized model structures.

For this reason, Invisio adopts a task-based modular model design. Each major editing operation is represented by a dedicated processing component derived from the shared `AIModel` abstraction. This approach is consistent with the Strategy Pattern used in the system, where different processing behaviors are encapsulated as interchangeable model strategies.

The advantage of this design is that it allows each task to evolve independently, supports targeted model improvement, and avoids forcing fundamentally different editing problems into a single shared solution. The trade-off is increased architectural complexity

compared with a monolithic model, but this complexity is managed through abstraction and factory-based model selection.

As a result, the system prioritizes modularity, flexibility, and task-specific quality over architectural minimalism.

7.7 Feature Richness vs Interface Simplicity

A common challenge in product-oriented system design is deciding how many features to expose directly to users. In image editing applications, adding more controls, parameters, and customization options may increase flexibility, but it can also overwhelm users and reduce interface clarity.

Since Invisio is designed for non-technical users, interface simplicity is treated as a primary usability requirement. Users are expected to express editing intent through high-level operations rather than manipulate low-level technical parameters.

Accordingly, the system deliberately limits the degree of feature exposure. Advanced model settings, numerical controls, and pixel-level editing tools are not directly presented in the user interface. Instead, the interface is organized around understandable editing goals such as removing an object, enhancing an image, or applying a style.

This trade-off sacrifices some expert-level configurability, but it results in a much more accessible and coherent user experience. It also aligns with the overall project vision of building an AI-assisted tool rather than a full professional editing suite.

7.8 Modularity vs Implementation Complexity

The architectural design of Invisio strongly emphasizes modularity. The system separates concerns across layers, controllers, views, sessions, and AI processing modules. It also uses design patterns such as Strategy and Facade in order to organize behavior cleanly.

This modularity provides important long-term advantages. It improves maintainability, supports extensibility, enables isolated testing, and makes the system easier to understand from a software engineering perspective.

At the same time, modularity increases design and implementation complexity. A highly modular system often requires more interfaces, more abstractions, and more coordination logic than a simpler monolithic implementation.

Invisio resolves this trade-off in favor of modularity because the project is not intended as a short-lived prototype only; it is designed as a structured and extensible system. The use of the `AIModel` abstraction, `AIModelFactory`, specialized controllers, and layered architecture demonstrates this preference.

The resulting system may be more complex internally than a minimal implementation, but it is significantly stronger in terms of software engineering quality and future evolution.

7.9 Ethical Capability vs Misuse Risk

AI-powered image editing systems inherently carry ethical risks. Features such as beautification, object removal, or image transformation can be valuable and creative, but they may also be misused to generate misleading content, reinforce harmful beauty standards, or manipulate images irresponsibly.

Ignoring these risks would simplify the design process, but it would also make the system less responsible and less aligned with contemporary expectations for ethical AI development.

Invisio addresses this trade-off by incorporating safety and responsibility into its design. NSFW detection is included as a validation step, privacy-preserving data handling is adopted by default, and the system’s editing workflow is constrained to avoid encouraging unrestricted or deceptive manipulation.

This does not eliminate all misuse risk, but it demonstrates that ethical concerns were considered as part of the engineering process rather than treated as an afterthought. The final design therefore aims to preserve beneficial capability while limiting obvious misuse pathways.

7.10 Summary of Engineering Decisions

The engineering decisions in Invisio reflect a consistent design philosophy centered on balance rather than extremity. Across multiple dimensions—automation, performance, privacy, security, usability, modularity, and ethics—the project avoids one-sided optimization and instead adopts solutions that support a coherent and deployable system.

These trade-offs are not incidental; they define the identity of the platform. The hybrid workflow reflects the balance between automation and control. The session-based architecture reflects the balance between privacy and convenience. The modular AI design reflects the balance between extensibility and implementation complexity. The interface design reflects the balance between capability and simplicity.

Taken together, these decisions show that Invisio was engineered not only as an AI application, but as a complete software system shaped by practical, architectural, and ethical considerations.

8. Security & Privacy Design

Security and privacy are not treated as secondary concerns in the design of Invisio. Since the system processes user-uploaded images, which may contain personal, sensitive, or visually identifiable information, the architecture has been designed from the beginning with protection, minimization, and controlled access in mind.

Rather than adding security mechanisms after defining the functional workflow, Invisio integrates both security-by-default and privacy-by-design principles directly into the system structure. This means that data protection, safe communication, controlled processing, and responsible handling of user content are embedded into the architecture at every stage of the editing lifecycle.

This design choice is especially important for an AI-powered image editing platform, because such systems not only process visual data but may also create, transform, or reconstruct sensitive content. As a result, privacy and security in Invisio are not only technical requirements but also ethical and professional obligations.

8.1 Privacy-by-Design Approach

Invisio follows a privacy-by-design approach in order to reduce data exposure and align the system with modern data protection expectations. The core principle of this approach is that user data should be processed only to the extent necessary for providing the requested functionality and should not be retained beyond that purpose.

In the default workflow, user-uploaded images are processed only within a temporary session context. The system is designed so that images are not stored permanently by default, and users are not required to create persistent accounts in order to use the primary editing functions. This minimizes the amount of personally relevant data collected or retained by the platform.

Another important aspect of the privacy-by-design model is the restriction of unnecessary data flow. Only the data required for the selected editing operation is carried through the processing pipeline. Long-term behavioral tracking, edit history persistence, and hidden profiling mechanisms are intentionally excluded from the system design.

This approach improves user trust, reduces the long-term security burden of the platform, and supports compliance-oriented thinking in relation to legal frameworks such as GDPR and KVKK.

8.2 Data Lifecycle

A central privacy mechanism in Invisio is the use of session-based processing. Every editing workflow takes place within a temporary runtime context that is created when the user uploads an image and is terminated after the workflow is completed or the session expires.

Within this session, the system may temporarily hold:

- the uploaded image
- selected editing requests
- intermediate processing artifacts such as masks
- generated output images

This information is treated as ephemeral data. It exists only for the duration required to complete the user's requested operation and is not intended for permanent retention. Once the session is finalized, the related data is removed through cleanup procedures managed by the session lifecycle logic.

This design significantly reduces the risks associated with storing user-generated visual content. By ensuring that data is not retained longer than necessary, the system limits the consequences of unauthorized access, accidental persistence, or long-term misuse.

8.3 Secure Communication

All communication between the frontend and backend in Invisio is designed to occur over secure communication channels. The system assumes the use of HTTPS in order to protect uploaded images, processing requests, and returned results while they are in transit between the client and server.

This decision is particularly important because the transferred data may contain sensitive visual content. Without transport-level protection, uploaded images and API responses could be exposed to interception or unauthorized observation.

By enforcing secure communication as a default architectural requirement, the system protects both confidentiality and integrity during transmission. It also aligns with widely accepted software engineering and web security practices for applications that handle user-generated content.

8.4 Input Validation & Safety

Since Invisio allows users to upload files to the system, input validation is a critical part of the security design. Uploaded content must be verified before it enters the AI processing pipeline in order to prevent malformed, unsupported, or potentially malicious inputs from consuming system resources or causing undefined behavior.

The validation process is designed to include:

- file format verification
- file size checking
- basic integrity validation

- type restrictions for supported image formats
- rejection of invalid or corrupted uploads

This validation occurs before AI inference begins. As a result, invalid requests are stopped early, which improves both security and efficiency. Preventing unsupported files from reaching deeper layers of the system also reduces the chance of processing failures and protects backend components from unnecessary exposure.

From a user perspective, validation failures are communicated through structured and understandable error responses. This ensures that protective checks do not appear as unexplained system failures.

8.5 Request Protection and Access Control

The system design includes request-level protection mechanisms in order to reduce abuse and ensure controlled access to processing resources. These protections are especially important in AI-powered systems, where backend operations may be computationally expensive and vulnerable to misuse through repeated or malformed requests.

The `SecurityController` acts as an interceptor-style protection component in the architecture. It is responsible for validating requests before they reach the core processing logic. Depending on the deployed configuration, this protection may include token validation, request inspection, and enforcement of rate-related constraints.

Rate limiting is an important part of this strategy because it helps prevent excessive request frequency from exhausting computational resources or degrading service quality for legitimate users. This is particularly relevant in systems that depend on limited GPU-backed processing capacity.

At the same time, the project does not require mandatory user authentication for the default academic workflow. This reflects a deliberate balance between accessibility and protection. The architecture, however, remains compatible with stronger authentication or authorization mechanisms if they are introduced in future versions.

8.6 Content Safety and NSFW Detection

A distinctive part of Invisio’s security and ethical design is the inclusion of content safety checks before further image processing is allowed. Since the system is capable of transforming and enhancing images, it is important to prevent misuse involving inappropriate or restricted visual content.

For this reason, the architecture includes an `NSFWDetectionModel` that acts as a safety validation component. Uploaded images are checked before entering the main editing pipeline. If the content is classified as inappropriate or restricted, the system blocks further processing and returns a controlled warning or rejection response.

This mechanism serves multiple purposes. It reduces the risk of the platform being used for unsafe or harmful visual manipulation, supports ethical boundaries in AI usage, and helps align the system with responsible content-handling expectations.

The NSFW control stage is therefore not just a functional filter, but part of the broader trust and safety architecture of the system.

8.7 Data Minimization and Limited Exposure

Invisio follows the principle of data minimization throughout its architecture. This principle requires the system to collect, process, and retain only the minimum amount of data necessary to fulfill the requested editing operation.

In practical terms, this means that the platform does not request unnecessary personal information, does not retain images by default, and does not maintain long-term behavioral histories for ordinary usage. Only operationally necessary data is passed through the workflow, and this data is scoped to the active session whenever possible.

The principle of limited exposure also applies to logging, monitoring, and error handling. System logs are designed to avoid storing raw image data or unnecessary user identifiers. Error messages shown to users are human-readable but do not reveal internal implementation details that could create security risks.

This approach strengthens both privacy and security by reducing the amount of sensitive information available within the system at any given time.

8.8 Optional Persistence and Consent-Based Sharing

Although the default Invisio workflow avoids persistent storage, the system design recognizes that certain optional features may require controlled data retention. The most important example is community sharing.

If a user explicitly chooses to share a processed image, the resulting output may be retained as public content in a dedicated community context. This persistence differs fundamentally from the default editing workflow because it is based on explicit user action and applies only to the final selected result rather than the full session history.

In such cases, the system design requires that personal or session-related metadata be removed before publication. This ensures that optional persistence does not violate the privacy principles of the main workflow.

This distinction between transient session data and explicitly authorized persistent output is important. It allows the system to support richer features without weakening its overall privacy model.

8.9 Error Handling as a Security Mechanism

Error handling in Invisio is also part of the security design. Poorly designed error reporting can expose internal system details, reveal backend structure, or unintentionally assist

malicious users in probing the platform.

To avoid this, the system returns structured and controlled error responses. These responses provide enough information for the user to understand what went wrong, such as invalid input or processing failure, but do not expose stack traces, model internals, or infrastructure-specific details.

This design improves security by limiting information leakage while also preserving usability. Users receive meaningful feedback, and the frontend can handle failures in a predictable way without requiring sensitive backend details.

As a result, failure behavior in Invisio is intentionally designed to be informative for legitimate users but unhelpful for hostile probing.

8.10 Security and Privacy Benefits of the Architecture

The architecture of Invisio provides several built-in advantages from a security and privacy perspective.

First, the session-based and non-persistent data handling model minimizes long-term exposure of user-uploaded content. Second, the separation between frontend interaction and backend AI processing confines sensitive operations to controlled server-side components. Third, structured validation and safety checks prevent problematic content from reaching deeper stages of the system. Fourth, the layered and modular design makes it easier to apply targeted protections at different parts of the architecture.

In combination, these decisions result in a system that is not only functional but also responsibly engineered. Security and privacy are reinforced by the architecture itself rather than depending solely on external policies or post-development safeguards.

8.11 Limitations and Areas Requiring Clarification

While the security and privacy design of Invisio is strong at the architectural level, certain implementation-level details should be documented explicitly in the final report to ensure complete transparency.

These include:

- whether temporary images were kept only in memory or also written to backend storage before cleanup
- the exact transport security configuration used in deployment
- whether authentication was actually implemented or only architecturally supported
- how rate limiting was enforced in practice
- whether optional persistence features such as community sharing were fully implemented or remained conceptual

These details do not weaken the architectural design, but they should be clarified to distinguish intended design behavior from final implementation status.

9. Implementation

The implementation of Invisio translates the architectural and design decisions of the project into a working web-based AI-assisted image editing system. The implementation process focused on preserving the core principles defined during analysis and design, including modularity, usability, privacy-by-design, and controlled AI integration.

From an implementation perspective, the system was developed as a client-server application in which the frontend and backend are separated and communicate through API-based interactions. The frontend is responsible for user interaction and visual presentation, while the backend manages request orchestration, temporary data handling, and AI-based image processing workflows.

A key implementation goal was to keep the system aligned with its intended design abstractions. For this reason, the implementation preserves the separation between presentation logic, control logic, and processing logic, instead of combining all responsibilities in a monolithic structure. Similarly, AI operations are handled as dedicated processing modules rather than ad hoc scripts embedded into request handlers.

The implementation also reflects the system’s privacy-oriented philosophy by ensuring that user-uploaded images are handled within temporary processing workflows rather than persistent storage-driven interaction models.

9.1 Technologies Used

The implementation of Invisio relies on a modern web and AI development stack chosen to support modularity, performance, maintainability, and practical responsiveness in an interactive image-editing environment.

The frontend is implemented using React and TypeScript. This combination provides a component-based user interface model together with stronger type safety, improved maintainability, and more predictable frontend development. The frontend stack is supported by Vite as the build tool and by Tailwind CSS for utility-oriented styling. Additional supporting tools such as PostCSS, Autoprefixer, Zustand, React Router, Axios, Framer Motion, and Lucide React contribute to styling compatibility, state handling, navigation, API communication, interaction design, and interface consistency across the platform.

The backend is implemented in Python using FastAPI. This framework was selected because it provides a clean structure for API development, strong support for asynchronous request handling, and compatibility with Python-based AI processing libraries. It also fits well with the architectural requirement of clear separation between API coordination and model execution. The backend stack is further supported by Uvicorn for

ASGI execution, Pydantic for structured validation, python-multipart for upload-oriented request handling, and async-compatible database tooling where persistence is required.

The AI and image processing layer is implemented through a hybrid stack that combines deep learning-based models with deterministic computer-vision algorithms. The system uses task-specific components such as U2-Net for segmentation, MobileSAM for light-weight mask-oriented region extraction, LaMa for inpainting, AdaIN for style transfer, EDSR-based enhancement through OpenCV DNN utilities, ECCV16-based colorization, and NSFW-oriented image classification for content safety. These learned components are supported by OpenCV, MediaPipe, NumPy, and Pillow for classical image processing, facial landmark analysis, preprocessing, postprocessing, geometric transformation, blending, denoising, sharpening, and other utility operations.

This hybrid implementation approach is important because it allows the system to avoid depending exclusively on heavy generative models in all workflows. Instead, learned and deterministic methods are combined in order to improve runtime efficiency, reduce hardware pressure in selected pipelines, and preserve responsiveness in a web-based human-in-the-loop editing environment.

For persistence and infrastructure support, the broader software stack includes PostgreSQL, SQLAlchemy, asyncpg, Docker, and Docker Compose. These technologies support optional persistent features such as authentication- and community-related data management, while the default editing workflow continues to prioritize temporary session-based handling in line with the project’s privacy-by-design principles.

The implementation and verification environment also includes development and testing tools such as pytest, httpx, Playwright, ESLint, and Git/GitHub. These tools improve testability, code quality, reproducibility, and collaborative maintainability across the project.

A supplementary configuration-level view of the implementation environment is also provided in Appendix J.

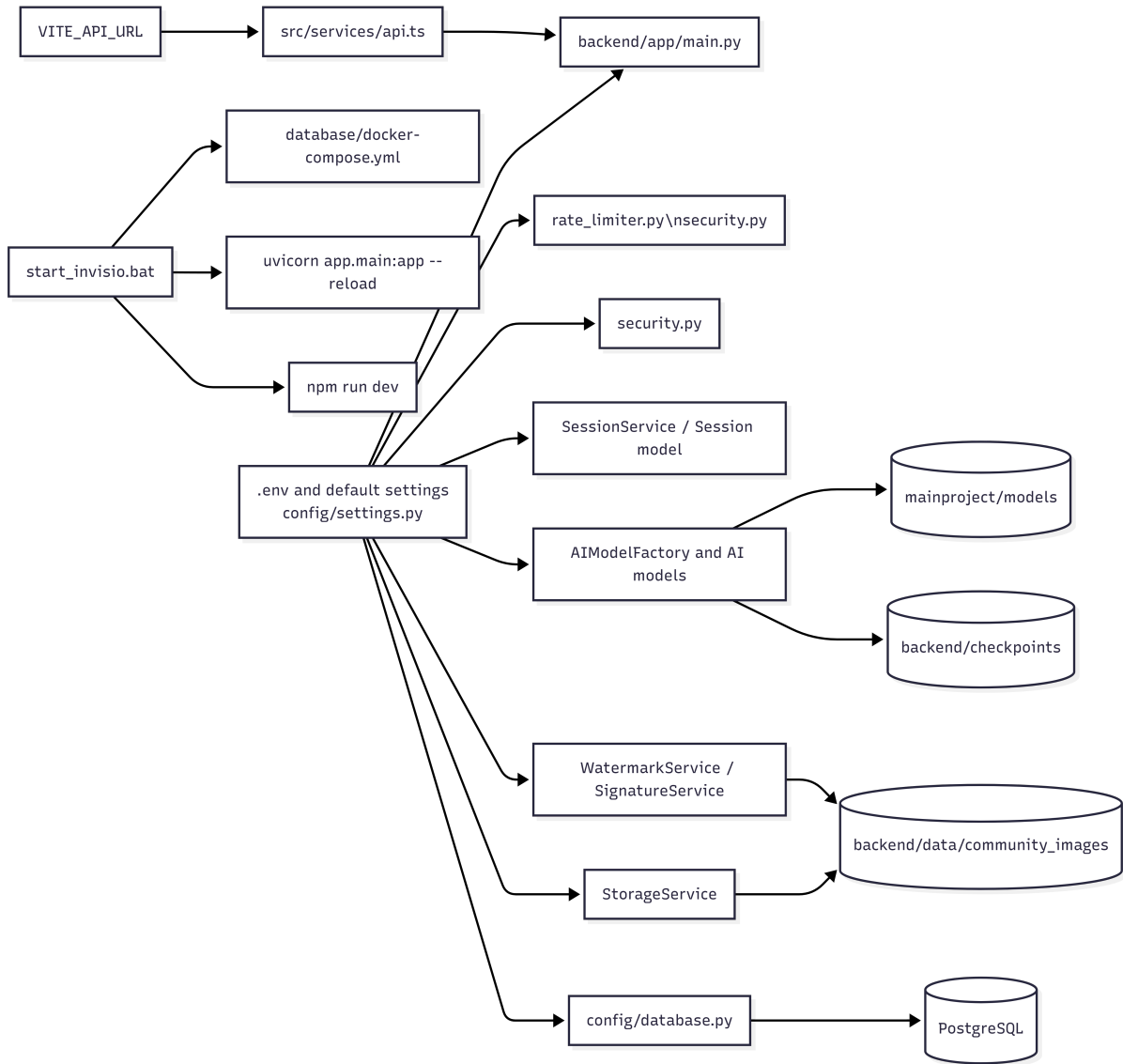


Figure 19: Configuration Dependency Diagram of the Invisio Implementation Environment

Figure 19 illustrates the configuration-oriented dependency structure of the implemented system. The diagram shows how frontend configuration elements, backend startup and execution files, environment settings, security-related modules, session and AI model services, storage components, local model resources, and database-related configuration are connected within the practical runtime environment. This representation complements the technology list by making explicit how the selected tools and configuration-bound modules interact in the implemented system.

This implementation-oriented dependency view is especially useful because it shows that the technology stack of Invisio is not merely a collection of independent tools, but a coordinated environment in which configuration, execution flow, model access, storage, and infrastructure components work together as part of the deployed application context.

9.2 Frontend Implementation

The frontend implementation of Invisio focuses on creating an accessible and task-oriented user experience. It is structured around reusable interface components corresponding to the main stages of the image editing workflow.

The primary frontend responsibilities include:

- collecting user input through file upload and feature selection interfaces
- sending image and editing requests to the backend
- displaying processing progress and intermediate feedback
- presenting preview results to the user
- enabling result download
- providing refinement interaction where applicable

The component structure follows the design described in the system architecture and detailed design sections. Views such as image upload, editing options, preview, refinement, and download are separated into dedicated interface units in order to maintain clarity and support modular development.

The frontend implementation deliberately avoids embedding business logic or AI-related operations into the user interface layer. Instead, it focuses on interaction flow and presentation while delegating validation, coordination, and processing logic to the backend. This implementation decision preserves the intended MVC-style separation of concerns and reduces coupling between interface behavior and backend processing details.

From a usability standpoint, the frontend is designed to present high-level editing actions rather than low-level technical controls. This ensures that the implemented system remains aligned with the original project goal of serving non-technical users.

9.3 Backend Implementation

The backend implementation acts as the operational core of the system. It is responsible for receiving and validating requests, managing temporary processing context, invoking the appropriate AI workflows, and returning results to the frontend in a structured manner.

In practice, the backend implementation includes mechanisms for:

- handling image upload requests
- validating uploaded file types and sizes
- creating and managing temporary sessions

- receiving editing requests and selecting the appropriate AI workflow
- coordinating preprocessing, inference, and postprocessing
- returning processed outputs and error states
- cleaning up session-related data after use

FastAPI provides the structural basis for implementing these operations in the form of clearly defined API endpoints and request handlers. The backend implementation is compatible with asynchronous request handling, which is important because some image processing operations may take noticeable time to complete.

The controller-based coordination logic described in the design phase is reflected in the backend implementation approach. Rather than merging all responsibilities into a single service or route handler, the implementation is organized around distinct operational responsibilities such as upload handling, editing coordination, session management, refinement, and output delivery.

This implementation structure helps preserve maintainability and makes it easier to map the final system back to its design documents.

9.4 AI Processing Implementation

The AI processing implementation is the part of the system responsible for executing image transformation tasks such as segmentation, inpainting, enhancement, style transfer, beautification, aging, and colorization. These operations are implemented as modular processing units that follow a shared execution structure while combining deep learning-based models with deterministic image-processing algorithms where appropriate.

At an implementation level, an AI processing workflow generally consists of the following stages:

- loading or accessing the required model or algorithmic processing pipeline
- preprocessing the input image into a model-compatible or operation-compatible representation
- executing inference or deterministic transformation
- converting the raw output into a usable image or mask result
- returning the processed output to the backend coordination layer

This staged structure reflects the abstract `AIModel` design defined earlier and supports consistency across different editing tasks. It also makes it easier to test, update, or replace individual processing modules without rewriting the surrounding system logic.

In practice, the implemented AI stack follows a hybrid architecture. Segmentation and region-aware mask generation rely on deep learning models such as `U2-Net` and

MobileSAM. Object removal and masked image reconstruction are handled through **LaMa**-based inpainting. Style transfer is implemented through an **AdaIN**-based pipeline, while grayscale colorization is supported through an **ECCV16** colorization architecture.

For portrait-oriented workflows, the implementation combines learned and deterministic methods. Face-aware region analysis is supported through **MediaPipe Face Landmarks**, which enables localized editing decisions in portrait enhancement and face-related processing. In beautification-related workflows, deterministic enhancement methods such as bilateral filtering and CLAHE are used to smooth skin regions while preserving important facial structure and local detail.

A particularly important implementation decision concerns the aging pipeline. Rather than relying on a heavier generative model at runtime, the implemented system adopts a more performance-oriented deterministic approach. In this workflow, masks obtained through lightweight segmentation support procedural wrinkle generation, texture modification, and tone manipulation in order to simulate aging or rejuvenation effects more efficiently.

Image enhancement and resolution-improvement workflows also follow a hybrid structure. Where appropriate, super-resolution support is provided through an **EDSR**-based pipeline operating via **OpenCV** DNN utilities. Additional deterministic methods such as Fast Non-Local Means and Unsharp Masking are used as supporting operations for denoising and sharpening in selected enhancement scenarios.

The implementation also includes a safety-oriented content validation stage through an **NSFW**-oriented image classification component. This prevents unsafe or restricted content from entering the main editing pipeline and reflects the project’s broader trust and safety goals.

The implementation of AI workflows must also account for practical constraints such as model loading time, GPU availability, input format normalization, memory usage, and output conversion. For this reason, the system deliberately avoids relying exclusively on heavy generative models in all workflows. Instead, it combines task-specific deep learning components with deterministic fallback or support algorithms in order to improve runtime efficiency and preserve responsiveness in an interactive web-based environment.

A detailed sequence-level representation of the object removal workflow is provided in Appendix B. A detailed sequence-level representation of the standalone dual-image background replacement workflow is provided in Appendix C. A detailed sequence-level representation of the standalone style transfer workflow is provided in Appendix D. A detailed sequence-level representation of the aging workflow is provided in Appendix E. A detailed sequence-level representation of the beautification workflow is provided in Appendix F. A detailed sequence-level representation of the resolution enhancement workflow is provided in Appendix G.

9.5 Implementation of Design Patterns

The implementation of Invisio reflects the design patterns introduced during the detailed design phase, particularly the Strategy Pattern and the Facade Pattern.

The Strategy Pattern is realized through the `AIModel` abstraction and its specialized processing modules. Each image editing capability is implemented as a distinct processing strategy that follows a common interface. This allows the system to select and execute different AI behaviors depending on the user's requested editing task without changing the higher-level control flow.

The Facade Pattern is reflected in the implementation role of the `EditingController`. AI-based image editing involves multiple internal operations, including request validation, session-based image retrieval, model selection, inference coordination, and output generation. Instead of exposing this internal complexity directly to the frontend, the `EditingController` provides a simplified entry point that coordinates the complete editing workflow.

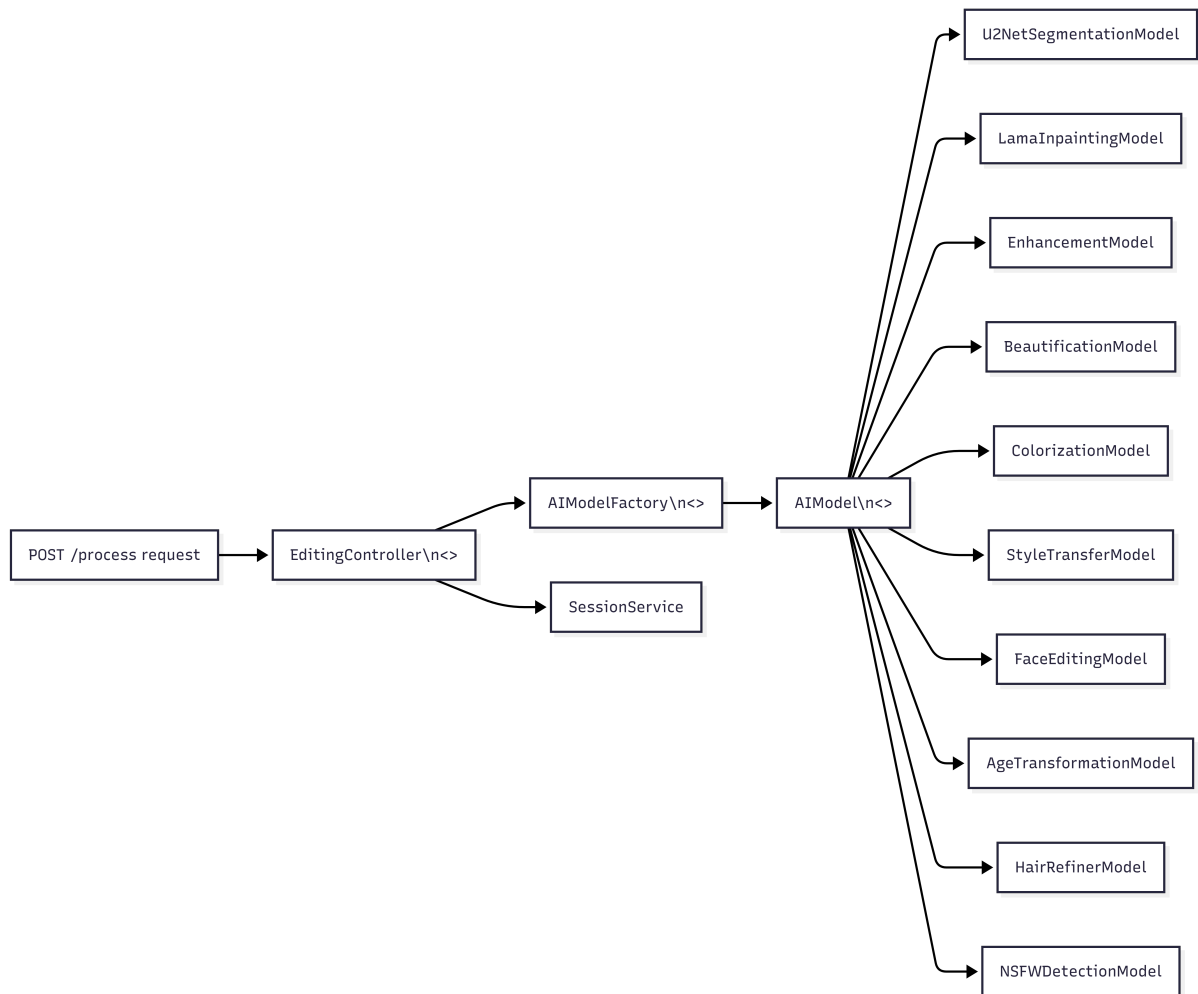


Figure 20: Facade, Factory, and Strategy Patterns in the Editing Subsystem

Figure 20 illustrates how the editing subsystem combines multiple design patterns in a

coordinated implementation structure. The diagram makes explicit that the `EditingController` acts as a facade over the editing workflow, the `AIModelFactory` is responsible for selecting and instantiating the appropriate model path, and the specialized AI model classes operate as interchangeable strategy implementations behind a shared processing abstraction. This representation clarifies that the editing subsystem was designed not as ad hoc controller logic, but as a pattern-driven and modular architecture.

These patterns improve the software quality of the implementation by increasing modularity, reducing coupling, and making the codebase more maintainable and extensible. They also ensure consistency between the design documents and the final implementation approach.

9.6 Session and Temporary Data Handling in Implementation

One of the most important implementation concerns in Invisio is how temporary data is handled during image processing. Since the project prioritizes privacy-by-design, the implementation is expected to preserve the principle that uploaded images are retained only for the time necessary to complete the requested workflow.

In the implemented workflow, temporary session context is used to maintain continuity between upload, processing, refinement, preview, and download stages. This session context allows the backend to associate the correct image and intermediate results with the active user interaction without relying on long-term persistence.

The implementation must therefore handle temporary storage, retrieval, expiration, and cleanup in a controlled manner. This is operationally important not only for privacy, but also for correctness, because different stages of the editing workflow depend on consistent access to the currently active data.

9.7 Integration of Frontend, Backend, and AI Components

A major implementation challenge in Invisio is the integration of three distinct but interdependent parts of the system: the user interface, the backend coordination layer, and the AI processing modules.

The frontend sends image uploads and editing requests to the backend through defined API calls. The backend validates these requests, resolves the intended workflow, and forwards the relevant data to the processing components. After the AI modules complete their work, the resulting output is returned through the backend and displayed by the frontend.

This integration must be consistent not only in terms of data format, but also in terms of workflow timing, state management, and failure handling. For example, uploaded files must be encoded and transferred correctly, processing state must remain associated with the correct session, and any errors generated by AI operations must be translated into responses that the frontend can present meaningfully.

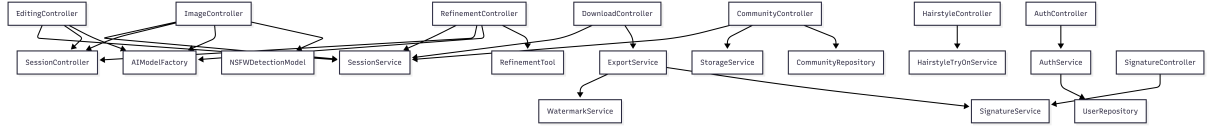


Figure 21: Service Interaction Diagram of the Invisio System

Figure 21 illustrates how the major services and coordination components of the system interact during execution. The diagram makes explicit the communication paths between frontend-facing request flow, backend orchestration logic, session-related handling, AI model access, storage-related services, and result-delivery mechanisms. This representation complements the integration discussion by showing that the system depends not only on isolated components, but on structured service-level collaboration across architectural boundaries.

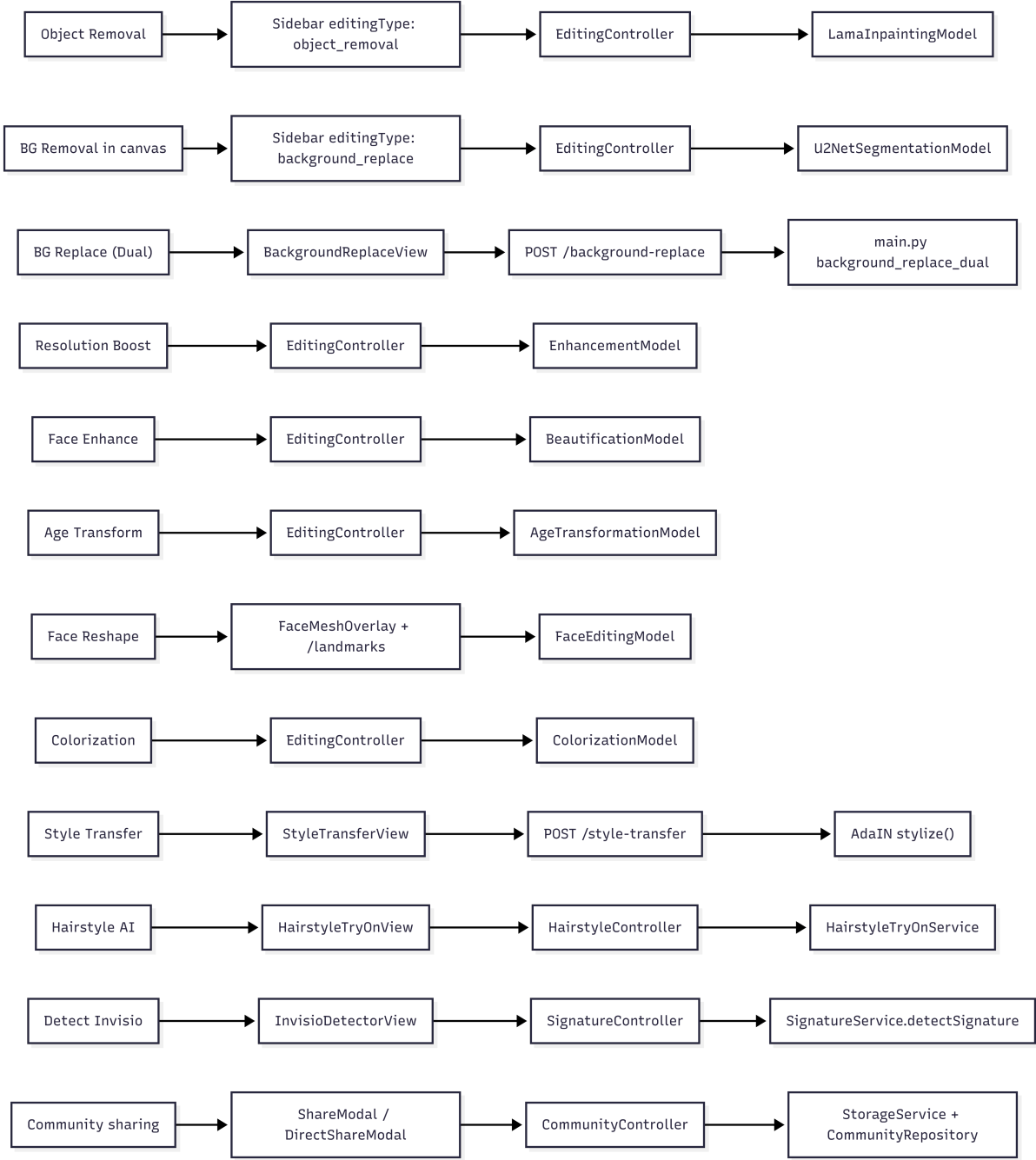


Figure 22: Feature-to-Implementation Mapping in the Invisio System

Figure 22 complements the service-level integration view by showing how user-facing features are connected to concrete implementation paths across the system. The diagram makes explicit the mapping between editing capabilities, their corresponding frontend entry points, backend controllers or API routes, and the AI models or services responsible for execution. In this way, it demonstrates that the implemented feature set of Invisio is not organized as a collection of isolated interface actions, but as a coordinated structure spanning presentation, orchestration, and processing layers.

Together, these diagrams show that the integration of Invisio operates at two related levels. The service interaction view explains how the main subsystems collaborate during

execution, while the feature-mapping view shows how specific user-visible capabilities are realized through those subsystem connections. This strengthens the implementation-level evidence that frontend behavior, backend coordination, and AI-based processing are aligned within a coherent system architecture.

The implemented system therefore depends on stable interfaces between these layers. This validates the importance of the API and interface standardization decisions made during the design phase.

A detailed sequence-level representation of the upload and generic processing workflow is provided in Appendix A. A supplementary feature-to-implementation mapping view is also provided in Appendix K. A supplementary service-level interaction view is also provided in Appendix L.

9.8 Deployment and Execution Environment

The implementation and testing of Invisio were carried out in an environment capable of supporting both web application execution and AI-based image processing workloads. Since model inference may benefit significantly from hardware acceleration, the execution environment is an important aspect of the system’s practical implementation.

According to the project documentation, the test environment includes a standard personal computer equipped with an AMD Ryzen 9 5900HX processor, 16 GB RAM, and an NVIDIA GeForce RTX 3060 laptop GPU. This environment is sufficient for experimenting with and evaluating AI-assisted image processing tasks in a local or development-oriented setting.

The software environment includes modern frontend and backend frameworks together with AI libraries and development tools. Containerization and reproducibility considerations are also reflected through the use of technologies such as Docker and Docker Compose in the broader system stack.

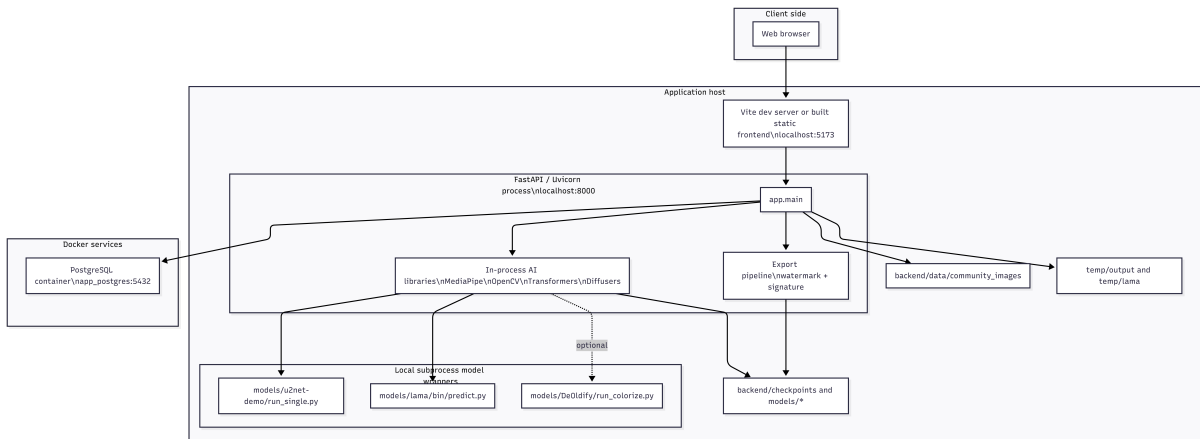


Figure 23: Deployment Diagram of the Invisio System

Figure 23 illustrates the deployment-oriented organization of the system, including

the main execution environment, application-side components, and their communication paths. The diagram makes explicit how the frontend, backend, processing modules, and supporting infrastructure are positioned within the runtime environment and how they interact during execution. This representation complements the implementation discussion by showing the system not only as a software design, but also as an executable deployment structure.

Although the system is designed conceptually for web-based deployment, the exact production deployment model should be described separately if the final implementation was actually deployed beyond a local or development environment.

9.9 Implementation Limitations and Practical Gaps

As in many academic software projects, the implementation scope of Invisio may differ in some respects from the full conceptual design described in earlier sections. Certain features may be fully functional, while others may remain partially implemented, prototyped, or documented mainly at the architectural level.

Potential implementation gaps that should be clarified in the final report include:

- which AI features were fully operational
- whether the refinement workflow was fully integrated end-to-end
- whether the community feature was implemented or remained conceptual
- whether authentication and advanced security controls were active in the final system
- whether the persistence model matched the original privacy-oriented design exactly in practice

Documenting these differences does not weaken the project. On the contrary, it improves the credibility of the report by clearly separating designed capabilities from fully realized implementation outcomes.

9.10 Consistency Between Design and Implementation

The implementation of Invisio reflects the major architectural ideas defined during the analysis, high-level design, and low-level design phases. The separation between frontend and backend, the modular organization of AI processing, the controller-oriented coordination model, the use of temporary sessions, and the application of Strategy and Facade patterns all demonstrate continuity between design and realization.

This consistency is important because it shows that the system was not implemented as a disconnected prototype, but as a software project guided by explicit engineering

decisions. Even where some parts of the conceptual design may remain incomplete in the final implementation, the overall structure preserves the intended design philosophy.

As a result, the implementation of Invisio can be understood as a practical realization of a carefully planned software architecture rather than an isolated collection of technical features.

10. Testing & Evaluation

10.1 Testing Objectives

The purpose of testing in the Invisio project is to verify that the implemented system behaves correctly, reliably, and securely under expected usage conditions. Since the platform integrates user interaction, backend coordination, session-based data handling, and AI-driven image processing, testing must evaluate both individual components and the complete workflow as a whole.

The testing process is designed to assess whether the system satisfies its functional requirements, maintains acceptable operational performance, and handles failure scenarios in a controlled manner. In particular, testing is important in Invisio because the correctness of the system depends not only on traditional software behavior but also on the quality and consistency of AI-assisted processing workflows.

The main testing objectives are:

- to verify that the core functional requirements are satisfied
- to ensure that frontend, backend, and AI modules operate correctly together
- to confirm that invalid inputs and unsafe content are handled securely
- to evaluate whether the system behaves reliably during full user workflows
- to identify implementation weaknesses, incomplete features, and future improvement areas

Testing in this project is therefore not limited to correctness validation; it also serves as a practical evaluation of the system’s robustness, usability, and architectural consistency.

10.2 Testing Strategy and Methodology

The testing strategy of Invisio is based on a multi-level evaluation approach in which different testing methods are applied to different parts of the system. This approach is necessary because the project includes user interface logic, backend request handling, session lifecycle management, and AI-powered processing modules that interact with one another.

The main testing methods considered in the project are:

- **Unit Testing** Used to verify the correctness of isolated modules such as file validation logic, session lifecycle functions, and selected AI-related processing functions.
- **Integration Testing** Used to evaluate whether independently developed system components work together correctly. This includes interaction between image upload, backend processing, preview generation, and result delivery mechanisms.
- **System Testing** Used to evaluate the complete end-to-end workflow of the application, including upload, processing, optional refinement, preview, and download.
- **Security Testing** Used to examine how the system behaves in the presence of invalid input, unsupported files, unsafe content, or potentially abusive request patterns.
- **Performance Testing** Used to assess practical responsiveness under different image sizes and processing workloads, with particular attention to AI inference time and temporary data cleanup behavior.
- **User-Oriented Testing** Used to evaluate the usability of the system from the perspective of non-technical users performing realistic editing tasks.

This layered testing strategy reflects the structure of the system itself. Since Invisio is both a software system and an AI-assisted application, evaluation must cover not only conventional program correctness but also workflow continuity, safety, and practical user experience.

10.3 Test Environment

The testing of Invisio was performed in a software and hardware environment capable of supporting both web application execution and AI-based image processing tasks. The objective of defining the test environment is to ensure that the observed results can be interpreted within the correct computational context.

According to the project test documentation, the hardware environment includes:

- **CPU:** AMD Ryzen 9 5900HX
- **RAM:** 16 GB
- **GPU:** NVIDIA GeForce RTX 3060 Laptop GPU

The software environment includes the major technologies used in the project stack, such as:

- **Operating System:** Windows
- **Frontend technologies:** React, TypeScript, Vite, Tailwind CSS, React Router, Axios, Framer Motion

- **Backend technologies:** Python, FastAPI, Uvicorn, SQLAlchemy, Pydantic, Pillow
- **AI and numerical processing tools:** PyTorch, NumPy
- **Database and infrastructure tools:** PostgreSQL, Docker, Docker Compose, asyncpg
- **Testing-related tools:** pytest, httpx

This environment is suitable for evaluating a development-stage AI-powered web platform and reflects the practical constraints under which the system was implemented and tested.

10.4 Functional Test Scope

The functional test scope of Invisio is derived from the system requirements defined earlier in the project. These tests focus on verifying whether the system supports its core user-facing and backend-supported capabilities as intended.

The main functional areas covered by testing include:

- **image upload and validation**
- **AI-based editing workflow initiation**
- **result preview generation**
- **processed image download**
- **object removal**
- **image enhancement**
- **neural style transfer**
- **portrait beautification**
- **background editing**
- **grayscale image colorization**
- **manual mask refinement**
- **hybrid editing workflow**
- **NSFW content detection**
- **temporary session-based data handling**
- **user authentication and access-related flows where applicable**

This coverage ensures that testing is aligned with the project's defined functional requirements rather than being limited to a small subset of visible interface actions.

10.5 Test Cases and Requirement Coverage

The project test plan defines a set of structured test cases mapped to the system's functional requirements. This traceability is important because it allows the development team to evaluate not only whether individual tests pass, but also whether the overall requirement set has been adequately exercised.

Representative test areas include:

- **s**uccessful upload of valid image files
- **r**ejection of unsupported file formats
- **v**erification of AI processing workflow for selected editing operations
- **c**orrect presentation of processed results in preview
- **s**uccessful download of generated images
- **p**erformance evaluation on large image files
- **s**ession-based deletion of temporary data
- **e**rror handling when processing is requested without a prior upload
- **v**alidation of object removal functionality
- **v**alidation of image enhancement behavior
- **v**erification of neural style transfer output generation
- **e**valuation of portrait beautification
- **v**alidation of background manipulation features
- **c**olorization of grayscale images
- **m**anual mask refinement behavior
- **h**ybrid workflow with refinement and regeneration
- **N**SWF or restricted content rejection
- **c**ommunity sharing behavior
- **a**uthentication-related interaction flows

The requirement traceability matrix in the test plan provides the basis for connecting these tests to formal requirement identifiers. This makes the evaluation process more systematic and improves the credibility of the report.

10.6 Test Results Summary

Based on the final testing status of the project, the previously pending tests can now be considered completed. This changes the overall evaluation of the system significantly, because the test results now show not only a stable core workflow but also validation of advanced AI-related and integrated system capabilities.

Tests verified as completed include:

- **upload** of valid images
- **rejection** of unsupported file types
- **full AI processing workflow verification**
- **preview display** of processed images
- **download** of processed outputs
- **performance testing** with large image files
- **session-based deletion** of temporary images
- **error handling** when processing is requested without prior image upload
- **object removal validation**
- **image enhancement validation**
- **style transfer verification**
- **beautification verification**
- **background editing verification**
- **grayscale image colorization**
- **manual mask refinement verification**
- **hybrid AI plus manual workflow verification**
- **NSFW or restricted content detection**
- **community sharing validation**
- **authentication-related interaction test**

With these results, the evaluation indicates that the system's major functional requirements have been exercised more comprehensively than suggested by the earlier test documentation. The completed test coverage now supports a stronger conclusion regarding the practical maturity of the project.

10.7 Evaluation of Core Functional Areas

The updated testing status allows a more confident assessment of the functional state of the system.

First, the core user-facing workflow is validated end-to-end. Upload, processing, preview, refinement where applicable, and download are all supported by completed tests. This indicates that the platform functions not merely as a set of isolated components, but as a coherent interactive workflow.

Second, the completion of AI-related feature tests strengthens the practical credibility of the system. Object removal, enhancement, style transfer, beautification, background editing, and colorization are no longer only architectural claims or partially tested functions; they are now part of the verified capability set of the system.

Third, the completion of refinement and hybrid workflow tests is particularly important because these are among the most distinctive aspects of Invisio. Their successful validation suggests that the system’s human-in-the-loop design is not only conceptually strong but also functionally realized.

Fourth, the completion of session-based data handling and performance-related tests provides stronger evidence that the system’s privacy-oriented and usability-oriented design choices were carried into implementation and evaluation.

Overall, the updated results support the conclusion that Invisio has progressed from a partially validated implementation foundation to a more comprehensively tested academic system.

10.8 Security and Validation Testing

Security-related testing in Invisio focuses on ensuring that the system rejects invalid or unsafe inputs before those inputs can consume significant backend resources or enter the AI processing pipeline.

The available documentation indicates that the following security-relevant behaviors were considered during testing:

- rejection of unsupported file formats
- validation failures when processing is requested without a valid uploaded image
- blocking of NSFW or restricted image content
- access-related or authentication-oriented checks
- request validation behavior
- IP-based rate limiting behavior at the backend protection layer
- authentication-route validation and account-oriented access checks

- controlled rejection of unauthorized or invalid security-sensitive requests through middleware and backend validation flow

These tests are important because they confirm that the system’s safety and correctness do not depend solely on successful usage scenarios. Instead, the design is also evaluated under failure and rejection conditions.

Particularly notable is the NSFW filtering test, which demonstrates that the project explicitly considered the misuse potential of AI-based image editing. This distinguishes the evaluation approach from purely feature-oriented test strategies.

In addition to the core validation cases, the final implementation also included security-oriented mechanisms such as IP-based rate limiting, authentication-related backend flows, and middleware-level request protection. For this reason, the security evaluation should be understood as covering not only invalid input and unsafe content rejection, but also backend-side protection against abusive request patterns and access-related misuse scenarios.

10.9 Performance and Scalability Considerations in Testing

Performance testing is especially important in Invisio because the system depends on AI-based image processing, which may introduce noticeable computational delay. A web-based platform must provide acceptable responsiveness even when deep learning models are involved.

The project documentation identifies performance-related concerns such as:

- processing time for large image files
- time required for AI editing operations
- behavior under multiple requests
- efficiency of temporary session cleanup

With the completion of previously pending performance-related testing, the project now has stronger evidence that the system was evaluated not only for correctness but also for practical responsiveness. This is important because usability in AI-assisted systems depends heavily on how long users must wait for results.

The successful completion of large-file and workflow-related performance testing suggests that the system was assessed under more realistic operating conditions and that performance was treated as an engineering requirement rather than an afterthought.

10.10 User-Centered Evaluation

In addition to technical verification, the test plan includes user-oriented evaluation. This is important because the project’s primary goal is not only to build a working AI system, but also to create a usable editing platform for non-technical users.

The user-centered evaluation focuses on how effectively users can:

- **upload** images
- **select** AI editing operations
- **understand** the workflow
- **adjust** results through refinement tools
- **download** final outputs

This type of evaluation is particularly relevant for Invisio because the system intentionally positions itself between fully automated tools and professional editing software. As a result, usability is a core success criterion rather than a secondary concern.

10.11 Testing Limitations

Although the final testing status of Invisio is stronger than the earlier documented draft state, some limitations should still be acknowledged in the evaluation section.

The main remaining limitations are no longer about missing completion of major functional tests, but about the depth and reporting style of the evaluation. These limitations may include:

- **lack** of detailed quantitative performance metrics in the written report
- **limited** reporting of user-centered evaluation results in measurable form
- **possible** differences between test completion status and the level of formal evidence recorded for each test
- **the** need to distinguish between feature correctness and output quality under highly diverse image conditions
- **the** need for more advanced long-term or large-scale stress testing if the system were to move closer to production use

These limitations do not change the fact that the major test set is now complete, but they remain relevant for accurately describing the maturity level and scope of the evaluation.

10.12 Overall Evaluation

Overall, the testing and evaluation process indicates that Invisio has moved beyond a partially verified academic prototype and now demonstrates a more comprehensively tested implementation across both core workflow and advanced functional areas.

The available results show that the project team approached testing systematically, with traceable test cases connected to requirements and multiple categories of evaluation covering correctness, integration, security, performance, and user interaction.

The completion of tests related to advanced AI features, refinement workflow, session cleanup, performance behavior, and community-related functionality substantially strengthens the final assessment of the system. These results suggest that the project is not limited to a strong design vision alone, but also includes meaningful implementation verification across its key subsystems.

Therefore, the overall evaluation supports a positive conclusion: Invisio is a technically coherent and functionally validated AI-assisted image editing system whose major architectural ideas are reflected in implemented and tested workflows.

11. Impact Analysis

The Invisio project is not only a technical software system but also a platform with potential social, economic, ethical, and technological implications. Since the system combines artificial intelligence, image manipulation, user-generated content, and privacy-sensitive processing, its impact extends beyond software functionality alone.

Impact analysis is important in this project because AI-assisted image editing systems can influence how users create, modify, and share visual content. These systems may generate benefits such as accessibility, productivity, and creative support, but they may also introduce risks related to misuse, unrealistic representation, or privacy concerns.

For this reason, the impact of Invisio should be evaluated in a balanced manner. The following subsections examine the project from multiple perspectives, including technological, economic, social, ethical, and environmental impact.

11.1 Technological Impact

From a technological perspective, Invisio contributes to the growing class of AI-assisted creative systems that make advanced visual processing more accessible to general users. The project demonstrates how modern deep learning techniques can be integrated into a structured software architecture to provide practical editing capabilities through a web-based interface.

One of the main technological impacts of the system is the combination of AI automation with limited user correction. This hybrid human-in-the-loop model offers an alternative to both fully manual and fully automated editing paradigms. In this sense, the project contributes to the broader idea that AI systems can be designed not only for automation, but also for collaborative interaction.

Invisio also has architectural value as a modular and extensible AI application. By using layered design, MVC structure, session-based processing, and model abstraction, the project shows how AI functionality can be integrated into a maintainable software engineering framework rather than implemented as isolated experimental scripts.

As a result, the technological impact of the project lies not only in the editing features themselves, but also in the demonstration of how AI-driven systems can be designed responsibly and systematically.

11.2 Economic Impact

Invisio has potential economic value because it aims to reduce the time, expertise, and software complexity typically required for image editing tasks. Many users who need image editing support—such as social media users, small businesses, digital content creators, and e-commerce sellers—may not have access to expensive professional software or advanced editing skills.

By automating tasks such as object removal, enhancement, background editing, and style-based transformation, the system can reduce the effort required to produce visually improved content. This may increase productivity for users who need fast editing results for commercial or personal purposes.

At the same time, AI-based editing platforms may also affect professional creative labor. As automation becomes more capable, some routine editing tasks that previously required manual work may become easier to perform with limited human effort. While this can improve efficiency, it may also contribute to changes in how certain low-complexity editing services are valued.

In the context of this academic project, the economic impact is primarily conceptual rather than market-proven. However, the system clearly illustrates how AI-assisted editing tools could lower access barriers and reduce operational cost in content production workflows.

11.3 Social Impact

The social impact of Invisio is closely related to how image editing technologies affect communication, self-presentation, and digital content creation. Visual media plays a central role in social platforms, online identity, advertising, and interpersonal communication. As a result, tools that make image editing easier can influence not only productivity but also how people represent themselves and interpret others.

On the positive side, Invisio can increase accessibility by allowing non-technical users to perform image editing tasks that would otherwise require significant training or expensive software. This may support creativity, self-expression, and broader participation in digital content creation.

However, image editing systems also create social risks. Features such as beautification, content modification, and object removal may contribute to unrealistic visual expectations, edited self-representation, or misleading presentation of reality. Even when used without malicious intent, such systems may still reinforce problematic norms or distort authenticity in digital spaces.

For this reason, the social impact of the project should be understood as mixed. The system provides empowerment and convenience, but it also exists within a broader environment in which image manipulation can influence perception, self-image, and social trust.

11.4 Ethical Impact

The ethical impact of Invisio is significant because the project involves AI-driven transformation of user-generated visual content. Ethical concerns arise in areas such as privacy, responsible manipulation, content misuse, and representation.

A positive ethical aspect of the project is its privacy-oriented design. By processing im-

ages in temporary sessions and avoiding persistent storage by default, the system reduces unnecessary data exposure and supports responsible data handling. This demonstrates that ethical considerations were integrated into the architecture rather than treated as an external concern.

Another positive aspect is the inclusion of content safety through NSFW detection. This shows an effort to prevent at least some forms of misuse and to define boundaries around acceptable system use.

At the same time, the editing capabilities of the platform may raise ethical concerns. Beautification and image transformation features can influence self-perception or support edited representations that diverge from reality. Object removal and visual enhancement can also be used to alter meaning or create misleading impressions.

Therefore, the ethical impact of Invisio depends not only on what the system can do, but also on how it is used. The project addresses this by embedding privacy, moderation, and controlled interaction principles into the design, but it does not eliminate the broader ethical questions associated with AI-based image manipulation.

11.5 Privacy and Regulatory Impact

Invisio has an important privacy-related impact because it demonstrates an approach to image processing that prioritizes data minimization and controlled retention. In many online systems, user-uploaded images are stored for extended periods, reused for analytics, or incorporated into broader data ecosystems. This creates regulatory and trust-related challenges.

By contrast, Invisio is designed around temporary session-based processing and default non-persistence. This makes the system conceptually more aligned with privacy-oriented engineering principles and supports compliance-minded thinking in relation to data protection frameworks such as GDPR and KVKK.

The project therefore has a regulatory impact in the sense that it serves as a model for how AI applications can be designed with privacy constraints built into the workflow. Even though this is an academic project rather than a production legal compliance case, the design decisions still reflect responsible awareness of legal and governance expectations surrounding personal data.

11.6 Environmental Impact

The environmental impact of Invisio is primarily related to computational resource usage. AI-based image processing requires more energy than conventional lightweight software operations, particularly when deep learning inference is executed on GPU-backed hardware.

Compared with large-scale generative AI systems, the computational footprint of Invisio is relatively limited. However, repeated model inference, high-resolution image

processing, and GPU dependency still imply energy consumption that should not be ignored.

From an engineering perspective, the project partially addresses this by prioritizing bounded performance and avoiding unnecessarily heavy processing models in the default workflow. This reflects an awareness that practical efficiency is important not only for usability but also for resource-conscious system behavior.

The environmental impact of the project is therefore moderate but real. While it is not the dominant concern in this system, energy-aware design remains relevant whenever AI-based computation is involved.

11.7 Educational and Academic Impact

As a graduation project, Invisio also has educational and academic impact. The project demonstrates the integration of software engineering principles, AI model usage, privacy-aware architecture, interface design, and testing methodology within a single coherent system.

This makes the project valuable not only as a technical artifact but also as a learning outcome. It reflects the ability to move from problem analysis to architectural design, detailed design, implementation, and evaluation while maintaining consistency across project documents.

Academically, the project contributes by showing how modern AI capabilities can be embedded into structured engineering processes rather than treated as standalone model demonstrations. This is especially relevant in educational contexts where the challenge is not merely to use AI, but to build complete and responsible software systems around it.

11.8 Positive Impact Summary

The main positive impacts of Invisio can be summarized as follows:

- **making advanced image editing more accessible to non-technical users**
- **reducing time and effort for common visual editing tasks**
- **demonstrating a hybrid AI-plus-user interaction model**
- **promoting privacy-aware design in image processing systems**
- **incorporating content safety mechanisms into an AI workflow**
- **serving as an educational example of integrated AI software engineering**

These benefits show that the system has value not only as a feature set, but also as a design approach.

11.9 Potential Negative Impact and Risk Summary

The potential negative impacts and risks associated with Invisio include:

- **misuse** of editing features to create misleading or altered visual representations
- **reinforcement** of unrealistic beauty standards through beautification functions
- **reduction** of authenticity in shared digital content
- **energy consumption** associated with AI inference
- **incomplete risk control** if advanced moderation or misuse detection is not fully implemented

These risks do not invalidate the project, but they should be acknowledged as part of responsible system evaluation. Recognizing both positive and negative impact improves the credibility and maturity of the final report.

11.10 Overall Impact Assessment

Overall, the impact of Invisio is best understood as balanced and multidimensional. The system has clear positive potential in accessibility, productivity, engineering design quality, and privacy-aware AI integration. At the same time, like many image editing systems, it operates in a domain where misuse, unrealistic representation, and ethical ambiguity are real concerns.

The project addresses these concerns through architectural choices such as temporary data handling, safety filtering, constrained refinement, and controlled system scope. These choices do not remove all risk, but they demonstrate an effort to build a more responsible AI-assisted editing platform.

For this reason, Invisio can be evaluated as a technically useful and ethically aware system whose impact depends not only on its functionality, but also on the design principles that shape its use.

12. Ethical Considerations

12.1 Ethics in AI-Assisted Image Editing

Ethical considerations are an essential part of the Invisio project because the system operates at the intersection of artificial intelligence, visual manipulation, and user-generated content. Unlike conventional software systems that process mostly structured or transactional data, AI-assisted image editing platforms directly affect how visual reality is represented, modified, and interpreted.

This creates ethical responsibilities in several areas, including responsible image transformation, privacy protection, prevention of misuse, and the social consequences of edited visual content. As a result, the development of Invisio required not only technical design decisions but also ethical judgment regarding what kind of system should be built and how its capabilities should be constrained.

The ethical position of the project is that image editing technologies should be designed to support user creativity and accessibility without ignoring privacy, misrepresentation risks, or harmful patterns of usage. This view is reflected throughout the architecture and feature design of the system.

12.2 Responsible Use of AI Capabilities

The use of artificial intelligence in image editing provides clear benefits in terms of automation, convenience, and accessibility. However, these same capabilities can also introduce ethical concerns when users are able to transform images in ways that alter meaning, exaggerate appearance, or create misleading outputs.

Invisio addresses this concern by framing AI as an assistive rather than unrestricted manipulative tool. The system does not expose advanced low-level controls that would encourage unrestricted visual fabrication. Instead, it provides structured editing operations and limited refinement mechanisms that preserve usability while constraining direct manipulation complexity.

This does not fully eliminate the possibility of misuse, but it reflects an ethical design intention: the system is meant to improve and simplify editing workflows, not to promote deceptive or uncontrolled image alteration as its primary purpose.

The design therefore treats AI capability as something that should be guided by boundaries, not simply maximized for technical power alone.

12.3 Privacy and User Consent

One of the most important ethical issues in image editing systems is the handling of user-uploaded content. Images may include personal identity, biometric information, private spaces, or sensitive visual material. For this reason, the ethical management of data is central to the project.

Invisio adopts a session-based and non-persistent processing approach in order to minimize unnecessary retention of user data. This reflects the ethical principle that users should not lose control over their personal visual content simply because they used an online editing service.

User consent is also treated as an important boundary for any form of persistence. The default workflow does not assume permission for long-term storage, public sharing, or secondary use. If optional sharing features are present, they are conceptually based on explicit user action rather than automatic retention.

This design shows that consent is not treated as an implied consequence of system usage, but as a separate and meaningful decision point where applicable.

12.4 Risk of Misleading or Harmful Representation

Image editing technologies always carry the risk of changing how reality is presented. This may occur in harmless contexts such as aesthetic adjustment, but it can also extend into ethically problematic areas such as misleading representation, distorted self-image, or intentional manipulation of visual meaning.

Features such as object removal, beautification, and enhancement can be especially sensitive in this regard. They may be used to alter the perceived truthfulness of an image or to create unrealistic representations of people and environments.

Invisio does not eliminate this ethical problem, because it is inherent to the broader domain of image editing. However, the project acknowledges this risk rather than ignoring it. The system's limited interaction model, safety-oriented content restrictions, and privacy-focused design indicate that ethical concerns were considered during system design.

This acknowledgement is important, because ethical maturity in software projects is not defined only by risk elimination, but also by honest recognition of the risks that remain.

12.5 Beautification and Representation Ethics

Beautification-related features deserve special ethical attention because they interact directly with human appearance and self-representation. AI-based beautification may improve image quality or produce aesthetically preferred results, but it may also reinforce unrealistic beauty norms or contribute to pressure around edited appearance.

This issue is particularly important in digital environments where images are shared publicly and where edited portraits may influence self-esteem, comparison, or expectations of appearance. Even when beautification is used voluntarily, the broader social consequences of such tools should not be ignored.

Invisio treats beautification as one feature among many rather than as the defining purpose of the platform. This reduces, but does not remove, the ethical sensitivity of the

feature. The project also avoids framing beautification in extreme or manipulative terms and places it within a broader set of editing utilities.

12.6 Safety Boundaries and NSFW Restrictions

An important ethical decision in Invisio is the inclusion of content safety boundaries. Since the system processes and transforms visual data, it is necessary to define at least some limits on what kinds of images should be allowed into the workflow.

The project addresses this through NSFW detection and rejection of inappropriate or restricted content before further processing. This feature represents more than a technical safeguard; it reflects the principle that responsible AI systems should include limits on harmful or clearly problematic use cases.

Although such filtering cannot solve every ethical challenge in image manipulation, it provides a baseline form of protection and indicates that the project does not treat capability as ethically neutral. Instead, it accepts that some system boundaries are necessary.

12.7 Fairness and Model Limitations

Ethical concerns in AI systems also include fairness and model behavior across diverse inputs. Image processing models may perform differently depending on factors such as lighting conditions, image quality, pose variation, facial structure, skin tone, or background complexity.

This means that an AI-assisted editing system may not provide equally accurate or equally satisfying results for all users and all image types. In the context of Invisio, such variation is both a technical and ethical issue, because uneven model performance can create unequal user experience or unintended bias in output quality.

The project documents acknowledge that AI performance may vary across image conditions and dataset limitations. This is an important ethical recognition, even if the project does not fully solve the problem of fairness and bias at a research level.

12.8 Transparency and User Awareness

Transparency is an important ethical principle in systems that use AI to transform user content. Users should be aware that the outputs they receive are generated or altered through automated processing rather than representing untouched originals.

Invisio supports this principle at the conceptual level by clearly positioning itself as an AI-assisted image editing platform. The workflow itself makes it clear that the image is being processed, transformed, and optionally refined by the system.

However, transparency is not only about naming the system correctly; it also concerns how clearly the interface communicates what kind of operation is taking place and what the result represents.

12.9 Ethical Value of Privacy-by-Design Architecture

One of the strongest ethical aspects of Invisio is its privacy-by-design architecture. In many software systems, privacy is treated as a policy issue rather than a structural property. By contrast, Invisio attempts to encode privacy directly into the workflow by using temporary session-based processing and avoiding default persistent storage.

This has ethical importance beyond legal compliance. It reflects respect for user autonomy, reduces unnecessary exposure of personal content, and acknowledges that visual data deserves careful treatment even in a technically experimental or academic setting.

The architectural choice to minimize data retention therefore functions both as a security measure and as an ethical statement about how user data should be handled.

12.10 Ethical Limitations of the Project

Although Invisio incorporates several ethically positive design choices, the project also has limitations from an ethical perspective.

The system cannot fully prevent users from applying editing tools in misleading ways. It also cannot guarantee fairness across all input conditions unless models are extensively evaluated on diverse data. Similarly, content filtering provides only partial protection and may not capture all harmful scenarios.

These limitations are important to state explicitly. Ethical design does not require claiming that all problems are solved; instead, it requires demonstrating awareness of what has been addressed, what remains unresolved, and what future work is needed.

12.11 Overall Ethical Assessment

Overall, the ethical design of Invisio can be described as cautious and responsible within the scope of an academic AI-assisted image editing project. The system incorporates privacy-aware data handling, content safety boundaries, constrained user interaction, and explicit awareness of social and representational risks.

At the same time, the project recognizes that image manipulation technologies inherently raise ethical challenges that cannot be fully removed through architecture alone. For this reason, the project’s ethical value lies not in claiming perfect safety, but in demonstrating that software engineering decisions were informed by ethical reflection rather than by functionality alone.

This makes Invisio not only a technical system, but also an example of how responsible design thinking can be incorporated into AI-based software development.

13. Contemporary Issues

13.1 AI Image Editing in the Contemporary Technology Landscape

Artificial intelligence has become one of the defining technologies of the contemporary digital era, and image editing is one of the domains most visibly transformed by this shift. In recent years, AI-assisted tools have made it possible to perform visual operations that previously required professional skill, specialized software, or substantial time. Tasks such as background removal, image enhancement, style transfer, face retouching, and content-aware editing are now increasingly available through accessible web and mobile platforms.

This broader technological context directly shapes the relevance of the Invisio project. The system is part of a contemporary movement toward AI-assisted creativity, where intelligent systems are no longer limited to analysis and prediction but also actively participate in content creation and transformation.

At the same time, the rise of such systems has introduced significant debates about trust, realism, privacy, authorship, and ethical boundaries. As a result, an image editing platform developed today cannot be evaluated purely as a technical utility; it must also be understood as part of an evolving social and technological environment in which AI-generated and AI-modified content is increasingly normalized.

13.2 Generative AI and the Shift from Assistance to Transformation

A major contemporary issue relevant to Invisio is the growing role of generative and transformative AI systems. Earlier software tools mainly supported users by providing editing functions under manual control. In contrast, modern AI systems increasingly make visual decisions on behalf of users by generating, reconstructing, or reinterpreting image content.

This represents a shift from software as a passive tool to software as an active co-creator. In image editing, this means that users may no longer simply adjust images manually; instead, they increasingly rely on AI models to determine object boundaries, fill missing regions, select enhancements, or alter stylistic properties.

Invisio reflects this shift, but it does so in a controlled manner. Rather than embracing unrestricted generation, the project is positioned around assisted transformation with limited user correction. This makes it relevant to a contemporary question in AI system design: how much control should remain with the user when AI systems become more capable of making creative and representational decisions?

13.3 Trust, Authenticity, and Edited Visual Content

One of the most important contemporary issues in AI-based image editing is the erosion of trust in visual content. As editing and generation tools become more powerful and

easier to use, it becomes increasingly difficult to distinguish between unmodified images and heavily altered outputs.

This affects not only artistic contexts, but also social communication, public discourse, and digital credibility. Edited images can shape how people perceive events, products, appearances, and identities. Even when edits are not malicious, the widespread normalization of visual modification may reduce confidence in the authenticity of digital images more broadly.

Invisio operates within this context. Features such as object removal, enhancement, and beautification are practical and useful, but they also participate in a larger environment where image manipulation can alter interpretation. For this reason, the project should be understood as part of a contemporary technological ecosystem in which visual realism and visual truth are no longer automatically aligned.

13.4 Deepfakes, Manipulation, and Misuse Risk

Another major contemporary issue related to AI-based visual systems is the broader problem of manipulative media technologies, including deepfakes and deceptive visual alteration. While Invisio is not designed as a deepfake generation system, it still belongs to the wider category of technologies capable of modifying visual representation.

This creates an important contextual concern. Any system that makes image alteration easier may also, indirectly, contribute to a media environment in which manipulated visual content becomes more common and harder to evaluate critically.

The relevance of this issue to Invisio lies not in claiming that the project creates deepfakes, but in recognizing that all image editing systems now exist within a public discourse shaped by concerns over misinformation, manipulation, and synthetic media. As a result, safety mechanisms such as NSFW filtering, constrained workflow design, and limited exposure of unrestricted controls become more important as part of responsible system design.

13.5 Privacy in an Era of Data-Driven AI Systems

Privacy is a particularly important contemporary issue because many modern AI systems depend on large-scale data collection, retention, and reuse. In such an environment, users are increasingly aware that uploaded content may be stored, analyzed, or repurposed beyond the immediate task they intended.

This concern is especially sensitive for image-based systems, where uploaded files may contain personal identity, biometric information, private spaces, or social context. For this reason, privacy expectations around AI image platforms are no longer optional user preferences; they are part of a broader contemporary demand for trustworthy digital systems.

Invisio responds to this issue through a session-based, non-persistent design philoso-

phy. This makes the project relevant to ongoing discussions about whether AI services should minimize retention by default rather than rely on broad implied consent. In this sense, the system can be seen as aligned with a contemporary shift toward privacy-aware system design.

13.6 Human-in-the-Loop as a Contemporary Design Response

A notable contemporary trend in AI system design is the recognition that full automation is not always the most desirable solution. In many domains, fully autonomous outputs may be efficient, but they can also reduce transparency, remove user agency, and make error correction more difficult.

This has led to renewed interest in human-in-the-loop design, where AI systems perform the main computational work but users retain a structured role in oversight, correction, or decision-making.

Invisio is highly relevant to this trend because its central interaction model is built precisely around this principle. The system does not rely on purely manual editing, but it also does not force users to passively accept AI-generated outputs. Instead, it creates a controlled collaborative relationship between human and system.

This makes the project part of a broader contemporary movement toward AI systems that are not only powerful, but also accountable, assistive, and interaction-aware.

13.7 Regulation, Governance, and Responsible AI

Another major contemporary issue is the increasing importance of AI governance, regulation, and responsible system design. As AI systems become more integrated into everyday tools, there is growing expectation that developers should consider not only technical performance but also risk, fairness, privacy, transparency, and misuse potential.

Invisio reflects this expectation through architectural choices such as privacy-by-design, safety filtering, structured error handling, and limited persistence. These choices are relevant because contemporary software development is increasingly evaluated not only by what systems can do, but also by whether they are built with awareness of their societal and ethical implications.

Although Invisio is an academic project rather than a production compliance platform, it still engages with this contemporary issue by adopting a design approach that takes responsibility seriously instead of focusing only on technical capability.

13.8 Contemporary Relevance of Accessible Creative Tools

The democratization of creative technology is another important contemporary issue connected to this project. AI-assisted tools are lowering the barrier to entry for tasks that once required specialized expertise, expensive software, or extensive training. This

can be empowering because it broadens participation in digital creativity and content production.

Invisio reflects this trend by aiming to make advanced image editing accessible to non-technical users through a web-based and task-oriented interface. This accessibility is one of the reasons the project is contemporary and relevant: it addresses the growing expectation that powerful digital tools should be usable by a broader population, not only by trained professionals.

At the same time, this democratization also increases the importance of safety, ethical framing, and responsible interaction design, because tools that are easier to use are also easier to misuse. This tension is central to contemporary software design in AI-enabled consumer systems.

13.9 Limitations in Addressing Contemporary Issues

Although Invisio is clearly connected to contemporary issues in AI, privacy, and digital media, the project does not solve all of these issues completely. It addresses some of them through architectural and interaction-level choices, but broader societal concerns such as misinformation, beauty standard reinforcement, fairness across all populations, and public trust in synthetic media remain larger problems that extend beyond a single academic system.

This limitation should be acknowledged explicitly. Contemporary relevance does not require complete resolution of all ongoing debates. Rather, it requires that the project recognizes the issues, positions itself responsibly, and avoids ignoring the context in which it operates.

13.10 Overall Contemporary Assessment

Overall, Invisio is strongly connected to contemporary developments in artificial intelligence, digital creativity, privacy-aware system design, and ethical software engineering. The project reflects current shifts toward AI-assisted content transformation, accessible creative tools, and human-centered intelligent interfaces.

At the same time, it is situated within a world where trust in images, concerns about manipulation, and expectations for responsible AI design are increasingly important. For this reason, the contemporary value of Invisio lies not only in its technical features, but also in the fact that it engages with the most relevant questions surrounding modern AI-enabled media systems.

14. Tools & Technologies

14.1 Overview of the Technology Stack

The development of Invisio required the integration of multiple technologies across frontend development, backend services, AI processing, data handling, testing, and deployment support. Since the system is a web-based AI-assisted image editing platform, the selected tools needed to support both interactive web application behavior and computationally intensive image processing tasks.

The technology stack was chosen with several engineering priorities in mind: modularity, maintainability, compatibility with AI workflows, ease of development, and suitability for a modern web-based architecture.

The overall stack can be grouped into the following categories:

- frontend development tools
- backend and API technologies
- AI and image processing libraries
- data and infrastructure tools
- testing and development support tools

Each of these categories contributes to a different layer of the system and supports the realization of the project's architectural goals.

14.2 Frontend Technologies

The frontend of Invisio is implemented using a modern web development stack designed for component-based interface construction, maintainability, and responsive user interaction.

The main frontend technologies include:

- **React** Used as the primary user interface library. React enables the development of reusable components and supports the structured implementation of the upload, editing, preview, refinement, and download views.
- **TypeScript** Used to add static typing to the frontend codebase. This improves reliability, readability, and maintainability by reducing runtime ambiguity and supporting safer component interaction.
- **Vite** Used as the frontend build and development tool. Vite provides fast local development and efficient bundling for modern frontend applications.

- **Tailwind CSS** Used for interface styling. Tailwind supports rapid and consistent development of the visual layout and helps maintain a clean and utility-driven styling structure.
- **PostCSS and Autoprefixer** Used as supporting frontend build and style processing tools in order to improve CSS compatibility and streamline the styling pipeline.
- **Zustand** Used for lightweight frontend state management where necessary. This supports controlled handling of UI-related state across the interaction flow.
- **React Router** Used for navigation and route-based interface organization within the frontend.
- **Axios** Used as the HTTP client for communication with backend API endpoints.
- **Framer Motion** Used to support visual transitions and interaction animations where appropriate in the user experience.
- **Lucide React** Used to provide a consistent icon set for user-facing interface actions and navigation-related visual elements.
- **HTML5 Canvas-based interaction utilities** Used to support brush-based refinement overlays and interactive visual editing behavior such as mask correction and face-mesh-guided interaction.

Together, these technologies provide a frontend stack suitable for building a structured, maintainable, and user-friendly web interface for the Invisio system.

14.3 Backend Technologies

The backend of Invisio is implemented using Python-based web technologies that are well suited for API development and integration with AI processing libraries.

The main backend technologies include:

- **Python** Used as the core backend programming language. Python is widely used in both web development and machine learning, making it an effective choice for a system that combines backend services with AI processing.
- **FastAPI** Used as the backend web framework. FastAPI supports structured API development, request validation, and asynchronous communication, all of which are useful for a modern AI-assisted web application.
- **Uvicorn** Used as the ASGI server for running the FastAPI application.
- **Pydantic** Used for structured data validation and schema management in backend request and response handling.

- **python-multipart** Used to support multipart/form-data handling for image upload workflows.
- **SQLAlchemy** Included in the broader stack as the ORM tool for database-related operations where needed.
- **asyncpg** Used as an asynchronous PostgreSQL driver in database-backed backend workflows.
- **StreamingResponse-based delivery support** Used where necessary for binary output delivery such as downloadable processed images and export-oriented response flows.

These backend tools support the implementation of secure and maintainable API endpoints while remaining compatible with the modular controller-oriented structure of the system.

14.4 AI and Image Processing Technologies

Because Invisio is built around AI-assisted image editing, the AI and image processing stack is one of the most important parts of the project. The implemented processing architecture combines deep learning-based models with deterministic computer-vision algorithms in order to balance output quality, runtime efficiency, and hardware-aware practicality.

The key technologies in this category include:

- **U2-Net** Used as the primary deep learning model for segmentation and background removal. It enables accurate separation of foreground objects from the background and supports mask generation in editing workflows.
- **MobileSAM** Used as a lightweight segmentation component in hair masking, facial masking, and aging-related workflows. It is particularly useful in low-latency and human-in-the-loop scenarios where interactive mask correction is required.
- **LaMa** Used for object removal and image completion through robust inpainting, especially in workflows where masked regions must be reconstructed in a visually coherent way.
- **MediaPipe Face Landmarks** Used for face-aware analysis by tracking dense facial landmark points. This supports localized portrait editing operations and enables region-specific processing in facial enhancement and reshaping workflows.
- **Deterministic aging pipeline** Used as a performance-oriented alternative to heavier generative approaches. In this workflow, masks obtained through segmentation support procedural wrinkle generation, skin-texture modification, and tone manipulation for aging and rejuvenation effects.

- **Bilateral Filtering and CLAHE** Used in beautification and portrait- enhancement workflows in order to smooth skin regions while preserving important facial structures and enhancing local contrast.
- **EDSR-based super-resolution through OpenCV DNN** Used for image sharpening and resolution enhancement in workflows that require super-resolution support.
- **AdaIN** Used for neural style transfer in order to transfer stylistic characteristics from one image to another efficiently in both integrated and stand- alone style-transfer workflows.
- **Fast Non-Local Means** Used for denoising in image-improvement workflows where grain or visual noise must be reduced before or after enhancement operations.
- **ECCV16 Colorization** Used as a deep learning-based colorization architecture for transforming grayscale images into colorized outputs.
- **NSFW CNN Classifier** Used as a safety-oriented classification component that analyzes uploaded images and prevents unsafe or restricted content from entering the main editing pipeline.
- **Unsharp Masking** Used as a supporting sharpening algorithm in final enhancement stages in order to improve perceived edge clarity and output crispness.
- **NumPy** Used for numerical array operations and intermediate data manipulation during preprocessing and postprocessing stages.
- **Pillow** Used for image loading, conversion, resizing, and general image preprocessing/postprocessing utilities.
- **OpenCV** Used extensively for classical image processing, geometric transformations, blending, enhancement-related utilities, seamless cloning, mask handling, and deterministic fallback workflows.

Together, these technologies form a hybrid AI and image-processing stack. Rather than relying exclusively on heavy generative models, the implemented system combines task- specific deep learning components such as U2-Net, MobileSAM, LaMa, AdaIN, EDSR, and ECCV16 Colorization with deterministic algorithms such as bilateral filtering, CLAHE, Fast Non-Local Means, and Unsharp Masking. This hybrid structure improves practical performance, reduces computational cost in selected workflows, and helps the system remain responsive in an interactive web-based environment.

14.5 Data, Storage, and Infrastructure Technologies

Although Invisio is designed around temporary and non-persistent data handling in its default workflow, the broader project stack includes tools related to data management and infrastructure support.

The technologies mentioned in the project environment include:

- **PostgreSQL** Used as the database system in the broader architecture where structured data persistence is needed for authentication, community-related data, and other optional persistent application features.
- **SQLAlchemy with asynchronous database access** Used to support ORM-based interaction with persistent entities while remaining compatible with modern async backend workflows.
- **asyncpg** Used as an asynchronous PostgreSQL driver in Python-based backend workflows.
- **Docker** Used for containerization support, helping standardize development and execution environments.
- **Docker Compose** Used for multi-service environment configuration, especially in development-stage database setup and related infrastructure workflows.
- **Local filesystem storage** Used for explicitly persisted community images, while the default editing workflow continues to rely on temporary session-oriented handling.

These tools support reproducibility, environment consistency, and extensibility. Even in a privacy-oriented system that minimizes persistent image storage, infrastructure technologies remain important for backend organization, deployment preparation, and optional feature support.

14.6 Testing and Development Tools

To support development quality and verification, the project also makes use of testing and development support tools.

The main tools in this category include:

- **pytest** Used for automated backend testing and verification of system behavior.
- **httpx** Used for testing API interactions in Python-based backend test workflows.
- **Playwright** Used for browser-oriented validation of selected frontend workflows and interface-level interaction behavior.

- **ESLint** Used to improve frontend code quality and consistency during development.
- **Git / GitHub** Used for version control, collaboration, and source code management.

These tools are important because they help ensure that the system is not only implemented, but also testable, reproducible, and collaboratively maintainable. In particular, version control plays a key role in tracking changes across a project that includes both software engineering and AI-related components.

14.7 Why These Technologies Were Chosen

The technologies used in Invisio were selected not as isolated tools, but as parts of a coherent technical strategy shaped by the project’s architectural, usability, performance, privacy, and maintainability requirements. Since the system combines web-based user interaction, backend coordination, AI-assisted image processing, and optional persistent community-oriented features, the chosen stack needed to support both software engineering structure and practical execution efficiency.

On the frontend side, React, TypeScript, and Vite were chosen because they support the development of a modular, maintainable, and responsive interface suitable for non-technical users. React enables reusable component-based UI construction, TypeScript improves reliability through stronger type safety, and Vite provides an efficient development workflow. Tailwind CSS, together with supporting tools such as PostCSS and Autoprefixer, helps maintain a consistent and scalable styling structure, while Zustand, React Router, Axios, Framer Motion, and Lucide React support state handling, navigation, API communication, interaction design, and visual consistency across the platform.

On the backend side, Python and FastAPI were chosen because they provide strong compatibility with AI-oriented libraries while also supporting clean API design and async-compatible request handling. Uvicorn, Pydantic, and python-multipart complement this stack by supporting practical execution, structured validation, and upload-oriented workflows. The backend technologies were therefore selected to preserve a clear separation between request orchestration, validation, and processing logic.

The AI and image processing stack was selected according to a hybrid engineering strategy. Rather than relying exclusively on heavy generative models for every feature, the system combines deep learning components such as U2-Net, MobileSAM, LaMa, AdaIN, EDSR-based enhancement, and ECCV16 colorization with deterministic computer-vision operations such as bilateral filtering, CLAHE, denoising, sharpening, and other classical image-processing utilities. This approach was chosen because it provides a more practical balance between visual quality, runtime efficiency, VRAM usage, and responsiveness in an interactive web environment.

MediaPipe and OpenCV were particularly important choices because the project requires region-aware face processing, classical vision support, geometric manipulation,

blending, mask handling, and fallback-friendly deterministic operations in addition to neural inference. These technologies strengthen the system by ensuring that AI workflows remain usable even when fully learned solutions are not the most efficient option.

For persistence and infrastructure support, PostgreSQL, SQLAlchemy, asyncpg, Docker, and Docker Compose were chosen because they support structured backend organization, reproducible development environments, and optional persistent features such as authentication and community sharing. At the same time, the overall architecture remains compatible with the project’s privacy-by-design approach because the default editing workflow continues to rely on temporary session-based handling rather than unnecessary long-term storage.

The selected development and testing tools were also chosen deliberately. `pytest` and `httpx` support backend-oriented verification, Playwright supports browser-level validation of selected interface workflows, and ESLint improves frontend code quality and consistency. Together with Git and GitHub, these tools help the project remain testable, maintainable, and collaboratively manageable.

An additional advantage of this technology combination is ecosystem compatibility. Since Invisio is both a web platform and an AI-assisted application, it is important that the selected tools work well together rather than functioning as disconnected technologies. The chosen stack therefore reflects not only availability or popularity, but also the need for architectural consistency, practical performance, and future extensibility.

As a result, the technologies used in Invisio were chosen because they support the core identity of the project: a modular, privacy-aware, human-in-the-loop, AI-assisted image editing platform that must remain both technically robust and practically usable.

14.8 Limitations and Clarifications

Although the technology stack is broad and modern, the final report should clearly distinguish between technologies that were:

- part of the intended system architecture
- configured in the development environment
- actively used in the final working implementation

This distinction is especially important for tools such as databases, authentication-related libraries, containerization components, or optional infrastructure technologies that may appear in the environment description but not in the final demonstrated system.

Making this distinction improves the precision and credibility of the final report.

15. General Discussion of the Project Outcome

The outcome of the Invisio project should be evaluated not only in terms of the final features delivered by the system, but also in terms of how those features were organized, implemented, integrated, and distributed across the team during the development process. As an AI-assisted web-based image editing platform, Invisio is not a single-function application. It is a structured software system that combines frontend interaction, backend coordination, AI-based image processing, privacy-oriented session handling, error management, and optional community-oriented functionality within a unified architecture.

From this perspective, the project outcome is meaningful because it reflects both a technical result and an engineering process. The team did not simply assemble isolated features, but developed a system whose parts correspond to the layered and modular architecture defined in the earlier design phases. The final implementation therefore represents the practical realization of a broader software engineering effort shaped by separation of concerns, maintainability, privacy-by-design, and controlled user interaction.

At the same time, discussing the project outcome requires more than listing completed functions. Since Invisio was developed collaboratively, the final result also depends on how responsibilities were divided across the system and how different technical areas were coordinated. Frontend views, backend controllers and services, AI-related modules, and supporting features such as security, session handling, and community-related components all contributed to the overall platform in different ways.

For this reason, the project outcome is discussed in this section through both narrative evaluation and structured responsibility summaries. The following subsections first describe the overall implementation plan and component-oriented realization of the system, and then present the main contribution areas through tables that summarize component responsibilities, ownership, and functional roles. This structure makes the development outcome easier to understand and also documents how the final system emerged as a coordinated team effort rather than a set of unrelated technical parts.

15.1 Implementation Plan and Realization

The development of Invisio was guided by the architectural and functional plans defined in the earlier project documents. At a high level, the system was intended to operate as a modular AI-assisted image editing platform with a clear separation between presentation, coordination, and processing responsibilities. This meant that the implementation process had to address not only individual feature development, but also the integration of frontend workflow, backend request handling, AI processing pipelines, temporary data management, and output delivery into a coherent application structure.

From a planning perspective, the project can be understood as progressing through

several interconnected implementation areas. The frontend side focused on providing a clear and accessible user workflow covering upload, editing selection, refinement, preview, download, and community-related screens. The backend side focused on request orchestration, validation, session lifecycle handling, and coordination of AI-based operations. The AI-related side focused on integrating editing capabilities into the system through reusable and modular processing logic. In parallel, testing, documentation, and design alignment were necessary to ensure that the resulting system remained consistent with the original project goals.

This implementation plan was important because the nature of the project required parallel development across multiple technical domains. Unlike a narrowly scoped software application, Invisio combines interface design, backend engineering, AI-powered processing, privacy-aware data handling, and system-level coordination. Therefore, the project outcome had to be achieved through staged and distributed work rather than through a single linear implementation process.

The final realization of the system reflects this planned structure to a meaningful degree. The project produced a platform in which the major workflow stages are represented as separate but connected components, and in which key architectural decisions—such as modularity, temporary session-based processing, and constrained user interaction—are visible in the implemented structure. This does not mean that every possible planned feature reached the same level of maturity, but it does mean that the overall project outcome remained aligned with the intended system organization.

In this sense, the project plan was not merely theoretical documentation. It served as a practical roadmap that shaped how development responsibilities were divided and how different components were connected during implementation. The following subsections therefore present the realized system outcome in a component-oriented way, beginning with the distribution of responsibilities across the main parts of the platform.

15.2 Task Distribution and Component Ownership

The development of Invisio required coordinated work across multiple domains, including frontend development, backend implementation, AI model integration, system design, testing, documentation, and project-level coordination. Because the project was carried out as a team-based graduation project, task distribution played an important role in ensuring that development could proceed in parallel while preserving architectural consistency across the system.

Responsibilities were assigned according to the technical needs of the project and the evolving development process. Rather than dividing work only by broad titles, the team's contribution structure emerged around concrete system components. This approach was especially appropriate for Invisio because the system itself is organized around clearly distinguished layers and modules. As a result, ownership could be associated with specific views, controllers, processing components, and supporting features in a way that matched

the software architecture.

This component-based distribution provided several advantages. First, it improved clarity by making it easier to identify which team member was primarily responsible for a given part of the platform. Second, it supported parallel development by allowing different areas of the system to progress simultaneously. Third, it improved traceability between the architectural design and the implementation effort, since many of the owned components correspond directly to system elements already defined in the frontend, backend, controller, and model-related sections of the report.

At the same time, task distribution should not be interpreted as rigid isolation between team members. Although ownership was assigned component-wise, the implementation of a complete platform still required collaboration during integration, debugging, testing, documentation, and review. In other words, the final project outcome reflects both individual ownership and collective system-building effort.

To document this structure more clearly, the following responsibility tables summarize the main components of the project, the team member primarily responsible for each one, and the role of that component within the overall platform. These tables provide a more concrete representation of how the final system was distributed across the team.

15.2.1 Frontend Components

Table 1: UI Components and Responsibilities

Component	Responsible	Description
UI Layout & Template	Emre Karadoğan	Defines the overall application shell, layout, and navigation structure. Acts as the main container managing routing between views, supports the auth gate logic, and provides shared interface elements such as the main layout and cookie consent banner.
Top Bar & Sidebar Navigation	Emre Karadoğan	Implements the top bar, left sidebar, and section-based navigation structure that enables users to move across the main parts of the platform consistently.
AuthView	Mehmet Umur Özü	Handles user authentication processes such as login, registration, login/session validation, token acquisition, and session-based interaction with the system.
UploadView	Emre Karadoğan	Provides the interface for users to upload images through a drag-and-drop based workflow, performs file type and size validation, and initiates the image submission and session creation process to the backend.
EditingView	Emre Karadoğan	Allows users to select AI-based editing operations and configure parameters. Sends editing requests to the system and manages interaction flow for processing.

Component	Responsible	Description
RefinementView	Mehmet Umur Özü	Enables users to manually refine AI-generated masks using brush-based tools. Supports undo, clear, mask overlay visualization, and active brush state handling as part of the human-in-the-loop refinement workflow.
PreviewView	Mehmet Umur Özü	Displays AI-processed image results before final confirmation. Allows users to review outputs and decide whether to refine or proceed.
DownloadView	Emre Karadoğan	Handles exporting and downloading of processed images. Provides format selection and triggers the final output delivery process.
CommunityView	Kübra Arslan	Displays publicly shared images from users through the main community feed. Supports browsing, infinite scroll, refresh behavior, and empty/loading states for community-generated content.
MySharedPostView	Kübra Arslan	Allows users to view and manage their own shared posts within the community and serves as a profile-oriented view that supports listing user posts and presenting summary-level statistics.
ErrorView	Emre Karadoğan	Displays structured error messages for validation, processing, or system failures. Ensures clear feedback and graceful error handling.
SessionExpiredView	Emre Karadoğan	Notifies users when their session has expired. Guides them to restart or reinitialize the interaction securely.
MainCanvasView	Emre Karadoğan	Displays original and processed images within the main editing workspace and supports zooming and compare mode for interactive visual inspection.
FaceMeshEditingView	Emre Karadoğan	Supports interactive face editing based on face mesh or landmark structures and enables region-aware facial manipulation in portrait-related workflows.
ToolParameterPanel	Emre Karadoğan	Presents tool-specific parameter controls such as sliders and toggles and manages apply and reset actions for selected editing operations.
BottomActionBar	Emre Karadoğan	Provides quick access to actions such as revert, compare, zoom, export, download, and share within the main editing workflow.
StyleTransferView	Kübra Arslan	Offers a standalone style transfer interface by allowing users to upload content and style images, adjust alpha values, and preview the generated result.
BackgroundReplaceView	Mehmet Umur Özü	Provides a standalone background replacement workflow by allowing foreground and background uploads and displaying the generated preview result.
DetectInvisioView	Mehmet Umur Özü	Displays the interface for hidden signature detection and presents outputs such as confidence scores and matched bits to the user.

Component	Responsible	Description
CommunityPostCard	Kübra Arslan	Represents an individual community post and supports actions such as liking, opening comments, adding comments, and deleting the post when the active user is the owner.
ShareModals	Kübra Arslan	Manages modal-based sharing workflows for both session-based outputs and standalone generated results before publishing them to the community layer.
HairColorChangeView	Mehmet Umur Özü	Provides the active hair color change interface by allowing users to pick color-reference for transformation workflows.

The frontend component distribution reflects the user-facing structure of Invisio. Since the system is intended for non-technical users, the frontend plays a central role in translating complex backend and AI-supported functionality into a simple and understandable workflow. The assigned frontend responsibilities therefore represent not only interface implementation, but also the realization of usability, accessibility, and interaction flow within the project.

15.2.2 Backend Components

Table 2: Backend Components and Responsibilities

Component	Responsible	Description
FastAPI Bootstrap	Mehmet Umur Özü	Implements the FastAPI application bootstrap layer, including lifespan management, CORS configuration, global exception handling, model preloading, and periodic session cleanup.
Security Middleware	Mehmet Umur Özü	Provides the backend security middleware responsible for upload MIME validation and security header configuration for incoming and outgoing requests.
IP Rate Limiting Middleware	Mehmet Umur Özü	Implements IP-based rate limiting middleware in order to protect backend resources against abuse, excessive traffic, and repeated automated requests.
Session-Based Core API Endpoints	Kübra Arslan	Implements the main session-based API routes, including <code>/upload</code> , <code>/process</code> , <code>/status/{id}</code> , <code>/download/{id}</code> , and <code>/session/{id}</code> , which manage the primary upload, processing, tracking, and download workflow.
Refinement and Regeneration Endpoints	Fatih Bayazıt	Implements the mask correction and re-generation routes <code>/refine</code> and <code>/regenerate</code> , supporting backend-side refinement and output re-creation workflows.

Component	Responsible	Description
Landmark Endpoint	Kübra Arslan	Implements the landmark retrieval endpoint <code>/landmarks/{session_id}</code> for face-aware and landmark-based editing workflows.
Standalone AI End-points	Mehmet Umur Özü	Implements standalone AI processing routes such as <code>/style-transfer</code> and <code>/background-replace</code> , enabling feature-specific workflows outside the main editing flow.
Auth Endpoints	Kübra Arslan	Implements authentication-related routes including <code>/auth/register</code> , <code>/auth/login</code> , <code>/auth/refresh</code> , <code>/auth/me</code> , and <code>/auth/unregister</code> for identity, access, and account-related operations.
Community Endpoints	Kübra Arslan	Implements community-related backend routes for share, direct share, feed retrieval, single post access, deletion, liking, commenting, user posts, and image serving.
Detect Invisio End-point	Emre Karadoğan	Implements the hidden signature detection route <code>/detect-invisio-image</code> and returns backend-side detection outputs such as verification confidence and related analysis results.
Hair Color Transfer Endpoint	Mehmet Umur Özü	Implements the hair color transfer backend route <code>/hair-service</code> ; additionally, the preset-based hair endpoint is preserved in retired form for compatibility and reference.
In-Memory Session Service	Emre Karadoğan	Provides privacy-oriented in-memory session management, including temporary upload/result storage, expiry handling, and cleanup of session data after processing.
Export Pipeline	Emre Karadoğan	Implements the export pipeline in which watermarking and hidden signature application are performed only at download time.
Local Storage Service	Kübra Arslan	Provides persistent local filesystem storage operations for community images, including writing and deleting publicly shared visual content.
Repository and ORM Layer	Kübra Arslan	Implements the repository and ORM layer used for authentication- and community-related persistence logic.
Async SQLAlchemy + PostgreSQL Connection Layer	Kübra Arslan	Implements asynchronous SQLAlchemy and PostgreSQL integration, including retry logic and request-scoped database session management.

The backend component distribution represents the coordination layer of the system. These components are responsible for handling requests, managing sessions, validating inputs, orchestrating AI workflows, and delivering outputs back to the user-facing layer. Because Invisio depends on controlled interaction between multiple subsystems, backend ownership is a particularly important part of the overall project outcome.

15.2.3 AI and Processing Components

Table 3: AI and Processing Components and Responsibilities

Component	Responsible	Description
AIModelFactory Architecture	Mehmet Umur Özü	Implements the model selection architecture that maps each editing request to the correct processing pipeline through a factory/strategy-based abstraction. This structure allows segmentation, inpainting, enhancement, style transfer, beautification, and other task-specific models to be invoked through a unified backend flow.
U2Net Integration	Fatih Bayazıt	Implements foreground segmentation using U2Net and uses the resulting mask in background removal and region-isolation workflows inside the main editing canvas.
Hybrid Mask Flow on U2Net Output	Fatih Bayazıt	Extends raw U2Net segmentation by allowing user brush input to add or remove mask regions, making the segmentation result editable before final downstream processing.
LaMa Integration	Fatih Bayazıt	Implements object removal and region completion through LaMa-based inpainting, allowing masked areas to be reconstructed during removal and regenerate operations.
Detect-Refine-Regenerate Pipeline	Mehmet Umur Özü	Implements the core hybrid workflow of the system: an initial mask is detected automatically, optionally refined by the user, and then reused to regenerate the final result through the processing pipeline.
Face Reshape Pipeline	Mehmet Umur Özü	Implements facial reshaping by combining MediaPipe Face Landmarker outputs with OpenCV-based geometric warping and seamless cloning so that face-aware edits remain visually coherent.
Enhancement Model	Mehmet Umur Özü & Fatih Bayazıt	Implements image enhancement through a dual-path pipeline that uses OpenCV super-resolution checkpoints when available and otherwise falls back to a deterministic image improvement process.
Beautification Model	Mehmet Umur Özü	Implements portrait-oriented beautification operations such as skin smoothing, tone balancing, and facial feature enhancement while preserving the overall portrait structure.
Aging/Rejuvenation Model	Mehmet Umur Özü	Implements a deterministic portrait transformation pipeline that simulates facial aging or rejuvenation effects without depending on an unstable external generation workflow.

Component	Responsible	Description
AdaIN Style Transfer Model	Mehmet Umur Özü	Implements AdaIN-based neural style transfer and supports both integrated model-layer usage and standalone style transfer processing through a dedicated feature flow.
ECCV16 Colorization Integration	Mehmet Umur Özü	Implements grayscale image colorization through an ECCV16-based deep learning architecture, allowing black-and-white images to be transformed into colorized outputs within the platform.
NSFW Detection Model	Mehmet Umur Özü & Emre Karadoğan	Implements a locally loaded NSFW detection stage that validates uploaded content before further editing is allowed, acting as a safety and misuse-prevention control in the processing pipeline.
Hair Color Change Module	Mehmet Umur Özü	Implements the hair color change workflow by detecting and isolating hair regions, applying controlled color transformation to the selected area, and preserving the surrounding portrait structure for visually consistent results.
Shared Portrait/Image Utilities	Mehmet Umur Özü & Fatih Bayazıt	Implements reusable low-level image and portrait processing helpers such as blending, color transfer, CLAHE, sharpening, and mask generation, which support multiple AI pipelines across enhancement, beautification, hair, and regeneration workflows.

The AI and processing component distribution captures the parts of the system most directly related to image transformation capabilities. These responsibilities include model selection, task-specific processing behavior, and the modular execution structure required for operations such as segmentation, inpainting, enhancement, style transfer, or related AI-supported features. This area is especially significant because it forms the technical core of the platform’s editing functionality.

15.2.4 Validation Through Representative Test Cases

In addition to component ownership and implementation responsibilities, the realized outcome of the Invisio project can also be understood through the test cases used to validate the system. Since the platform combines frontend interaction, backend coordination, session-based processing, and AI-driven transformation workflows, the project outcome is meaningful not only because these components were implemented, but also because they were exercised and verified through representative functional and integration-oriented test scenarios.

For this reason, the following table summarizes selected test cases that reflect the validation of core system capabilities. These cases do not replace the detailed testing and evaluation section of the report; rather, they provide a compact view of how the major

implemented workflows were checked in practice and how the final system moved beyond architectural design into verified behavior.

Table 4: Representative Test Cases and Results

Test ID	Description	Input(s)	Expected Output	Tester	Result
TC-01	Verify that the system allows users to upload a valid image file.	Valid image file (JPG/PNG).	The image is successfully uploaded and displayed.	Emre Karadoğan	Passed
TC-02	Verify that the system rejects unsupported file formats.	Invalid file type (.txt, .pdf).	The system rejects the file and shows an error message.	Emre Karadoğan	Passed
TC-03	Verify AI processing workflow for a selected editing operation.	Uploaded image + selected AI editing feature.	The AI module processes the image and generates an edited result.	Mehmet Umur Özü & Emre Karadoğan	Passed
TC-04	Verify that processed results are displayed correctly in preview.	Processed image from AI module.	The preview screen shows the edited image correctly.	Emre Karadoğan	Passed
TC-05	Verify downloading of processed images.	Processed image ready for export.	Image is downloaded successfully and matches preview.	Emre Karadoğan	Passed
TC-06	Evaluate system performance with large image files.	High-resolution image.	Image processed within acceptable time.	Fatih Bayazıt	Passed
TC-07	Verify session-based temporary image handling.	Uploaded image during active session.	Image reset after session completion by option.	Mehmet Umur Özü	Passed
TC-08	Verify system behavior when processing is requested without image upload.	Processing request without image.	The system displays an error message.	Emre Karadoğan	Passed
TC-09	Verify object removal feature using AI inpainting.	Image containing removable object.	Object removed and background reconstructed.	Fatih Bayazıt	Passed
TC-10	Verify image enhancement feature.	Low-quality image.	The system improves brightness, contrast, or color balance.	Mehmet Umur Özü	Passed
TC-11	Verify neural style transfer feature.	Uploaded image + selected artistic style.	The system generates a stylized version of the image.	Kübra Arslan	Passed
TC-12	Verify facial beautification feature.	Portrait image containing face.	The system enhances facial features naturally.	Mehmet Umur Özü	Passed
TC-13	Verify background editing functionality.	Image with background.	Background removed, blurred, or replaced.	Mehmet Umur Özü & Fatih Bayazıt	Passed
TC-14	Verify black-and-white image colorization.	Grayscale image.	The system generates colored images.	Fatih Bayazıt	Passed
TC-15	Verify manual mask refinement tool.	AI-generated mask + user brush input.	Mask updated according to user corrections.	Mehmet Umur Özü	Passed
TC-16	Verify hybrid AI + manual editing workflow.	Masked image by AI + refinement adjustments.	System reprocesses image using corrected mask.	Mehmet Umur Özü	Passed
TC-17	Verify NSFW or restricted content detection.	The user uploads an image containing inappropriate or restricted content.	The system detects the NSFW content, blocks further processing, and displays an appropriate warning message. The image is not processed or stored.	Emre Karadoğan	Passed
TC-18	Verify community sharing feature.	The user chooses to share the final image.	Image published to community gallery without metadata.	Kübra Arslan	Passed

Test ID	Description	Input(s)	Expected Output	Tester	Result
TC-19	Verify user authentication with the sign in and login screen.	The user enters personal information to sign in or login.	The user creates a unique account and reaches it with personal information.	Mehmet Umur Özü	Passed
TC-20 (Extra test case)	Verify image authentication with the trademark.	Processed and downloaded image	The user downloads trademark authenticated image.	Mehmet Umur Özü	Passed

15.3 Discussion of the Realized Outcome

When considered as a whole, the final outcome of Invisio reflects a system that was developed with clear architectural intent and realized through component-based team coordination. The project outcome is therefore stronger than a simple feature checklist. It demonstrates how a graduation project can move from conceptual design into a more structured implementation where different team members contribute to distinct yet connected parts of the same platform.

This is particularly important in the context of a full-stack AI-assisted system. In such a project, success does not depend only on whether an AI model runs correctly or whether a single screen is implemented. It depends on whether the interface, backend, data lifecycle, processing logic, safety measures, and user workflow operate together in a consistent and maintainable way. The component and responsibility distribution shown in this section helps make that broader engineering outcome visible.

For this reason, the general discussion of the project outcome should be understood as a discussion of both realization and organization. Invisio is meaningful not only because of what it aims to do, but also because of how its structure, ownership model, and modular implementation process contributed to producing a coherent academic software system.

16. What Worked Well

Several aspects of the project worked particularly well from a software engineering perspective.

First, the architectural design is one of the strongest parts of the system. The use of client–server separation, layered structure, MVC organization, controller-based coordination, temporary session handling, and modular AI abstraction creates a design that is clear, extensible, and academically strong.

Second, the project demonstrates thoughtful engineering trade-offs rather than feature-driven design alone. Decisions related to privacy, statelessness, usability, security, and human-in-the-loop refinement were treated as architectural priorities. This improves the overall quality of the system and distinguishes it from simpler prototype-style projects.

Third, the inclusion of design patterns such as Strategy and Facade supports maintainability and strengthens consistency between the design documents and the implementation approach.

Fourth, the project’s privacy-focused philosophy is particularly successful as a conceptual strength. The decision to avoid persistent storage by default and process images within temporary sessions makes the project more responsible and more aligned with contemporary expectations for AI systems.

Finally, the testing plan and requirement traceability structure show that the project was approached systematically rather than informally, even where some parts of testing remain incomplete in documented results.

16.1 Main Technical Challenges

The project also involved several significant technical challenges.

One of the main challenges is the complexity of AI integration in a full software system. It is one thing to run an individual model successfully in isolation, but it is much more difficult to integrate AI processing into a complete workflow that includes upload handling, temporary data management, user interaction, preview generation, and controlled result delivery.

Another major challenge is balancing performance with usability. AI models can produce strong visual outputs, but they may also introduce latency and resource dependency, especially in a web-based system where responsiveness matters. This makes deployment and workflow design more difficult than in conventional software systems.

The refinement workflow introduces an additional challenge because it requires continuity between AI-generated intermediate results and user-driven corrections. Supporting this in a clean and privacy-preserving way increases both implementation complexity and testing requirements.

Security and safety also add complexity. Input validation, NSFW filtering, structured error handling, and session lifecycle control all require careful coordination across different parts of the system.

Overall, the technical challenge of Invisio lies less in any single feature and more in the difficulty of bringing many interdependent concerns together into one consistent platform.

16.2 Gaps Between Design and Implementation

As with many academic projects, some difference between architectural ambition and implementation detail is natural. However, the final state of Invisio suggests that this gap is smaller than what earlier draft evaluation materials implied.

The design documents describe a rich and modular architecture including multiple AI modules, session-based privacy controls, refinement workflows, community sharing capability, and structured controller logic. With the completion of previously pending

tests, the implemented system now aligns more closely with this broader design vision.

This means that the project should no longer be characterized primarily as a system with major unverified advanced features. Instead, it is more accurate to say that the project achieved a substantial degree of consistency between design intent and implemented behavior, even if some documentation details may still require clarification.

16.3 Strength of the Project as an Academic System

From an academic perspective, one of the strongest qualities of Invisio is that it is documented and structured as a complete engineering project rather than as a narrow technical demo. The project includes requirement analysis, architecture design, low-level design, test planning, ethical analysis, impact evaluation, and implementation discussion.

This breadth gives the project value beyond simple functionality. It shows that the team approached the system as a software engineering problem involving design trade-offs, risk analysis, modularity, privacy, and long-term maintainability.

The project is also strong academically because it connects contemporary AI capabilities with responsible system design. Rather than treating AI as an isolated feature, the project embeds it within a broader framework of usability, ethical awareness, and architectural clarity.

As a result, even where the implementation may not fully exhaust the design vision, the project still stands as a substantial and well-reasoned academic contribution.

16.4 Limitations of the Current System

Despite its strengths, the current system has several limitations that should be discussed clearly.

First, the system depends on AI model performance, which may vary depending on image characteristics such as resolution, lighting, pose, or scene complexity. This means that output quality cannot be guaranteed uniformly across all inputs.

Second, the system is limited to image-based workflows and does not support video processing, offline use, or real-time continuous editing. These limitations are partly intentional design constraints and partly practical scope boundaries.

Third, some advanced features described at the design level may not yet be fully implemented, fully integrated, or fully validated in practice. This includes areas such as performance characterization, refinement continuity, session cleanup assurance, and optional community or access-control features.

Fourth, the project does not fully solve broader ethical and fairness issues associated with AI-based image transformation. While it addresses these concerns architecturally, it does not perform deep fairness research or complete misuse prevention.

Acknowledging these limitations improves the credibility of the report and provides a clearer foundation for future work.

16.5 Lessons Learned

The development of Invisio provides several important lessons.

One key lesson is that successful AI system development requires much more than integrating machine learning models. Building a real application around AI requires careful coordination of architecture, interface design, temporary data handling, validation, testing, and user experience.

A second lesson is that privacy and safety should be built into the architecture from the beginning rather than added later. Decisions about session lifecycle, storage, request validation, and content filtering affect the structure of the system itself and are difficult to retrofit cleanly.

A third lesson is that software design patterns and modular abstractions are highly valuable in AI-enabled systems. The use of Strategy and Facade patterns, together with layered design, made it easier to reason about the system and maintain consistency between architecture and implementation.

A fourth lesson is that testing AI-assisted systems requires broader thinking than ordinary feature testing. Correctness, usability, latency, intermediate workflow continuity, and safety behavior all matter simultaneously.

Overall, the project shows that AI-based software engineering is fundamentally multidisciplinary: it combines software architecture, machine learning integration, product design, security, and ethics into a single development problem.

16.6 Discussion of Project Maturity

In its current form, Invisio can be described as a strong academic system with meaningful implementation progress, a mature design philosophy, and broader functional validation than earlier draft evaluation records suggested. It is clearly more than a conceptual proposal, because it includes implemented workflows, tested features, and integrated system components across both basic and advanced functionality.

The completion of tests related to AI processing, refinement, hybrid workflow, performance, and session behavior indicates that the system has reached a stronger maturity level within the context of a graduation project. While it is still not appropriate to describe it as a fully industrialized product with production-scale infrastructure and long-term field validation, it has clearly progressed beyond a partially validated prototype.

This should be viewed positively. The project demonstrates that the team was capable not only of designing at system level, but also of realizing and validating a substantial portion of that design in practice.

16.7 Overall Discussion

Overall, Invisio should be understood as a carefully designed and substantially realized AI-assisted image editing platform that succeeds strongly at the level of architecture,

design logic, responsible system framing, and functional validation. Its greatest strengths lie in its modular structure, hybrid human-in-the-loop philosophy, privacy-aware data lifecycle, and broad engineering completeness as an academic project.

With the completion of previously pending tests, the project now presents a stronger implementation profile than before. The system is no longer supported only by architectural reasoning and limited workflow validation, but by a wider set of completed tests covering advanced AI functionality, refinement behavior, performance-related evaluation, and session-based processing.

Its remaining limitations are therefore less about unverified core capability and more about the natural boundaries of an academic project when compared with production systems, such as large-scale deployment maturity, exhaustive stress testing, and deeper quantitative reporting.

For these reasons, the discussion supports a stronger final conclusion: Invisio is a technically meaningful, ethically aware, and academically strong project whose implementation and validation level significantly exceed that of a simple prototype.

17. Conclusion

17.1 Project Conclusion

This report presented the full analysis, design, implementation, and evaluation process of Invisio, an AI-assisted, web-based image editing platform developed as a graduation project. The system was designed to address an important gap in existing image editing technologies: the lack of balance between full automation and meaningful user control.

From the beginning of the project, the main objective was to create a platform that allows non-technical users to perform advanced image editing tasks through a simple and accessible interface, while still preserving the possibility of limited corrective interaction. In response to this objective, Invisio adopted a hybrid human-in-the-loop design in which artificial intelligence performs the main editing operations and users can refine certain outputs through structured interaction mechanisms.

The final system demonstrates that this objective was successfully translated into a coherent software architecture. Through the use of client-server separation, layered design, MVC organization, temporary session-based data handling, modular AI abstractions, and structured controller logic, the project established a technically sound and maintainable system foundation.

The implementation also reflects the project's broader engineering goals. Privacy-by-design, security-by-default, AI modularity, and interface simplicity were treated as core requirements rather than optional enhancements. This gives the system value not only as a functional application, but also as an example of responsible AI-oriented software engineering.

17.2 Achievement of Project Objectives

The main project objectives were achieved to a substantial degree.

The system provides a web-based interaction model for image upload, AI-assisted processing, preview, optional refinement, and result download. It incorporates multiple AI-based editing operations within a unified workflow and supports the hybrid interaction model that distinguishes the project from fully manual or fully automated alternatives.

The architectural objectives were also achieved successfully. The project established clear separation of concerns, used modular abstractions for AI processing, and integrated design patterns such as Strategy and Facade to improve extensibility and maintainability. In addition, privacy-preserving temporary session handling was incorporated as a core structural feature of the system.

The evaluation process further strengthens this conclusion. With the completion of the previously pending tests, the project now demonstrates verified functionality across both foundational and advanced areas, including AI processing workflows, refinement behavior, performance-related evaluation, session-based data handling, and safety-oriented controls.

Taken together, these results indicate that the project met not only its conceptual goals but also a significant portion of its practical implementation and validation goals.

17.3 Final Assessment of the System

Invisio can be assessed as a technically meaningful and academically strong AI-assisted software system. Its strength lies not merely in the number of features it includes, but in the consistency of its engineering logic. The project combines requirement analysis, architectural reasoning, detailed system design, implementation, testing, ethical reflection, and impact awareness into a unified whole.

The system's most distinctive contribution is the combination of AI automation with constrained user correction under a privacy-aware architecture. This design positions the project meaningfully within the contemporary landscape of AI-enabled image editing tools and demonstrates that accessibility, control, and responsibility can be pursued together.

Although the project remains within the realistic limits of an academic graduation context and should not be confused with a fully industrialized production platform, its overall maturity clearly exceeds that of a simple prototype or feature demonstration. The system has both design depth and implementation credibility.

17.4 Closing Statement

In conclusion, Invisio demonstrates that it is possible to build an AI-assisted image editing platform that is not only functionally useful, but also architecturally disciplined, privacy-aware, and ethically conscious. The project shows that successful AI software

development requires more than model integration; it requires careful consideration of interaction design, system modularity, security, testing, and responsible data handling.

As a graduation project, Invisio stands as a substantial example of how contemporary artificial intelligence technologies can be incorporated into a complete software engineering process. It therefore represents both a technical achievement and a meaningful learning outcome for the development team.

18. Future Work

18.1 General Future Development Direction

Although Invisio has reached a meaningful level of architectural maturity and functional validation as an academic project, there are several clear directions in which the system can be extended and improved. These future development areas involve not only adding new features, but also strengthening system robustness, scalability, usability, and production readiness.

The current version of the project provides a strong foundation for future work because its architecture is modular, extensible, and organized around clearly defined responsibilities. This means that future development can build on the existing system structure without requiring complete redesign.

The following subsections outline the most important directions for future improvement.

18.2 Expansion of AI Capabilities

One of the most direct future development paths for Invisio is the extension of its AI feature set. Although the system already includes multiple image editing operations, additional model capabilities could further increase its usefulness and competitiveness.

Potential future AI-related improvements include:

- **higher-quality** object removal with more advanced inpainting approaches
- **improved** portrait enhancement and face-aware editing
- **additional** style transfer options and more controllable artistic transformations
- **more** robust background editing under difficult visual conditions
- **super-resolution** support for low-quality images
- **more** adaptive enhancement models for different image domains

Another important future direction is improving model performance and robustness. This may involve retraining or fine-tuning models on better-curated datasets, improving output

consistency across diverse image conditions, or introducing stronger evaluation of fairness and generalization.

These improvements would increase both the practical quality and the research depth of the system.

18.3 Stronger Quantitative Evaluation

While the project includes meaningful testing and validation, future work could strengthen the evaluation layer through more detailed quantitative analysis.

Examples include:

- measuring average inference time for each AI feature
- reporting upload-to-result latency under different image sizes
- evaluating memory and GPU utilization during processing
- comparing output quality across multiple image categories
- conducting more formal usability studies with structured feedback
- evaluating consistency of results across diverse user inputs

A stronger quantitative evaluation framework would make the project more convincing not only as an engineering system, but also as a research-oriented AI application. It would allow future versions of the system to be compared more systematically and would improve the evidence base for architectural and model-level decisions.

18.4 Production-Grade Scalability Improvements

If Invisio were to evolve beyond an academic prototype into a production-oriented platform, scalability would become a major focus area.

Future work in this direction could include:

- introducing queue-based processing for long-running AI tasks
- separating heavy AI inference workloads into dedicated processing services
- supporting horizontal backend scaling for concurrent users
- integrating resource-aware job scheduling for GPU workloads
- improving monitoring and observability through metrics, tracing, and health checks
- optimizing model loading and reuse for higher throughput

These improvements would make the system better suited for handling larger usage volumes and would reduce the gap between academic implementation and production deployment practices.

18.5 Security and Privacy Enhancements

Although the current system already follows privacy-aware and security-oriented design principles, future work could strengthen this area further.

Possible extensions include:

- **more** formal authentication and authorization mechanisms
- stronger access control policies for optional shared content
- **more** advanced abuse prevention and rate-limiting strategies
- improved auditability of processing events without exposing sensitive data
- **more** explicit user consent interfaces for optional persistence or sharing
- stricter deployment hardening through reverse proxies, secrets management, and infrastructure isolation

Another important future direction is deeper privacy verification. This could include more explicit documentation of temporary data handling, clearer user-facing privacy communication, and stronger distinction between ephemeral data, optional public content, and any future training-related data states.

Such improvements would strengthen the project’s trustworthiness and prepare it more effectively for real-world deployment.

18.6 Improvement of Human-in-the-Loop Interaction

The hybrid human-in-the-loop workflow is one of the defining characteristics of Invisio, and it also represents an important opportunity for future refinement.

Possible improvements include:

- **more** intuitive refinement tools for mask correction
- better visual feedback during refinement
- support for undo/redo during correction stages
- **more** interactive previews before final processing
- adaptive refinement suggestions generated by the system itself
- better alignment between AI-generated masks and user correction expectations

Improving this interaction model would make the platform more effective in the exact area where it differentiates itself from fully automated AI tools. It would also strengthen the project’s contribution to user-centered AI system design.

18.7 Better Support for Diverse Users and Inputs

Another important area for future work is improving the system’s robustness across diverse users, image types, and visual conditions.

This may include:

- testing and improving model behavior across different lighting conditions, backgrounds, and resolutions
- evaluating portrait-related features on more diverse faces and visual characteristics
- improving fairness and consistency across heterogeneous data
- increasing resilience to difficult inputs such as low-quality or partially occluded images

These improvements are important not only from a technical perspective but also from an ethical one. A system that performs well only under limited visual conditions cannot fully meet the expectations of fairness, accessibility, and broad usability.

18.8 Community and Collaboration Features

If the project were to continue evolving, the optional community aspect could also be expanded carefully.

Possible future work includes:

- richer public gallery features
- moderation workflows for shared content
- more detailed privacy controls for public posting
- selective sharing options
- community feedback mechanisms
- stronger separation between private workflow and public publication layers

These features would need to be implemented carefully in order to preserve the project’s privacy-first identity. As such, community expansion should remain opt-in and strongly consent-based.

18.9 Cross-Platform and Product Evolution

Future work may also involve broader product-level evolution beyond the current web-based academic implementation.

Examples include:

- **mobile-friendly** versions of the interface
- **cloud-native** deployment models
- **integration** with external storage or content systems under explicit user control
- **plugin-style** extensibility for new editing modules
- **more polished** user onboarding and guided workflows

Such developments would move the project closer to a user-facing product while preserving the engineering ideas established in the current system.

18.10 Research-Oriented Future Work

From a research perspective, Invisio could also be extended beyond application development into a richer experimental platform.

Potential research-oriented directions include:

- **comparative** evaluation of different AI models for the same editing task
- **user studies** on trust and satisfaction in hybrid editing workflows
- **fairness** analysis across different visual demographics and input categories
- **privacy-aware** AI processing architectures for image-based systems
- **investigation** of how constrained refinement affects perceived control and output quality

These directions would allow the project to contribute not only as a software artifact, but also as a research case in human-centered and responsible AI system design.

18.11 Final Future Work Assessment

Overall, the future of Invisio lies in expanding from a strong academic system into a more scalable, measurable, and polished AI-assisted platform. The current architecture already provides a suitable foundation for this evolution because it was designed with modularity, abstraction, and controlled system boundaries in mind.

For this reason, future work does not require abandoning the existing system vision. Instead, it involves deepening and extending that vision through better models, stronger evaluation, improved interaction design, production-level scalability, and more advanced trust and safety mechanisms.

This makes Invisio a project with meaningful continuation potential rather than a closed one-time implementation.

19. References

References

- [1] Adobe, “Add layer masks,” <https://www.adobe.com/products/photoshop/masking.html>, 2026, adobe Photoshop Help, accessed 2026-04-18.
- [2] GIMP Team, “Gimp documentation,” <https://www.gimp.org/docs/>, 2026, official GIMP documentation, accessed 2026-04-18.
- [3] remove.bg, “What is remove.bg?” <https://www.remove.bg/help/a/what-is-remove-bg-%3F.pict>, 2026, official help page, accessed 2026-04-18.
- [4] —, “How can i use remove.bg?” <https://www.remove.bg/help/a/how-can-i-use-remove-bg.pls>, 2026, official help page, accessed 2026-04-18.
- [5] Canva, “Ai photo editor – instant photo editing with ai,” <https://www.canva.com/features/ai-photo-editing>, 2026, official Canva feature page, accessed 2026-04-18.
- [6] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [8] L. Bass, P. C. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley Professional, 2021.
- [9] OWASP Foundation, “Owasp top 10 api security risks – 2023,” <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>, 2023, official OWASP API Security Top 10, accessed 2026-04-18.
- [10] European Union, “Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 (GDPR),” <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679>, 2016, official EUR-Lex text, accessed 2026-04-18.
- [11] Kişisel Verileri Koruma Kurumu, “Personal data protection law no. 6698,” <https://www.kvkk.gov.tr/Icerik/6649/Personal-Data-Protection-Law>, 2016, official English translation, accessed 2026-04-18.
- [12] E. Tabassi, “Artificial intelligence risk management framework (AI RMF 1.0),” National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, 2023. [Online]. Available: <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10>

- [13] UNESCO, “Recommendation on the ethics of artificial intelligence,” <https://www.unesco.org/en/uuid/node/bdf82f7b-f7f6-48ec-8f52-7ef2240761ce>, 2021, uNESCO recommendation, accessed 2026-04-18.
- [14] European Union, “Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act),” <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>, 2024, official EUR-Lex text, accessed 2026-04-18.
- [15] Meta and React Contributors, “React documentation,” <https://react.dev/>, 2026, official React documentation, accessed 2026-04-18.
- [16] S. Ramírez, “Fastapi documentation,” <https://fastapi.tiangolo.com/>, 2026, official FastAPI documentation, accessed 2026-04-18.
- [17] PyTorch Contributors, “Pytorch documentation,” <https://docs.pytorch.org/docs/main/>, 2026, official PyTorch documentation, accessed 2026-04-18.
- [18] Docker, Inc., “What is docker?” <https://docs.docker.com/engine/docker-overview/>, 2026, official Docker documentation, accessed 2026-04-18.
- [19] PostgreSQL Global Development Group, “Postgresql documentation,” <https://www.postgresql.org/docs/>, 2026, official PostgreSQL documentation, accessed 2026-04-18.
- [20] Tailwind Labs, “Tailwind css documentation,” <https://tailwindcss.com/docs/utility-first>, 2026, official Tailwind CSS documentation, accessed 2026-04-18.
- [21] Vite Contributors, “Vite: Next generation frontend tooling,” <https://vite.dev/>, 2026, official Vite documentation, accessed 2026-04-18.
- [22] Motion Contributors, “Motion for react documentation,” <https://motion.dev/docs/react>, 2026, official Motion documentation, accessed 2026-04-18.
- [23] Microsoft, “Playwright documentation,” <https://playwright.dev/docs/intro>, 2026, official Playwright documentation, accessed 2026-04-18.
- [24] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems 27 (NeurIPS 2014)*, 2014.
- [25] O. Ronneberger, P. Fischer, and T. Brox, “U-net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, ser. LNCS, vol. 9351. Springer, 2015, pp. 234–241.
- [26] L. A. Gatys, A. S. Ecker, and M. Bethge, “A neural algorithm of artistic style,” *arXiv preprint arXiv:1508.06576*, 2015.

- [27] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. A. Efros, “Context encoders: Feature learning by inpainting,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2536–2544.
- [28] C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang, and W. Shi, “Photo-realistic single image super-resolution using a generative adversarial network,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4681–4690.
- [29] N. Diamant, D. Zadok, C. Baskin, E. Schwartz, and A. M. Bronstein, “Beholder-GAN: Generation and beautification of facial images with conditioning on their beauty level,” in *2019 IEEE International Conference on Image Processing (ICIP)*, 2019.
- [30] Y. Y. Boykov and M. Jolly, “Interactive graph cuts for optimal boundary & region segmentation of objects in N-D images,” in *Proceedings of the Eighth IEEE International Conference on Computer Vision (ICCV)*, vol. 1, 2001, pp. 105–112.
- [31] C. Rother, V. Kolmogorov, and A. Blake, “GrabCut: Interactive foreground extraction using iterated graph cuts,” *ACM Transactions on Graphics*, vol. 23, no. 3, pp. 309–314, 2004.
- [32] N. Xu, B. Price, S. Cohen, J. Yang, and T. S. Huang, “Deep interactive object selection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 373–381.
- [33] K. Maninis, S. Caelles, J. Pont-Tuset, and L. V. Gool, “Deep extreme cut: From extreme points to object segmentation,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 616–625.
- [34] T. Portenier, Q. Hu, A. Szabó, S. A. Bigdeli, P. Favaro, and M. Zwicker, “Faceshop: Deep sketch-based face image editing,” vol. 37, no. 4, 2018.
- [35] Y. Jo and J. Park, “SC-FEGAN: Face editing generative adversarial network with user’s sketch and color,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1745–1753.
- [36] Y. Jing, Y. Yang, Z. Feng, J. Ye, Y. Yu, and M. Song, “Neural style transfer: A review,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 26, no. 11, pp. 3365–3385, 2020.
- [37] C. Lee, Z. Liu, L. Wu, and P. Luo, “MaskGAN: Towards diverse and interactive facial image manipulation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 5549–5558.

- [38] S. Anwar, M. Tahir, C. Li, A. Mian, F. S. Khan, and A. W. Muzaffar, “Image colorization: A survey and dataset,” *arXiv preprint arXiv:2008.10774*, 2020.
- [39] P. Shamsolmoali, M. Zareapoor, E. Granger, H. Zhou, R. Wang, M. E. Celebi, and J. Yang, “Image synthesis with adversarial networks: A comprehensive survey and case studies,” *Information Fusion*, vol. 72, pp. 126–146, 2021.
- [40] H. Ling, K. Kreis, D. Li, S. W. Kim, A. Torralba, and S. Fidler, “EditGAN: High-precision semantic image editing,” in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, 2021.
- [41] K. Sofiuk, I. A. Petrov, and A. Konushin, “Reviving iterative training with mask guidance for interactive segmentation,” in *2022 IEEE International Conference on Image Processing (ICIP)*, 2022, pp. 3141–3145.
- [42] X. Chen, Z. Zhao, Y. Zhang, M. Duan, D. Qi, and H. Zhao, “Focalclick: Towards practical interactive image segmentation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 1300–1309.
- [43] J. Gui, Z. Sun, Y. Wen, D. Tao, and J. Ye, “A review on generative adversarial networks: Algorithms, theory, and applications,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 4, pp. 3313–3332, 2023.
- [44] J. Gan and J. Liu, “Applied research on face image beautification based on a generative adversarial network,” *Electronics*, vol. 13, no. 23, p. 4780, 2024.
- [45] TO BE FILLED, “Survey on Image Colorization,” *Electronics*, 2024, placeholder entry. Exact bibliographic information could not be verified from the provided citation text.
- [46] T. Peng, Z. Wu, Y. Zhou, J. Zhu, and L. Lin, “ISFB-GAN: Interpretable semantic face beautification with generative adversarial network,” *Expert Systems with Applications*, vol. 236, p. 121131, 2024.
- [47] X. Wu, K. Xu, and P. Hall, “A survey of image synthesis and editing with generative adversarial networks,” *Tsinghua Science and Technology*, vol. 22, no. 6, pp. 660–674, 2017.
- [48] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, “Gans trained by a two time-scale update rule converge to a local nash equilibrium,” in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017.
- [49] Y. Xu, M. Xia, K. Hu, S. Zhou, and L. Weng, “Style transfer review: Traditional machine learning to deep learning,” *Information*, vol. 16, no. 2, p. 157, 2025.

- [50] S. Zhang, Y. Qi, and J. Wu, “Applying deep learning for style transfer in digital art: Enhancing creative expression through neural networks,” *Scientific Reports*, vol. 15, p. 11744, 2025.
- [51] TO BE CONFIRMED, “Bridging the metrics gap in image style transfer: A comprehensive survey of models and criteria,” *Neurocomputing*, vol. 624, p. 129430, 2025, author list should be added from the final published version.
- [52] Z. Sordo, E. Chagnon, and D. Ushizima, “A review on generative AI for text-to-image and image-to-image generation and implications to scientific images,” *arXiv preprint arXiv:2502.21151*, 2025.

20. Appendix

20.1 Appendix A

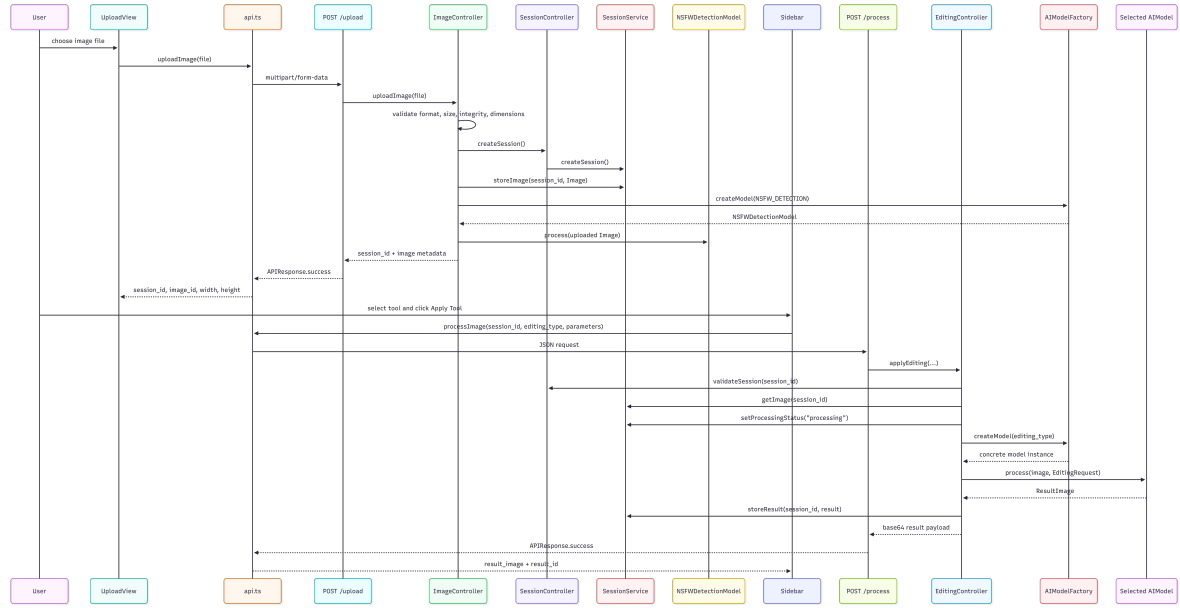


Figure 24: Sequence Diagram – Upload and Generic Processing Request

20.2 Appendix B

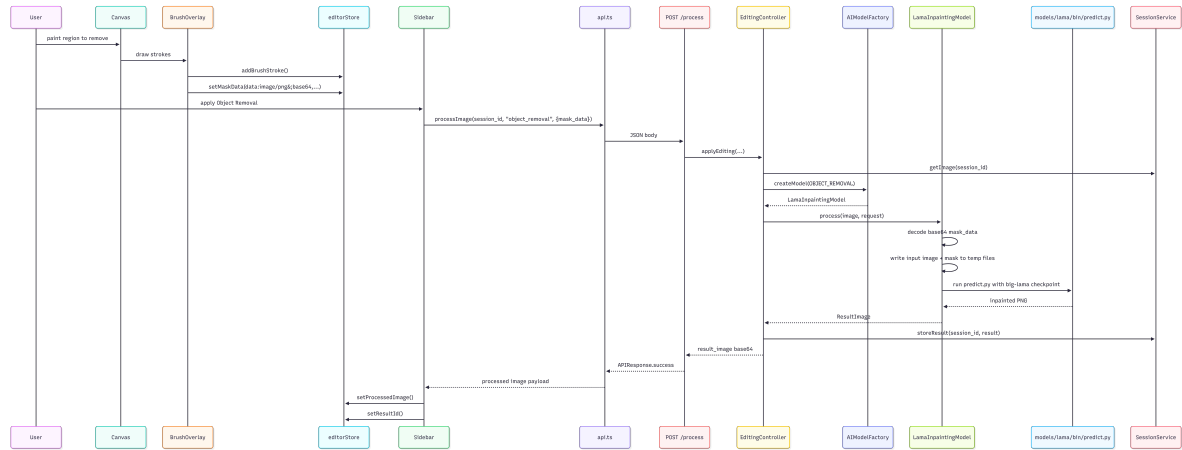


Figure 25: Sequence Diagram – Object Removal Flow

20.3 Appendix C

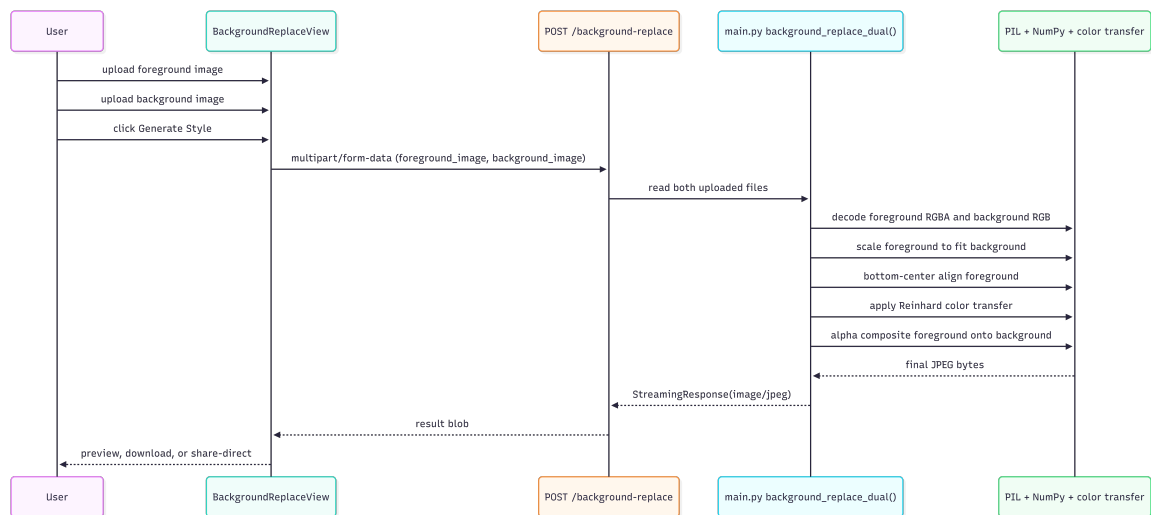


Figure 26: Sequence Diagram – Standalone Dual-Image Background Replacement Flow

20.4 Appendix D

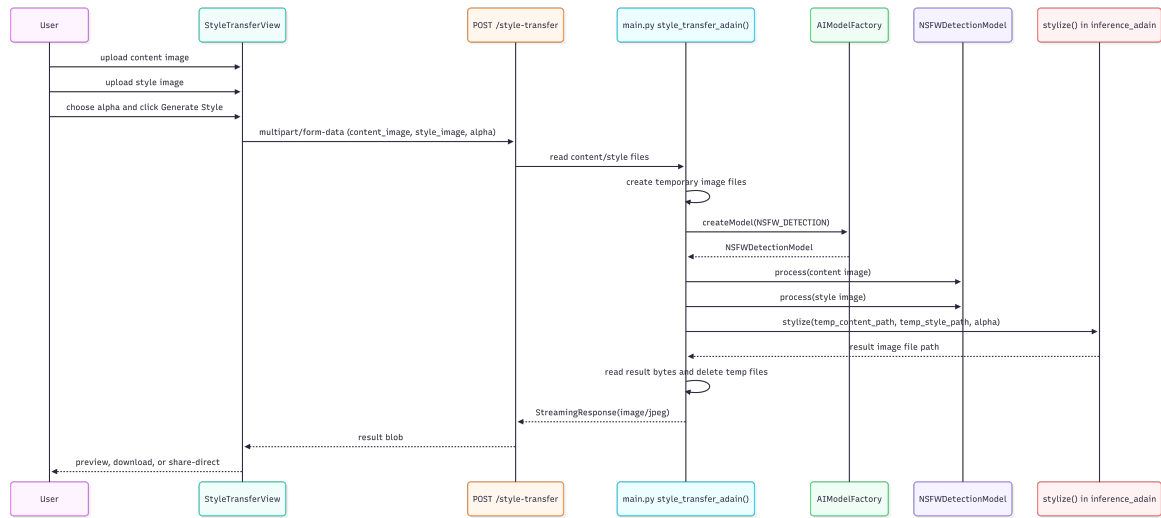


Figure 27: Sequence Diagram – Standalone Style Transfer Flow

20.5 Appendix E

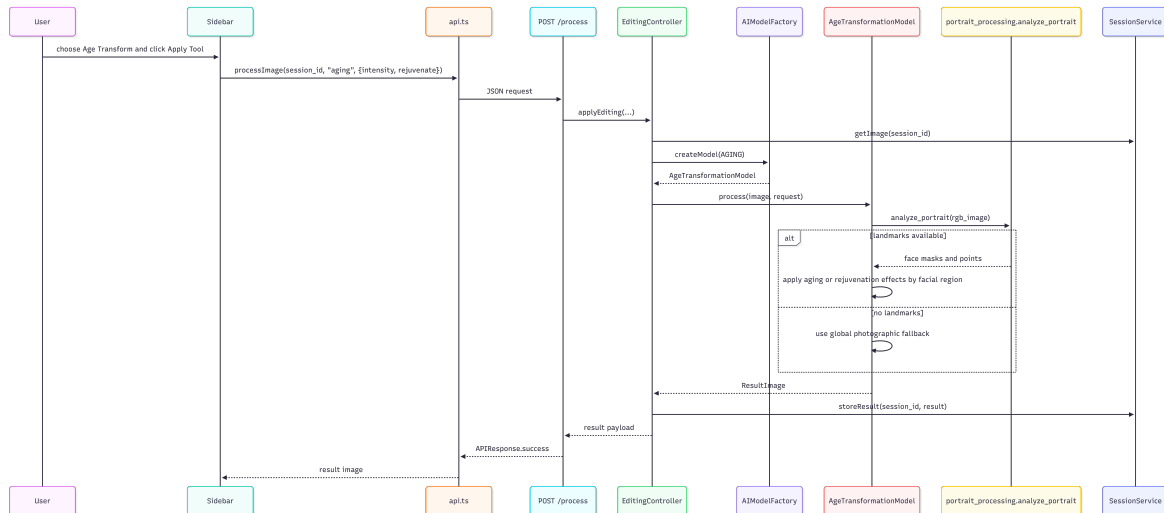


Figure 28: Sequence Diagram – Aging Flow

20.6 Appendix F

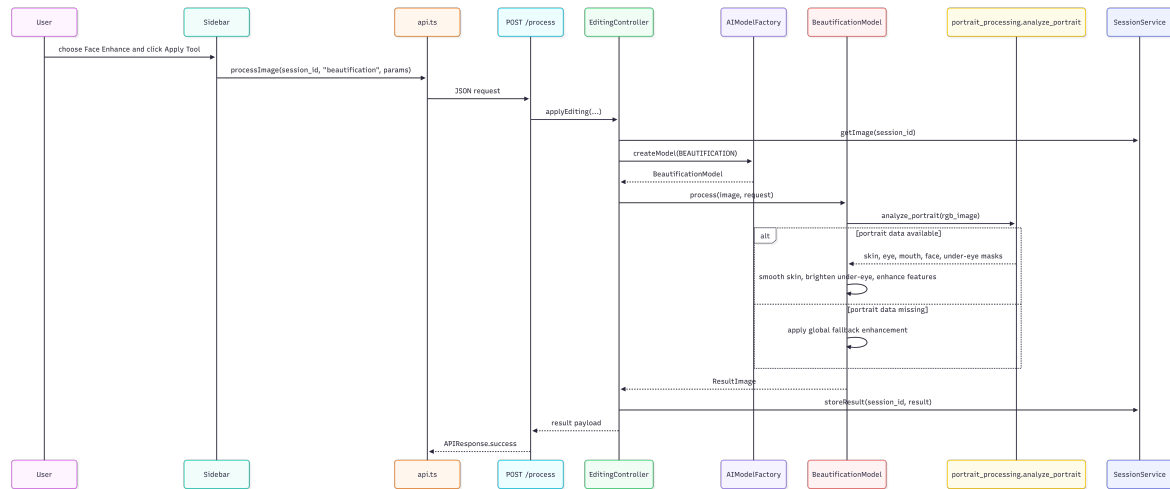


Figure 29: Sequence Diagram – Beautification Flow

20.7 Appendix G

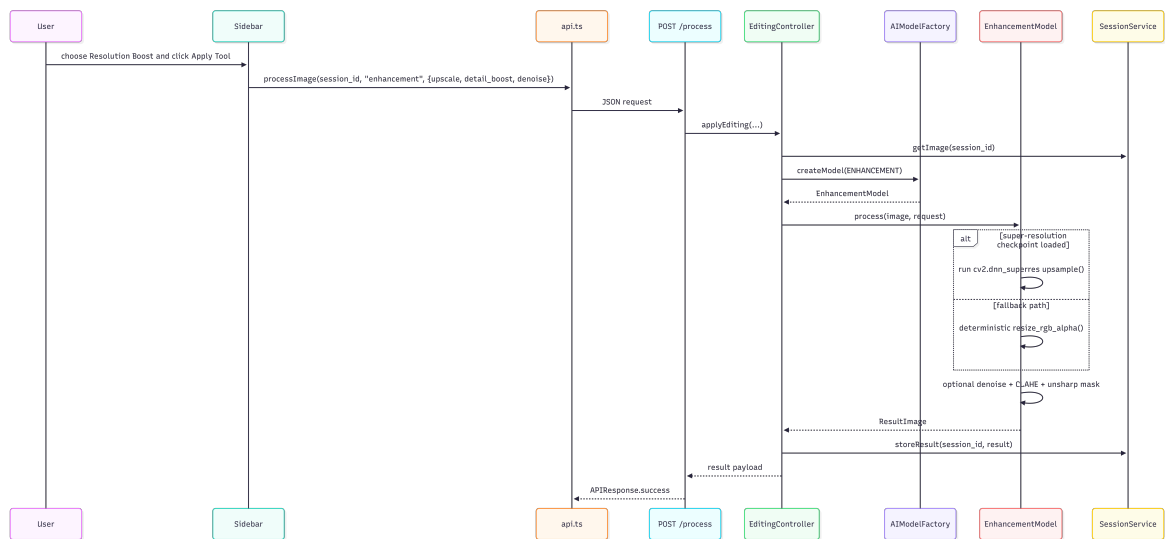


Figure 30: Sequence Diagram – Resolution Enhancement Flow

20.8 Appendix H

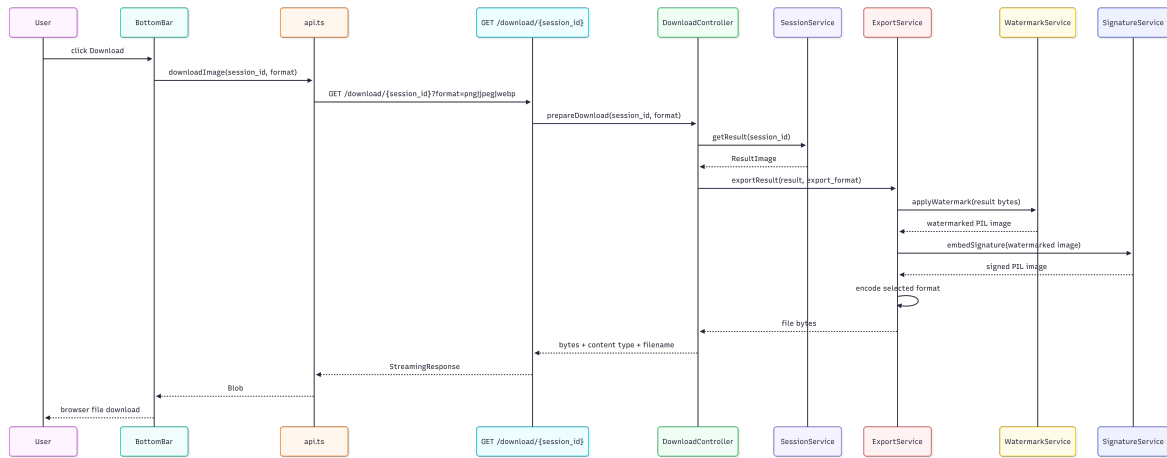


Figure 31: Sequence Diagram – Final Download and Export Flow

20.9 Appendix I

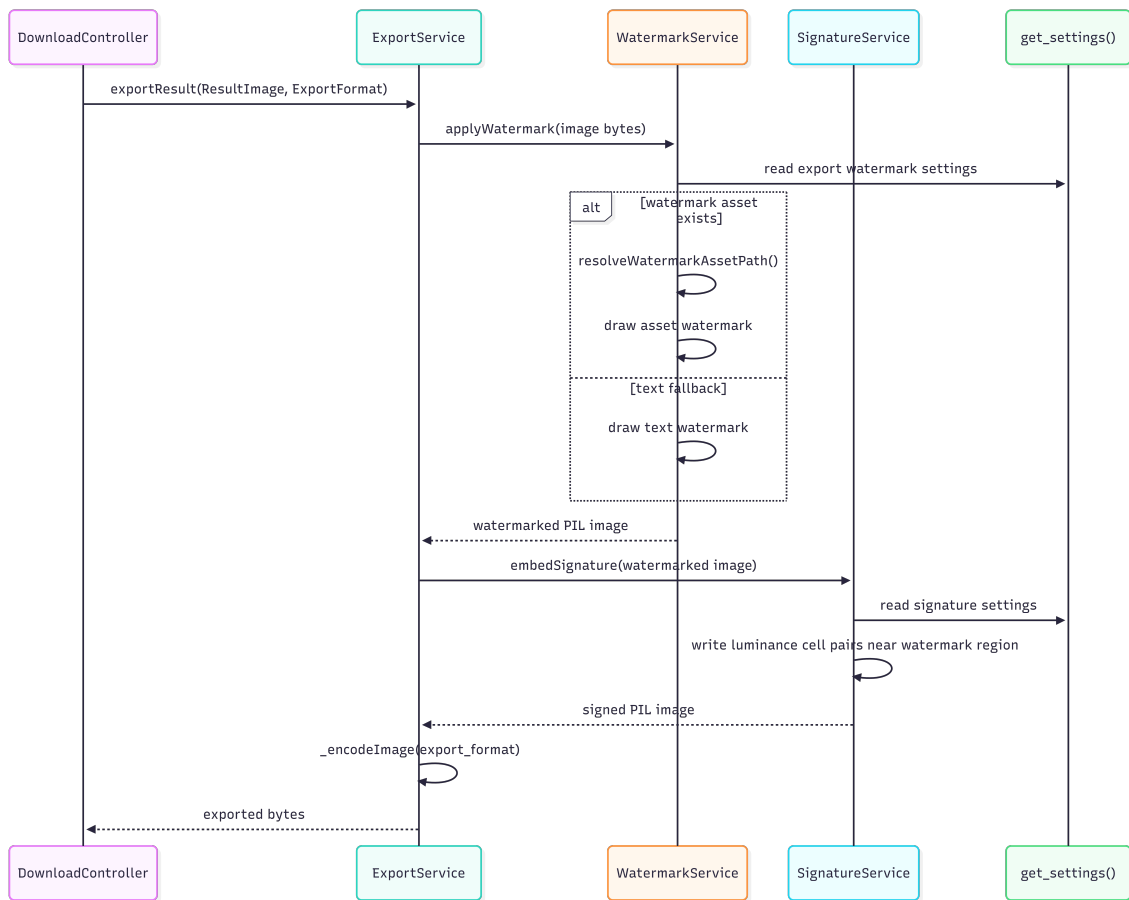


Figure 32: Sequence Diagram – Watermark Application Flow

20.10 Appendix J

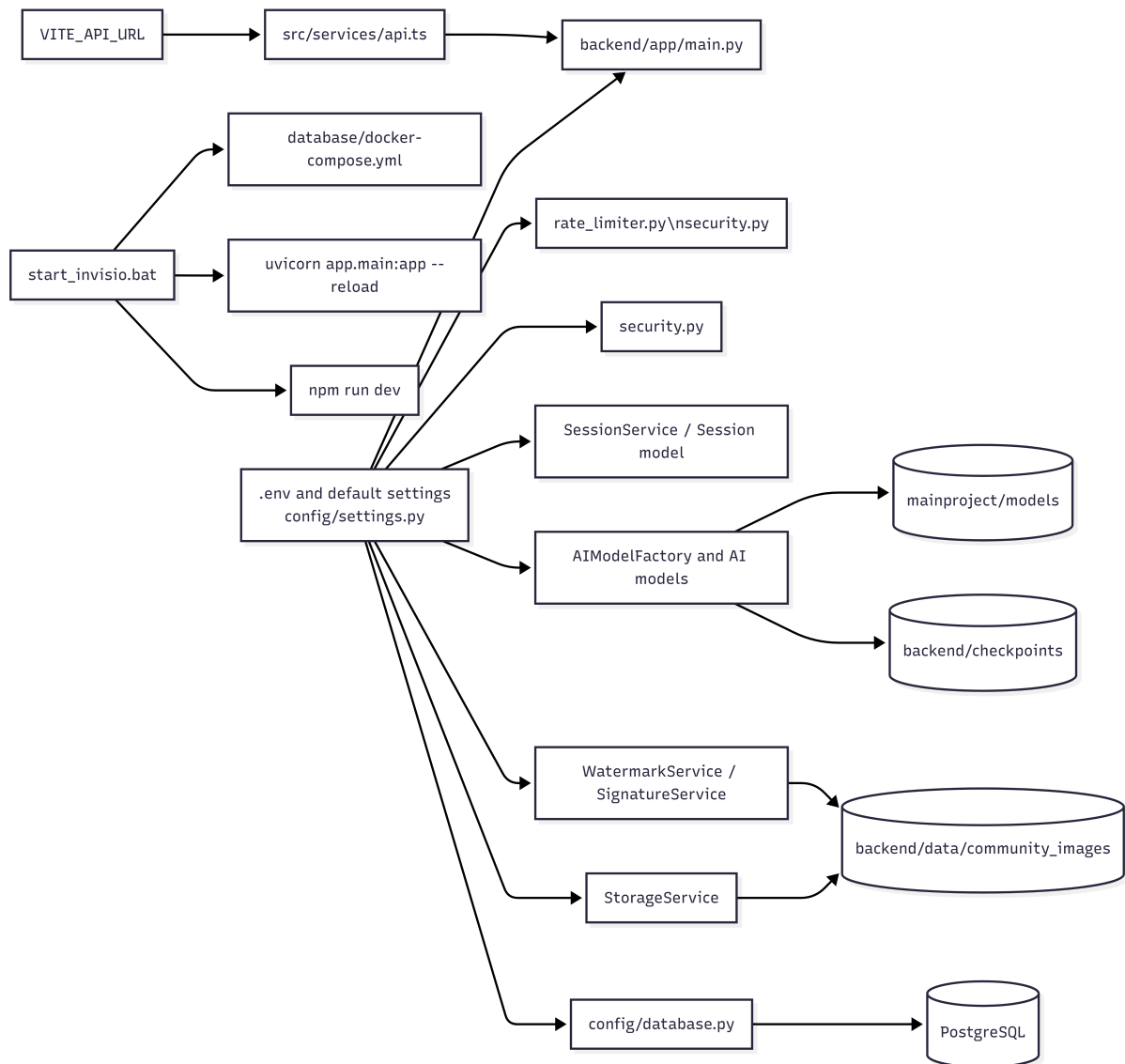


Figure 33: Configuration Dependency Diagram of the Invisio Implementation Environment

20.11 Appendix K

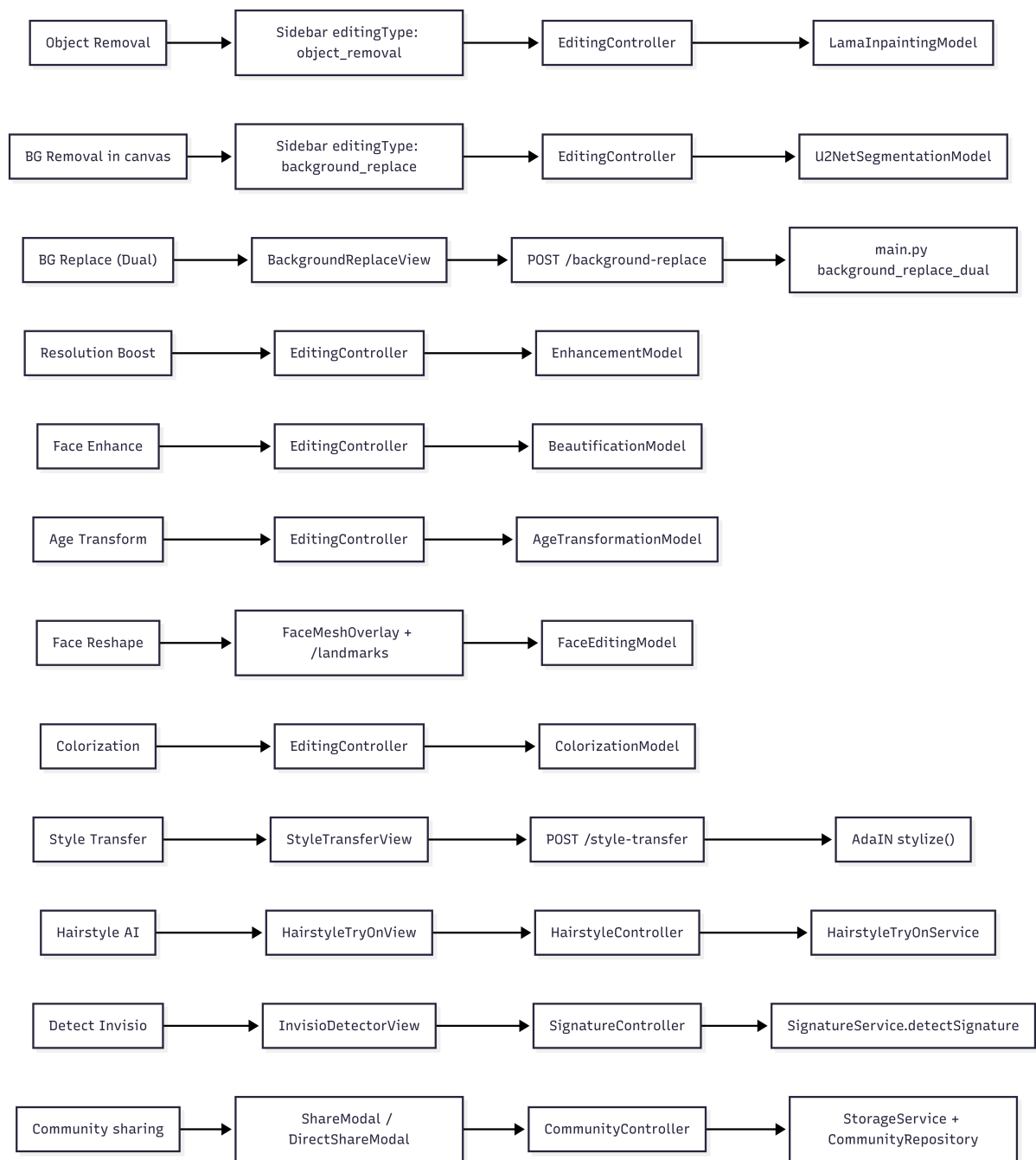


Figure 34: Feature-to-Implementation Mapping in the Invisio System

20.12 Appendix L

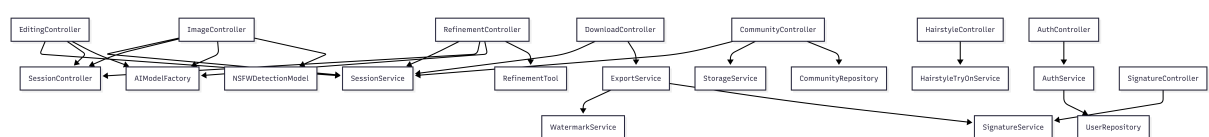


Figure 35: Service Interaction Diagram of the Invisio System